

软件工程题库考试复习题

目 录

一、选择题 (28 题)	7
二、名词解释 (100 题)	13
1. 语句覆盖	13
2. 软件	13
3. 项目	13
4. 通信聚合	13
5. 分布图	13
6. 判定覆盖	13
7. 软件工程	13
8. 配置管理	13
9. 信息聚合 (未找到答案)	13
10. 组件图	14
11. 条件覆盖	14
12. 软件危机	14
13. 配置项	14
14. 数据聚合 (未找到答案)	14
15. 活动图	14
16. 路径覆盖	14
17. 软件生存周期	14
18. 基线	14
19. 控制耦合	14
20. 协作图	15
21. 条件组合覆盖	15
22. 软件过程	15
23. 里程碑	15
24. 标记耦合	15
25. 时序图	15
26. 等价类划分	15
27. 软件基本过程	15
28. 项目范围管理	16
29. 数据耦合	16
30. 状态图	16
31. 边界值测试	16
32. 软件支持过程	16
33. 项目整体管理	16
34. 预防性维护	16
35. 对象图	16
36. 基本路径测试	16
37. 软件组织过程	17
38. 软件度量	17
39. 适应性维护	17
40. 类图	17
41. 黑盒测试	17
42. 过程框架	17
43. 功能点分析	17
44. 结构化程序设计	17

45.用例图.....	17
46.白盒测试.....	18
47.软件能力成熟度模型.....	18
48.工作分解结构.....	18
49.非功能性需求.....	18
50.统一建模语言 UML.....	18
51.单元测试.....	18
52.个体软件过程.....	18
53.COCOMO 模型.....	18
54.信息隐蔽.....	18
55.面向对象.....	19
56.集成测试.....	19
57.团队软件过程.....	19
58.项目计划评审技术.....	19
59.内聚.....	19
60.主动对象.....	19
61.确认测试.....	19
62.过程模式（未找到答案）.....	19
63.RMMM 计划（Risk Mitigation, Monitoring and Management Plan）.....	20
64.耦合.....	20
65.多态性.....	20
66.系统测试.....	20
67.统一过程.....	20
68.软件质量.....	20
69.体系结构.....	20
70.消息.....	20
71.压力测试.....	21
72.瀑布模型.....	21
73.结构复杂性度量（未找到答案）.....	21
74.决策表.....	21
75.继承.....	21
76.测试配置.....	21
77.快速原型模型.....	21
78.软件复杂性度量.....	22
79.数据字典.....	22
80.封装.....	22
81.静态测试.....	22
82.增量模型.....	22
83.软件可靠性.....	22
84.上下文数据流图.....	22
85.对象/类.....	22
86.动态测试.....	23
87.螺旋模型.....	23
88.错误播种（植入）模型.....	23
89.数据流图.....	23
90.静态视图.....	23
91.Beta 测试.....	23
92.极限编程.....	23
93.软件质量模型（未找到答案）.....	24
94.业务流程图.....	24

95.动态视图（未找到答案）	24
96.因果图.....	24
97.过程规范（未找到答案）	24
98.基于时间的缺陷到达模式（未找到答案）	24
99.过程模型.....	24
100.领域分析.....	25
三、简答题（100 题）	25
1.简述软件安全性与软件可靠性的异同。	25
2.简述软件开发的可行性研究的目的和内容。	25
3.什么是面向数据流的设计，数据流包括哪几种？	25
4.什么时候测试结束，为什么？	25
5.什么是对象，对象有哪几种形式？	26
6.什么是 McCabe 复杂性度量，如何计算？	26
7.软件项目管理的主要方面是什么？	26
8.怎样完成从数据流图到程序结构图的转换？	26
9.如何估算程序中的错误？	26
10.什么是类，类与对象间是什么关系？	27
11.软件的缺陷为什么在软件开发和维护过程中会扩大？	27
12.成本估算方法有哪几种？	27
13.面向数据流的设计方法包含那些步骤？	27
14.什么叫测试用例，如何设计测试用例.....	27
15.面向对象方法特征有哪些？	27
16.什么是 CMM 关键过程域，举例说明其含义。	28
17.影响软件成本的主要因素有哪些具体方面？	28
18.画数据流图原则是什么？	28
19.什么叫基本路径测试，如何确定基本路径？	28
20.简述 USE/CASE 的作用。	28
21.软件质量的特性有哪些，怎样度量？	29
22.什么是 COCOMO 成本估算模型？	29
23.数据流图作用是什么？	29
24.简述回溯测试和作用。	29
25.面向对象设计涉及那几个主要活动？	30
26.Boehm 和 McCall 等人从哪些方面来确定软件质量，他们有什么异同？	30
27.什么是软件的逆向工程和再工程？	30
28.简述数据字典的作用.....	30
29.纠错技术中的归纳法和演绎法的中心思想是什么？	30
30.面向对象设计如何体现抽象信息、隐藏和模块化这三个概念的？	31
31.影响软件质量的因素分哪两大类？	31
32.如何衡量软件成本，各表示什么含义？	31
33.需求分析的任务是什么？	31
34.什么是因果图，用于何处？	32
35.举例说明什么是类的整体部分结构.....	32
36.软件质量的含义是什么？	32
37.如何安排软件开发进度，应该考虑哪些问题？	32
38.什么是结构化分析，有什么特点？	32
39.什么是边界值分析，与等价类划分有什么不同？	32
40.举例说明什么是类的组装结构关系.....	33
41.原型开发由哪些步骤？	33
42.什么是软件配置管理，有什么作用？	33
43.原型开发技术的目的，特点是什么？	33

44.简述在确定等价划分测试用例设计中遵循的原则。.....	33
45.领域分析的目标是什么，依据是什么？.....	34
46.原型开发包括那几类，目的是什么？.....	34
47.基线在配置管理中有什么作用？.....	34
48.简述总体设计的一般过程？.....	34
49.什么是软件测试中逻辑覆盖，一般软件测试至少应保证哪些覆盖？.....	34
50.类的开发有几个途径，如何进行类的开发？.....	34
51.原型具有什么特点？.....	34
52.软件维护的基本内容是什么？.....	35
53.解释体系结构中深度、宽度、扇出、扇入对软件的影响。.....	35
54.为什么说在确认测试阶段发现需求理解的错误，要修改就要付出更大的代价？.....	35
55.什么是 OOA 建模语言，它应该包括哪些方面？.....	35
56.为什么要进行阶段评审？.....	36
57.软件维护分几类，每类的内容是什么？.....	36
58.什么叫模块化，为什么要划分模块？.....	36
59.比较在组装测试中，自顶向下与自底向上、深度优先与宽度优先的优缺点。.....	36
60.简述 RUP (Rational Unified Process) 的基本内容.....	37
RUP (Rational Unified Process, 统一软件开发过程, 统一软件过程) 是一个面向对象且基于网络的程序开发方法论。.....	
内容：迭代式开发、管理需求、体系结构、可视化建模、验证软件质量、控制软件变更	
61.简述模型在软件开发中的作用。.....	37
62.软件维护有哪些特点？.....	37
63.模块化的三个重要特征是什么，阐明各自的作用。.....	38
64.单元测试时，为什么需要开发驱动模块和桩模块？.....	38
65.什么是 Use Cases Model，主要作用是什么？.....	38
66.什么是需求工程？.....	38
67.简述软件的维护过程。.....	38
68.简述层次方框图与软件结构图的异同点。.....	39
69.简述黑盒子测试和白盒子测试及其适用性。.....	39
70.什么是 RUP (Rational Unified Process) 的 4+1 视图？.....	39
71.简述软件分析员在系统分析中的任务和作用。.....	40
72.软件的可维护性与哪些因素有关，如何提高软件的可维护性？.....	40
73.事务型软件结构图有什么特点，原因是什么？.....	40
74.单元测试、组装测试和确认测试各自主要的目标是什么，相互有什么关系？.....	40
75.Coad/Yourdon 的 OOA 模型包括那几个方面？.....	41
76.面向对象生存期模型与传统的生存期模型有什么区别？.....	41
77.软件工程的文档分哪两大类，主要作用是什么？.....	41
78.详细设计的任务是什么？.....	42
79.为什么把软件测试的目标定义为只是发现错误？.....	42
80.什么是对象关系模型和对象行为模型，有什么不同？.....	42
81.软件工程是如何克服软件危机的？.....	42
82.软件开发主要由哪些方面风险，如何进行软件风险分析？.....	43
83.程序的编码风格主要体现在哪几个方面？.....	43
84.简述软件测试中桩模块和驱动模块的作用。.....	43
85.如何从需求文档中识别类和对象.....	44
86.软件危机主要有哪些表现？.....	44
87.什么是软件维护的副作用，如何防止软件维护的副作用？.....	44
88.好的软件体系结构设计应遵循哪些原则？.....	44
89.简述渐增式测试方法与非渐增式测试方法的优劣。.....	44
90.UML 中有哪些动态建模的工具，各表达什么内容？.....	45

91.软件技术审查和管理复审的作用是什么？	45
92.软件的可维护性与那些软件质量特性有关？	45
93.什么是模块独立性，如何衡量模块独立性？	45
94.软件测试有几种方法，每种方法的特点是什么？	46
95.UML 中的接口类有什么特点，举例说明。	46
96.常见的软件开发模型有那些，各有什么特点？	47
97.CMM 与 ISO9000 有什么异同。	47
98.什么是耦合，如何衡量模块的耦合度？	47
99.软件测试分几种类型，主要解决什么问题？	47
100.简要说明 RUP 中阶段、核心 workflow 和迭代的关系。	48
四、论述题（20 题）	48
1.论述面向对象方法对传统方法的优势和问题	48
2.什么是面向对象开发过程，讨论各阶段任务和要点	48
3.用覆盖的观点讨论面向对象的软件测试策略	48
4.结合软件工程要素，论述面向对象方法的思想	49
5.论述需求工程过程，说明各阶段关系	49
6.什么是软件可维护性，讨论软件工程与可维护性关系	49
7.论述软件工程中的文档类型和作用	50
8.论述需求工程的过程和方法，讨论其有效性	50
9.讨论软件测试与软件可靠性的关系，如何计算可靠性	50
10.什么是软件可测试性，讨论软件工程与软件可测试性	51
11.论述软件测试的基本任务，方法和策略及其在现代软件工程的地位	51
12.论述软件测试与软件质量的关系	51
13.讨论不同软件过程模型与软件开发的关系	52
14.针对敏捷方法的十二条核心实践，结合软件工程问题论述其有效性	54
15.针对软件危机，论述传统软件工程方法和敏捷方法的解决思路	54
16.论述过程、方法和工具在软件工程实践中的关系	55
17.针对不同软件方法，论述项目管理要点	55
18.论述软件项目特点和管理要点	56
19.论述软件估算的困难和解决方法	57
20.什么是软件性能，论述软件开发中如何提高软件的性能	57
五、应用题（20 题）	58
1.假设一家工厂的采购部每天需要一张定货报表，报表按零件编号排序，表中列出所有需要再次定货的零件。对于每个需要再次定货的零件应该列出下述数据：零件编号，零件名称，定货数量，当前价格，主要供应者，次要供应者。零件入库或出库称为事务，通过放在仓库中的显示终端把事务报告给定货系统。当某种零件的库存数量少于库存量临界值时就应该再次定货。	58
2.现在有一个医院病房监护系统，用户提出的系统功能要求如下：	60
3.连锁超市进货操作流程为：厂家持“定货单”及“厂家送货单”送货，库管科检查货物。	63
4.设计一个简单的学生选课系统，系统能够注册符合要求的学生，学生注册以后就能选课。选课过程如下：	64
5.某铁路售票系统，其相关角色可简化为乘客、售票员和管理人员。业务过程如下：	66
6.某个银行的存取款业务处理系统有以下功能：	68
7.某公司准备开发公共汽车刷卡计费系统，具体需求如下：	69
8.现在有一系统正在进行集成测试，模块 A 提供的文件接口能够读取 txt 格式的文本文件，不论该文件是否被其它进程使用。请补充完成如下测试用例中的 (1)、(2)、(3) 和 (4) 部分。	70
9.依据下面给出的 N—S 图画出其流程图，在此基础上设计最少测试用例，实现语句覆盖和条件组合覆盖。	71
10.有一款 8 位微机，其十六进制常数定义为：以 0x 或 0X 开头的数是十六进制整数，其值的范围是 -7f~7f (表示十六进制的大小写字母不加区别)，如 0X13, 0X6A, -0X3c。根据上述条件使用等价划分法设计测试用例。	73
11.下面是某程序的流程图	73
12.某网站开发新的邮件系统供用户使用，现需要做系统测试。请以“输入正确的用户邮件地址和错误的密码”为例，完成如下的测试用例中 (1)、(2)、(3) 和 (4) 部分。	74

13. 下列是一段伪码程序: 74
14. 图书馆管理系统中, 读者包括教师、学生 (学生中有本科生、研究生、) 等等, 图书包括杂志、各类书籍, 请用 UML 建立 UseCase 图 (5 分), 读者和图书的类图 (10 分), 图书的状态图 (5 分), 以及借还书过程的活动图 (10 分)。 76
15. 由键盘输入一个足够大的数, 判断该数是否为素数 (除一和本身外不能被其他数整除)。 78
16. 某考务处理系统具有以下功能: 81
17. 某房屋租赁服务系统主要功能如下: 系统对出租的房屋信息、房主信息、需要租房的人员信息, 以及租房人和房主的会面情况进行管理和维护。 81
18. 某医院病人住院与治疗结帐业务处理过程及说明如下: 82
19. 某制造企业的物料出入库管理的工作流程叙述如下: 83
20. 某大学校园一卡通管理系统包括以下功能: 84

主要题型和重点

- 一、选择题 20 分
- 二、名词解释 5 题*3 分=15 分 题库共 100 道
- 三、简答题 4 题*5 分=20 分
- 四、论述 1 题*15 分=15 分
- 五、应用 30 分

软件测试:

- 1、白盒测试: 基本路径 (画流程图、关键路径、测试用例);
- 2、决策表、因果图 (表述方式、列出条件、设计用例)

建模:

- 1、传统结构: 数据流程图 (怎么画)、数据制表等;
- 2、UML、面向对象简答应用 (use case、类图、状态图、交互图: 了解) 表述、目的、数据字典

软件工程与方法复习

一、选择题 (28 题)

1、软件生存期模型是从软件项目需求定义开始到软件被废弃使用为止, 跨越整个生存期的系统开发、运行和维护所实施的全部过程、活动和任务的结构框架。到目前为止, 存在的软件生存期模型有: 演化模型, 螺旋模型, 智能模型, 喷泉模型, 瀑布模型 等。

瀑布模型 螺旋模型 快速原型模型 增量模型 喷泉模型 统一过程 快速原型模型 快速原型

2、软件需求分析方法包括原型化方法和结构分析方法。软件原型化方法是在研究分析阶段的方法和技术中产生的, 但是也可用以面向软件开发的其他阶段。由于软件项目的特点和运行原形的目的的不同, 原型主要有三种不同的作用类型: 探索型, 实验型, 进化型。

探索型 的目的是要弄清目标系统的需求, 确定所希望的特性, 研究多种方案的可行性。它主要针对开发目标模糊, 用户和开发者对项目都缺乏经验的情况。

实验型 的目的用于大规模开发和实现之前, 考核方案是否合适, 规格说明书是否可靠。

进化型 的目的不在于改进规格说明, 而是将系统建造的易于变化, 在改进原型的过程中, 逐步将原型变成最终系统。它将原型方法的思想扩展到软件开发的全过程, 适合于满足需求的变动。

3、由于运用原型的目的和方式不同, 在使用原型时可采用以下两种不同的策略:

(1)、废弃策略：先构造一个功能简单而且质量要求不高的模型系统，针对这个模型系统反复进行分析修改，形成比较好的设计思想，据此设计出完整、准确、一致、可靠的最终系统，系统构造完成后，原来的模型被废弃不用。它对应于 探索型 和 实验型。

(2)、追加策略：先构造一个功能简单而且质量要求不高的模型系统作为最终系统的核心，然后不断扩充修改，逐步追加新的要求，最后成为最终的系统。它对应于 进化型。

3、在软件工程的设计阶段中，有三种常用的设计方法：结构化设计方法 SD、Jackson 方法和 Parnas 方法。SD 方法侧重于 用数据流图表示系统的分解，且用数据字典和说明分别表示数据和加工的含义；Jackson 方法侧重于 由数据结构导出模块结构；Parnas 方法的主要思想 将可能引起变化的因素隐藏在某有关模块内部，是这些因素变化时的影响范围受到限制。从七十年代中期到九十年代早期，SD 是最为常用的方法。Parnas 方法只提供了重要的设计准则，没有规定出具体的工作步骤。

4、程序的正确性证明是指 用逻辑方法证明程序终止且满足给定的程序规范，常用的正确性证明方法之一为 Hoare 方法。在程序中插入的逻辑公式称为 Hoare 公式。当 Hoare 公式 出现在循环中，并且它具有如下性质：只要第一次进入选换时 Hoare 公式 为真，那么第 N 次进入循环时 Hoare 公式 仍为真。此时，这样的 Hoare 公式 就称为 断言。如果公式 E 在执行程序段“Z:=Z*A; Y:=Y-1”之前为真，则执行该程序段之后 E 仍保持为真。

A: ① 给它一组数据，检验其执行结果是否正确

② 用逻辑方法证明程序终止且满足给定的程序规范

③ 逐条语句检查程序是否有错

④ 证明程序终止

⑤ 证明程序满足给定的程序规范

B: ① SD

② Jackson

③ Hoare

④ Parnas

C, D: ① Horn 子句

② 循环不变式

③ 断言

④ 验证条件

⑤ Hoare 公式

E: ① $A*Z = Y$

② $A*Y = Z$

③ $Z=AY$

④ $AB = Z* AY$

⑤ $Y > 0 \wedge Z = AY$

5、软件语言主要包括需求定义语言、功能性语言、设计性语言、程序设计性语言和文档语言等。功能性语言用来书写软件的功能规约，如 Z 语言。函数式程序设计语言和逻辑式程序设计语言都属于 申述式 语言。逻辑式语言的基础是 谓词逻辑，如 PROLOG 是逻辑式程序设计语言，它的基本运算单位是 Horn 子句。函数式语言是一种面向 值 的语言。

6、软件方法学是以软件方法为研究对象的学科。

从开发风范上看，可分为 自顶向下的开发方法与自底向上的开发方法。

从性质上看，可分为 形式方法与非形式方法。

从适应范围来看，可分为 整体性方法与局部性方法。

形式方法的目的是把软件作为数学来重新发现。形式方法被用来避免系统中的 歧义性、不完全性、不一致性。

软件自动化方法是指利用计算机使软件的设计实现自动化的方法和相关的技术。软件自动化的实现途径有四种：过程途径、归纳途径、演绎途径、转换途径。

7、开发软件时，对于提高软件开发人员效率至关重要是 程序开发环境。软件工程中描述软件生存周期的瀑布模型一般包括计划、需求分析、设计、编码、测试、维护等几个阶段。其中，设计阶段在管理上可以分成 概要设计 和 详细设计 两个步骤。

8、软件需求分析的任务不应包括 结构化程序设计。进行需求分析可使用多种工具，但 PAD 图 是不适用的。

在需求分析中, 分析员要从用户那里解决的最重要的问题是 要让软件做什么。需求规格说明书的内容不应当包括 对算法的详细过程的描述。该文档在软件开发中具有重要的作用, 但其作用不应当包括 软件可行性分析的依据。

9、**块间联系和块内联系是评价程序结构质量的重要标准**。联系的方式、共用信息的作用、共用信息的数量和接口的 简单性 等因素决定了块间联系的大小。在块内联系中, 功能内聚 的块内联系最强。SD 方法的总原则是使每个模块执行 一个 功能, 模块间传递 数据型 参数, 模块通过 标准引用 语句调用其他模块。

10、1960 年底 Dijkstra 提倡的 结构化程序设计 是一种有效的提高程序设计效率的方法。Dijkstra 为了使程序结构易于理解, 把基本控制结构限于顺序、选择、重复 3 种, 应避免使用 GOTO 语句。结构化程序设计 不仅提高程序设计的生产率, 同时也容易进行程序的 维护。

11、程序的 3 种基本控制是 顺序, 条件, 循环。它们的共同点是 只有一个入口和一个出口。结构化程序设计的一种基本方法是 逐步求精法。软件测试的目的是 发现程序中的错误。软件调试的目的是 找出错误所在并改正之。

12、在面向对象软件开发过程中特别重视复用。软件构件应独立于当初开发它们的应用而存在。在以后的应用开发中, 可以调整这些独立构件以适应新问题的需要。因此, 应使得类成为一个 可复用 的单元。这样就有一个 类 生存期问题。类 生存期有自己的步骤, 与任何特定应用的开发 无关。按照这些步骤, 可以完整地描述一个基本 实体, 而不仅仅考虑当前正在开发的系统。系统开发的各个阶段都可能会标识新的类。随着各个新类的标识, 类 生存期引导开发工作逐个阶段循序渐进。在设计和实现类时, 应尽可能利用既存类提供为当前应用所需要的功能, 利用既存类的 3 个可能途径是: 演化 复用既存类, 对既存类进行修改以得到满足要求的类, 重新开始进行开发。

13、测试大型软件通常由 单元测试、集成测试、确认 Validation 测试组成。确认测试主要寻找与软件 需求规格 说明不一致的错误。语句覆盖、判定覆盖、条件覆盖和路径覆盖都是白盒测试法设计测试用例的覆盖准则, 在这些覆盖准则中最弱的准则是 语句覆盖, 最强的准则是 路径覆盖。此外, 还有多种黑盒测试的设计测试用例方法, 如 因果图。

14、如何评价软件的质量一直是软件技术人员所关心的问题, 目前已有多种软件质量模型来描述软件的质量特性。ISO/IEC 9126 是国际标准化组织在 1991 年提出的软件质量标准。它由三个层次组成, 第一层是质量特性, 第二层是质量子特性, 第三层是度量指标。六个质量特性是: 功能性、可靠性、易使用性、效率、可维护性和 可移植性。

其中功能性包括质量子特性 完备性、一致性、可追踪性;

可靠性包括质量子特性 一致性;

易使用性包括质量子特性 操作性、培训性、产品文档完备性;

可维护性包括质量子特性 一致性、可见性、产品文档完备性、清晰性、模块性、自描述性、简单性、结构性。

15、软件复用是使用已有的软件产品 如设计、代码、文档等 来开发新的软件系统的过程。软件复用的形式大体可分为垂直式复用和水平式复用。垂直式复用是指 同一应用领域 中的复用, 水平式复用是指 不同应用领域 中的复用。

为了提高构件 Component 的复用率, 通常要求构件具有较好的 通用性和可变性。为了将不同软件生产商在不同软硬件平台上开发的构件组装成一个系统, 必须解决异构平台的各构件间的互操作问题。

目前国际上已出现了一些支持互操作的构件标准, 典型的有国际对象管理组织 OMG 推荐的 CORBA 和 Microsoft 公司推出的 DCOM。

16、软件测试通常可分为单元测试、集成测试、确认测试和系统测试, 其中确认测试主要用于发现 需求分析 阶

段的错误。在集成测试时，通常可采用自顶向下增殖式集成和自底向上增殖式集成。

在自底向上增殖式集成时，对每个被集成的模块 要设计驱动模块，但不必设计桩模块。

对那些为众多用户开发的软件，如操作系统，编译程序，通常还要进行 α 测试和 β 测试，以发现可能只有最终用户才能发现的错误，其中， α 测试是指最终用户在 开发环境下，开发人员在场 的情况下所进行的测试， β 测试是指最终用户在 用户的实际使用环境下，开发人员不在场 的情况下所进行的测试。

在软件维护阶段，当修改软件后，除了进行常规的测试外，还应进行 回归 测试。

17、类常常被看作是一个抽象数据类型的实现，更合适的是把类看作是 概念 的一个模型。事实上，类是单个 语义 单元。类的用户能够操纵的操作叫做类的 界面。类定义的其余部分给出数据定义和辅助功能定义，包括类的实现。

类的实现常常包括了其它类的实例，这些实例 应受保护不 被其它对象存取，包括同一个类的其它实例。类的实现可能还包括某些私有方法，实现它们的类可以使用，而其它任何对象都不能使用。

类，就它是一个数据值的聚合的意义来看，与 Pascal 中的记录或 C 中的结构类似，但又有区别。类扩展了通常的记录语义，可提供各种级别的 可访问性。

18、由 rumbaugh 等人提出一种面向对象方法叫做对象模型化技术 OMT，既三视点技术，它要求把分析时收集的信息建立在 3 个模型中。

第一个模型是 对象模型，它的作用是描述系统的静态结构，包括构成系统的对象和类、它们的属性和操作，以及它们之间的联系。

第二个模型是 动态模型，它描述系统的控制逻辑，主要涉及系统中各个对象和类的时序及变化状况。动态模型 包括两种图，即 状态迁移图 和 事件追踪图。状态迁移图 描述每一类对象的动态行为，事件追踪图 描述发生于系统执行过程中的某一个特定场景。

第三个模型是 功能模型，它着重于描述系统内部数据的传送与处理，它由多个数据流图组成。

19、软件维护是软件生存期的最后阶段。软件工程学针对维护工作的主要目的是提高 软件的可维护性，降低 维护的代价。软件的 可测试性、可理解性、可修改性 是决定软件可维护性的基本因素。

20、软件的再工程是软件的一类工程活动，它能够使人们： I 增加对软件的了解； II 准备或直接提高软件自身的可维护性、可复用性 或演化性。第 II 部分旨在改善软件的 静态质量，使得软件更容易为人们服务。纯粹是处于改善性能的代码优化 不属于 软件再工程。逆向工程属于上述软件再工程的第 I 部分。

21、从供选择的答案中选出同下列关于软件测试的各条叙述关系最密切的字句，分别填入 A、B、C、D、E 中。

代码审查 对可靠性要求很高的软件，例如操作系统，由第三者对源代码进行逐行检查。

退化测试 已有的软件改版时，由于受到变更的影响，改版强正常的功能可能发生异常，性能也可能下降。因此，对变更的软件进行测试是必要的。

白盒测试 在意识到被测试的模块的内部结构或算法的情况下进行测试。

原型 为了确认用户的需求，先做出系统的主要部分，提交给用户使用。

桩 在测试具有层次结构的大型软件时，有一种方法是从上层模块开始，由上到下进行测试。此时，有必要用一些模块替代尚未测试的下层模块。

22、在面向对象软件设计过程中，应按如下要求进行类的设计：只有类的共有界面的成员才能成为使用类的操作，

这就是软件设计的信息隐蔽准则。

当且仅当一个操作对类的实例的用户有用时它才是类公共界面的成员，这是软件设计的最小界面准则。

由同属一个类的操作负担存取或加工类的数据，这是软件设计的高内聚准则。

两个类之间的交互应当仅设计参数表，这是软件设计的显式信息传递准则。

每个派生类应当做基类的特殊化来开发，而基类所具有的公共截面成为派生类的共有界面的一个子集，这是软件设计的继承性准则。

23、集成测试也叫做组装测试或联合测试。通常，在单元测试的基础上，将所有模块按照设计要求组装成为系统。子系统的集成测试特别成为部件测试，它所做的工作是要找出子系统和系统需求规格说明之间的不一致。

24、软件可维护性是纠正软件系统出现的错误和缺陷，以及为满足新的要求进行修改、扩充的容易程度。目前广泛使用7个特性来衡量软件的可维护性，其中就有可靠性、可理解性、可使用性。其中，可靠性和可理解性主要在改正性维护中侧重应用，可使用性主要在适应性维护和完善性维护中侧重应用。

25、软件的再工程技术主要有改进软件、获取、保存及扩充软件知识和理解软件。分析、度量属于理解软件，知识库和变换属于获取、保存及扩充软件知识。复用工程属于改进软件。

26、所谓一个构件，可以是一个类型、类或其他的工作成品。对于构件，应当按可复用的要求设计、实现、打包、编写文档。构件是内聚的，并具有相当稳定的公开的接口。这里所说的构件是基于面向对象技术的。

27、软件项目进度管理有许多方法，但IPO不是常用的进度控制图示方法。在几种进度控制图示方法中，甘特图难以表达多个子任务间的逻辑关系，使用PERT不仅能表达子任务之间的逻辑关系，而且可以找出关键子任务。在PERT中，用带箭头的边表示任务，用圆圈结点表示事件。

28、在考虑各种软件开发资源时，人力是最重要的资源。如果把软件开发所需要的资源画成一个金字塔形：在塔的上层是最基本的资源人力，在底部为工具。工具包括硬件资源和软件资源。宿主机、目标机和其他硬件设备属于硬件资源。IPSE工具属于软件资源中的框架工具。

二、名词解释（100题）

1.语句覆盖

属于白盒测试方法之一，选择足够的测试用例，使得程序中的每个语句至少都能被执行一次

2.软件

软件是计算机系统中与硬件相互依存的部分，它是包括程序、数据及相关文档的完整集合

3.项目

项目是一件事情、一项独一无二的任务，也可以理解为是在一定的时间和一定的预算内所要达到的预期目的。项目是指在一定的约束条件下（主要是限定时间、限定资源），具有明确目标的一次性任务。项目是一系列具有特定目标，有明确开始和终止日期，资金有限，消耗资源的活动和任务。

4.通信聚合

通信内聚（信息内聚）：**模块中的所有功能都操作相同的数据结构，即模块内各个组成部分都使用相同的数据或产生相同的数据结构。**

5.分布图

分布图反映了系统中**软件和硬件的物理架构**，表示系统运行时的处理节点以及节点中组件的配置。

6. 判定覆盖

属于**白盒测试**方法之一，所谓判定覆盖，就是**设计若干个测试用例**，运行被测程序，**使得程序中每判定的取真分支和取假分支至少评价一次**

7.软件工程

软件工程是研究和应用如何以系统化的、规范的、可度量的方法去开发、运行和维护软件，即把工程化方法应用到软件上。

软件工程是指导计算机软件开发和维护的工程学科。采用工程的概念、原理、技术和方法来开发与维护软件，把经过时间考验而证明正确的管理技术和当前能够得到的最好的技术方法结合起来，以经济地开发出高质量的软件并有效地维护它，这就是软件工程。

8.配置管理

配置管理(Configuration Management, CM)是**通过技术或行政手段对软件产品及其开发过程和生命周期进行控制、规范的一系列措施**。配置管理的目标是记录软件产品的演化过程，确保软件开发者在软件生命周期中各个阶段都能得到精确的产品配置。

9.信息内聚（通信内聚）

通信内聚（信息内聚）：**模块中的所有功能都操作相同的数据结构，即模块内各个组成部分都使用相同的数据或产生相同的数据结构。**

10.组件图

组件图描述组件以及它们之间的关系，表示系统的**静态实现视图**。

11.条件覆盖

条件覆盖是指选择足够的测试用例，使得运行这些测试用例后，要使**每个判定中每个条件的可能取值至少满足一次，但未必能覆盖全部分支，即每个条件至少有一次真值、有一次假值。**

12.软件危机

软件危机是指在计算机软件的开发和维护过程中所遇到的一系列严重问题。

软件危机是指在**计算机软件开发和维护过程中，由于技术、管理或需求等多方面因素引发的系统性挑战，表现为项目失控、质量缺陷、成本超支及维护困难等问题。**

13.配置项

配置中的实体，它满足最终使用要求，并且对于特定的引用，它可被唯一确定。

配置项是指为配置管理设计的硬件、软件或两者的集合。它在配置管理中作为单个实体来对待。配置管理通过管理不断演化和完善的软件产品，确保开发者在软件生命周期中的各个阶段都能得到精确的产品配置。配置项可以包含

计算机可执行源代码、目标码、数据库、计算机不可执行的文档、源程序清单、测试用例等

14.数据耦合

数据耦合 (Data Coupling)：当模块间的交互仅仅通过**参数来传递基本数据类型**时。这是最佳的耦合方式。如上面的赋值操作，或者如下的电费计算。

15.活动图

活动图反映系统中从一个活动到另一个活动的流程，**强调对象间的控制流程**。
表示连续的活动流，通常用来描述一个操作所需要的活动。

16.路径覆盖

路径覆盖指设计足够的测试用例，覆盖被测程序中所有可能的路径。

路径覆盖要求设计足够多的测试用例，在**白盒测试法**中，**覆盖程度最高的就是路径覆盖**，因为其覆盖程序中**所有可能的路径**。

17.软件生存周期（软件生命周期）

软件生存周期是指软件产品**从考虑其概念开始到该软件产品交付使用，直至最终退役为止的整个过程**，一般包括计划、分析、设计、实现、测试、集成、交付、维护等阶段。

软件生命周期是指软件**从开始构思与研发到不再使用而消亡的过程**。一般软件生命周期可以分为以下几个阶段：可行性研究与计划、需求分析、概要设计、详细设计、实现、集成测试、确认测试、使用和维护。

*扩展概念：**软件生存周期模型=软件过程模型+软件生命周期模型**

软件生存期模型也被称为软件过程模型，是从软件项目需求定义直至软件运行维护为止，跨越整个生命周期的系统开发、运行和维护所实施的全部过程、活动和任务的结构框架。

18.基线

基线是软件生存期中各开发阶段的一个特定点，它的作用是把开发各阶段工作的划分更加明确化，使本来连接的工作在这些点上断开，以便于检查与肯定阶段成果。

基线是一个软件**配置管理**的概念，它能够帮助我们在**不严重阻碍合理变更的条件下控制变更**，IEEE 中这样定义基线：**已经通过正式审评和批准的规格说明或产品**，他可以作为进一步开发的基础，并且只有通过正式的变更控制规程才能修改它

19.控制耦合

控制耦合是一个模块在界面上**传递一个信号（如开关值、标志量等）控制另一个模块**，接收信号的模块的动作根据信号值进行调整。

20.协作图

协作图按时间和空间的顺序描述系统元素之间的交互和关系,侧重于描述消息的组成关系。

21.条件组合覆盖

在白盒测试法中，选择足够的测试用例，使所有判定中**各条件判断结果的所有组合**至少出现一次，满足这种覆盖标准成为条件组合覆盖。

22.软件过程

软件过程是人们**用于开发和维护软件及其相关过程的一系列活动**，包括**软件工程活动**和**软件管理活动**。人们在开

发和维护软件及其相关产品时所涉及的各种活动、方法、实践和改革等。其中软件相关产品包括软件项目计划、设计文档、程序代码、测试用例和用户手册等。

***扩展：**

软件过程能力：当遵循某个软件过程时所能达到的**期望效果**，它可以有效预测企业接收新的软件项目时可能得到的结果。

23.里程碑

里程碑一般是项目中完成阶段性工作的标志，标志着上一个阶段结束、下一个阶段开始，将一个过程性的任务用一个结论性的标志来描述，明确任务的起止点。一系列的起止点就构成了引导整个项目进展的里程碑。里程碑定义了当前阶段完成的标准和下一新阶段启动的条件和前提。

24.标记耦合

标记耦合是指**模块间通过参数传递复杂的内部数据结构**，其实传递的是数据结构的地址，这种数据结构的变化将使相关的模块发生变化。

25.时序图

时序图描述了对象之间传递消息的时间顺序，用来表示用例中的行为顺序，**是强调消息时间顺序的交互图**。

26.等价类划分

等价类划分法是一种典型的、重要的**黑盒测试方法**，它将程序所有可能的输入数据（有效的和无效的）划分成若干个等价类。然后从每个部分中选取具有代表性的数据当做测试用例进行合理的分类，测试用例由有效等价类和无效等价类的代表组成，从而保证测试用例具有完整性和代表性。

27.软件基本过程

软件过程 (Software Process) 是指一套关于项目的阶段、状态、方法、技术和开发、维护软件的人员以及相关 Artifacts（计划、文档、模型、编码、测试、手册等）组成。软件过程可概括为三类：**基本过程类、支持过程类和组织过程类**。基本过程类包括获取过程、供应过程、开发过程、运作过程、维护过程和管理过程。

28.项目范围管理

项目范围管理是指对项目包括什么与不包括什么进行定义并控制的过程。这个过程用于确保项目组和项目干系人对做为项目结果的项目产品以及生产这些产品所用到的过程有一个共同的理解。

29. 开源软件

开源软件是一种**源代码可以任意获取**的计算机软件，这种**软件的版权持有人在软件协议的规定之下保留一部分权利并允许用户学习、修改以及以任何目的向任何人分发该软件**。

扩展概念：

自由软件表示的是那些赋予用户 运行、复制、分发、学习、修改并改进软件这些自由的软件。

免费软件就是免费提供给用户使用的软件，但是其免费的时候，通常都会有其他的限制，比如其源码不一定会公开，而且使用者也并没有使用、复制、研究、修改和再散布的权利。

30.状态图

状态图通常是对类描述的补充，它说明该类的对象所有可能的状态 以及哪些事件将导致的改变。

31.边界值测试（边界值分析）

边界值分析是黑盒测试的一种方法。在等价类划分的基础上，针对输入数据或操作条件的边界值或极限值进行测试。

32.软件支持过程

支持过程类包括文档过程、配置管理过程、质量保证过程、验证过程、确认过程、联合评审过程、审计过程以及问题解决过程。

33.项目整体管理

也叫集成管理，保证项目各个要素相互协调所需要的管理过程，它需要在相互影响的项目目标和方案中作出平衡，以满足或超出项目干系人的需求和愿望

34.预防性维护

预防性维护是为了改进应用程序的可靠性和可维护性，为了适应未来的软硬件环境的变化，应主动增加预防性的新的功能，以使应用系统适应各类变化而不被淘汰。

35.对象图

对象图是类的实例，它表示系统执行在某一时间点的一个可能的快照（snapshot）。

36.基本路径测试

是在程序控制流图的基础上，通过分析控制构造的环路复杂性，导出基本可执行路径集合，从而设计测试用例的方法。设计出的测试用例要保证在测试中程序的每个可执行语句至少执行一次

37.软件组织过程

组织过程类包括基础设施过程、改进过程以及培训过程。

38.软件度量

是对软件开发项目、过程及其产品进行数据定义、收集以及分析的持续性定量化过程，目的在于对此加以理解、预测、评估、控制和改善。

39.适应性维护

适应性维护：是指使应用软件适应信息技术变化和管理需求变化而进行的修改。

扩展考其他几个维护解释：

- **改正性维护：**是指改正在系统开发阶段已发生而系统测试阶段尚未发现的错误。
- **适应性维护：**是指使应用软件适应信息技术变化和管理需求变化而进行的修改。
- **完善性维护：**是指为扩充功能和改善性能而进行的修改，主要是指对已有的软件系统增加一些在系统分析和设计阶段中没有规定的功能与性能特征。
- **预防性维护：**是为了改进应用程序的可靠性和可维护性，为了适应未来的软硬件环境的变化，应主动增加预防性的新的功能，以使应用系统适应各类变化而不被淘汰。

40.类图

类图描述系统的静态结构，表示系统中的类以及类与类之间的关系。

一个系统可以有多张类图，一个类也可以出现在几张类图中

41.黑盒测试

黑盒测试把被测试对象看成一个黑盒子，测试人员完全不考虑程序的内部结构和处理过程，只在软件的接口处进行测试，依据需求规格说明书，检查程序是否满足功能要求。

黑盒测试也称功能测试，它是通过测试来检测每个功能是否都能正常使用。在测试中，把程序看作一个不能打开的黑盒子，在完全不考虑程序内部结构和内部特性的情况下，在程序接口进行测试，它只检查程序功能是否按照需求规格说明书的规定正常使用，程序是否能适当地接收输入数据而产生正确的输出信息。黑盒测试着眼于程序外部结构，不考虑内部逻辑结构，主要针对软件界面和软件功能进行测试。

42.过程框架

定义了若干小的框架活动，为完整的软件开发过程建立基础

43.功能点分析

是面向功能的软件度量方法，注意力集中于软件系统的“功能性”和“实用性”

44.结构化程序设计

进行大型程序设计的一种方法。整个程序是由各个部分按一定的层次结构组织起来的。每一层次中规定只使用几种基本的程序结构。从最高层开始，依次向下，逐步求精。设计出的程序条理清楚、容易理解、容易调试与维护

45.用例图

表示各类外部参与者与系统所提供的用例之间的连接。一个用例是系统所提供的一个功能的描述。给出了用户所感受到的系统行为，但不描述系统如何实现该功能。

46.白盒测试

白盒测试也称结构测试或逻辑驱动测试，该方法把测试对象看作一个打开的盒子，测试人员须了解程序的内部结构和处理过程，以检查处理过程的细节为基础，对程序中尽可能多的逻辑路径进行测试，检验内部控制结构和数据结构是否有错，实际的运行状态与预期的状态是否一致。

47.软件能力成熟度模型(CMMI)

成熟度可以用于指示组织加强其软件过程能力的潜力。

软件过程成熟度是指一个特定的软件过程被显式定义、管理、度量、控制和执行的程度。

软件能力成熟度模型是对于软件组织在定义、实施、度量、控制和改善其软件过程的实践中各个发展阶段的描述。CMM的核心是把软件开发视为一个过程，并根据这一原则对软件开发和维护进行过程监控和研究，以使其更加科学化、标准化、使企业能够更好地实现商业目标。

CMMI 是一种基于成果的绩效解决方案模型，能为组织带来更快、更好、更经济的结果。

CMMI 是全球公认的能够提升和增强组织能力和绩效的标准。

CMMI 提供了一条优先级明确的路径，用于构建和实施新的能力，从而持续交付可衡量的结果和成果。

48.工作分解结构

以可交付成果为导向对项目要素进行的分组，它归纳和定义了项目的整个工作范围每下降一层代表对项目工作的更详细定义。

49.非功能性需求

非功能性需求 (Non-functional requirement) 是指依据一些条件判断系统运作情形或其特性，而不是针对系统特定行为的需求。非功能性需求一般被称为系统的品质，或限制、品质属性、品质目标、品质服务需求、非行为性的需求。

非功能性需求可以分为以下二类：

- (1) 运行品质 (Execution qualities)，在系统运行时观察到的品质，例如安全性及易用性等。
- (2) 发展品质 (Evolution qualities)，与软件系统结构及开发过程有关的品质，例如软件可测试性 (testability)、可维护性、可扩展性、可伸缩性 (scalability) 等。

50.统一建模语言 UML

它是一个支持模型化和软件系统开发的图形化语言,为软件开发的所有阶段提供模型化和可视化支持，包括由需求分析到规格，到构造和配置。是一种面向对象的建模语言，它是运用统一的、标准化的标记和定义实现对软件系统进行面向对象的描述和建模。

51.单元测试

集中对用源代码实现的每个程序单元进行测试，检查各个模块是否正确地实现了规定的功能。

52.个体软件过程

就是为使软件工程师更好地工作而设计的一个框架，它指出如何估计和计划工作，如何按照这些计划来跟踪自己的性能，以及如何提高程序的质量。

53.COCOMO 模型

构造性成本模型，它是在静态、单变量模型的基础上构造出来的。它是一种精确、易于使用的，基于模型的成本估算方法。

54.信息隐蔽

信息隐蔽指在设计和确定模块时,使得一个模块内包含信息(过程或数据),对于不需要这些信息的其他模块来说,是不能访问的。在面向对象方法中，信息隐蔽是通过对象的封装性来实现的。信息隐蔽的概念与模块的独立性直接相关。

55.面向对象

首先根据客户需求抽象出业务对象；然后对需求进行合理分层，构建相对独立的业务模块；之后设计业务逻辑，利用多态、继承、封装、抽象的编程思想，实现业务需求；最后通过整合各模块，达到高内聚、低耦合的效果，从而满足客户要求。

56.集成测试

集成测试,也叫组装测试或联合测试。在单元测试的基础上,将所有模块按照设计要求 (如根据结构图 组装成为子系统或系统, 进行集成测试。

57.团队软件过程

团队软件过程是为开发软件产品的开发团队提供指导，TSP 的早期实践侧重于帮助开发团队改善其质量和生产率，以使其更好的满足成本及进度的目标。加上 PSP 帮助高绩效的工程师在一个团队中工作，来开发有质量保证的软件产品，生产安全的软件产品，改进组织中的过程管理。

58.项目计划评审技术

计划评审技术就是工程项目当作一种系统，用网络图或者表格或者矩阵来表示各项具体工作的先后顺序和相互关系，以时间为中心，找出从开工到完工所需要时间的最长路线，并围绕关键路线对系统进行统筹规划，合理安排以及对各项工作的完成进度进行严密的控制，以达到用最少的时间和资源消耗来完成系统预定目标的一种计划与控制方法。

59.内聚

内聚是模块功能强度（一个模块内部各个元素彼此结合的紧密程度）的度量。

60.主动对象

主动对象内部包含一个**线程**，可以自动完成动作或改变状态。主动对象是内部拥有自己的控制线程的对象。

61.确认测试

检查已实现的软件是否满足了需求规格说明中所确定的各种需求，以及软件配置是否完全、正确。（过程：功能性测试---软件配置复查---验收测试--- α 测试和 β 测试）

62.过程模式（未找到答案）

所谓软件过程模型就是一种开发策略，这种策略针对软件工程的各个阶段提供了一套范形，使工程的进展达到预期的目的。对一个软件的开发无论其大小，我们都需要选择一个合适的软件过程模型，这种选择基于项目和应用的性质、采用的方法、需要的控制，以及要交付的产品的特点。一个错误模型的选择，将迷失我们的开发方向。

63.RMMM 计划 (Risk Mitigation, Monitoring and Management Plan)

翻译为：风险缓解，监测和管理计划，又称软件项目风险管理，是软件项目管理的重要内容。在进行软件项目风险管理时，要辩识风险，评估它们出现的概率及产生的影响，然后建立一个规划来管理风险。风险管理的主要目标是预防风险。

软件项目风险是指在软件开发过程中遇到的预算和进度等方面的问题以及这些问题对软件项目的影响。软件项目风险会影响项目计划的实现，如果项目风险变成现实，就有可能影响项目的进度，增加项目的成本，甚至使软件项目不能实现。

64.耦合

耦合是模块之间的相对独立性（互相连接的紧密程度）的度量。

65.多态性

同一操作作用于不同的**类的实例**，将产生不同的执行结果，即不同类的**对象**收到相同的消息时，得到不同的结果。对象根据所接受的消息而做出动作，同样的消息被不同的对象接收时可能导致完全不同的行为，这种现象称为多态性。

66.系统测试

是将通过确认测试的软件，作为整个基于计算机系统的一个元素，与计算机硬件、外部设备、某些支持软件、数据和人员等其他系统元素结合在一起，在实际运行（使用）环境下，对计算机系统进行一系列的**组装测试和确认测试**。

67.统一过程

是软件工程的过程。它提供了在开发组织中分派任务和责任的纪律化方法。它的目标是在可预见的日程和预算前提下，确保满足最终用户需求的高质量产品。

统一过程模型是一种“**用例驱动，以体系结构为核心，迭代及增量**”的软件过程框架，由 UML 方法和工具支持。

68.软件质量

与软件产品满足规定的和隐含的需求的能力有关的特性或特性的全体。或“所有描述计算机软件优秀程度的特性的组合。”

69.体系结构

体系结构包括一组部件以及部件之间的联系。

70.消息

消息，软件对象之间进行交互作用和通讯是利用消息的。在面向对象的程序设计中，消息是指一个类实例和另一个类实例之间传递的信息。

71.压力测试

在软件工程中，压力测试是对系统不断施加压力的测试，是通过确定一个系统的瓶颈或者不能接受的性能点，来获得系统能提供的最大服务级别的测试。

72.瀑布模型

将软件生存周期各个活动规定为依线性顺序连接的若干阶段的一种软件开发模型。它包括可行性分析、项目开发计划、需求分析、概要设计、详细设计、编码、测试和维护。

瀑布模型是将软件生存周期的各项活动规定为按固定顺序而连接的若干阶段工作，形如瀑布流水，最终得到软件产品。瀑布模型核心思想是按工序将问题简化，将功能的实现与设计分开，便于分工协作，即采用结构化的分析与设计方法将逻辑实现与物理实现分开。将软件生命周期划分为制定计划、需求分析、软件设计、程序编写、软件测试和运行维护等六个基本活动，并且规定了它们自上而下、相互衔接的固定次序，如同瀑布流水，逐级下落。

73.结构复杂性度量（未找到答案）

软件度量的根本目的是为了管理的需要,利用度量来改进软件过程。人们是无法管理不能度量的事物。没有对软件过程的可见度就无法管理;而没有对见到的事物有适当的度量或适当的准则去判断、评估和决策,也无法进行优秀的管理。

结构复杂性度量就是为了管理需要，通过工程化的方法去衡量软件内部的结构复杂性。

74.决策表

决策表又称判断表，是一种呈表格状的图形工具，适用于描述处理判断条件较多，各条件又相互组合、有多种决策方案的情况。精确而简洁描述复杂逻辑的方式，将多个条件与这些条件满足后要执行动作相对应。但不同于传统程序语言中的控制语句，决策表能将多个独立的条件和多个动作直接的联系清晰的表示出来

75.继承

继承是指一个对象直接使用另一对象的属性和方法。

76.测试配置

测试配置是指在软件测试过程中，为了保证测试的正确性和有效性，需要对测试环境、测试数据、测试工具等进行配置。测试配置的目的在于提高软件测试的效率和质量

77.快速原型模型（原型模型）

快速原型模型是一种迭代的软件开发模型，它强调在开发过程中不断迭代，逐步完善产品的功能。开发人员可以快速创建原型，与客户进行交互和反馈，以便更好地理解 and 确认需求。实现客户或未来的用户与系统的交互，用户或客户对原型进行评价，进一步细化待开发软件的需求。通过逐步调整原型使其满足客户的要求，开发人员可以确定客户的真正需求是什么；第二步则在第一步的基础上开发客户满意的软件产品。这个模型适用于需要快速开发并频繁更新的项目。

原型模型又称快速原型模型。它是在开发真实系统之前，构造一个原型，在该原型的基础上，逐渐完成整个系统的开发工作。

原型是在软件开发过程中用于验证和改进设计的预备模型。通过创建原型，开发人员能够更好地与用户交流，了解和确认软件系统的功能和界面等设计要素。原型有助于减少需求变更和开发错误的风险。

78.软件复杂性度量

软件复杂性度量是指对软件系统的复杂性进行度量的过程。常用的软件复杂性度量方法包括代码行度量法、McCabe 度量法等。

79.数据字典

数据字典是关于数据的信息的集合，也就是对数据流图中包含的所有元素的定义的集合。

精确地、严格地定义了**每个与系统相关的数据元素**，并以字典式顺序将它们组织起来，使得用户和分析员对所有的输入、输出、存储成分和中间计算有共同的理解。

80.封装

隐藏对象的属性和实现细节，仅对外公开接口,控制在程序中属性的读和修改的访问级别。

封装就是将抽象得到的数据和行为（或功能）相结合，形成一个有机的整体，也就是将数据与操作数据的源代码进行有机的结合，形成“类”，其中数据和函数都是类的成员。

81.静态测试

静态测试是指无须执行被测代码，而是借助专用的软件测试工具评审软件文档或程序，度量程序静态复杂度，检查软件是否符合编程标准，借以发现编写的程序的不足之处，减少错误出现的概率。

静态测试指被测程序不在机器上运行，而是采用人工检测和计算机辅助静态分析的手段对程序进行检测。

82.增量模型

增量模型与原型实现模型和其他演化方法一样，本质上是迭代的，但与原型实现不一样的是其强调每一个增量均发布一个可操作产品。早期的增量是最终产品的“可拆卸”版本，但提供了为用户服务的功能，并且为用户提供了评估的平台。

增量模型是在项目的开发过程中以一系列的增量方式开发系统。

增量方式包括增量开发和增量提交。

增量开发是指在项目开发过程中以一定的时间间隔开发部分工作软件；

增量提交是指在开发周期内，以一定的时间间隔增量方式向用户提交工作软件及相应文档。增量开发和增量提交可以同时使用，也可单独使用。

迭代是软件开发过程中的一个重要概念，指的是将软件开发划分为多个可重复的步骤。在每一次迭代中，开发团队会对软件进行设计、编码、测试和评估等环节，从而逐步改进软件系统。

83.软件可靠性

软件可靠性是指在规定的条件下,在规定的时间内,软件不引起系统失效的概率;在规定的时期内,在所述条件下程序执行所要求的功能的能力。

84.上下文数据流图

上下文数据流图是一种数据流图，它用于表达一个系统的概览，只包含一个代表整个系统的程序。这个程序会被分解成更多细节的主要程序，每个主要程序可能会进一步分解以提供更多细节

85.对象/类

类是对象的抽象，而对象是类的具体实例。类是抽象的，不占用内存，而对象是具体的，占用存储空间。

具有相同或相似性质的对象的抽象就是类。

对象是人們要进行研究的任何事物，从最简单的整数到复杂的飞机等均可看作对象，它不仅能表示具体的事物，还能表示抽象的规则、计划或事件。类的具体化就是对象，也可以说类的实例是对象。

抽象是认识复杂现象过程中使用的思维工具即抽出事物本质的共同的特征而暂不考虑它的细节，不考虑其它因素。

86.动态测试

所谓软件的动态测试，就是通过运行软件来检验软件的动态行为和运行结果的正确性。目前，动态测试也是公司的测试工作的主要方式

87.螺旋模型

对于复杂的大型软件，开发一个原型往往达不到要求。螺旋模型将瀑布模型与演化模型结合起来，并且添加两种模型均忽略的风险分析。螺旋模型沿着螺线旋转，分 4 个方面的活动：制定计划、风险分析、实施工程、客户评估。沿螺旋线自内向外每旋转一圈，便开发出一个更为完善的、新的软件版本

88.错误播种（植入）模型

错误播种（植入）模型是一种用于估算程序中残留错误总数的模型，即在进行测试之前，由专人在程序中随机地植入一些错误（称为带有标记的错误），测试人员测试之后，通过统计测试人员发现的原有错误和植入错误的比例来估算程序中残留错误总数

89.数据流图

数据流图：Data Flow Diagram 简称 DFD，它从数据传递和加工角度，以图形方式来表达系统的逻辑功能、数据在系统内部的逻辑流向和逻辑变换过程，是结构化系统分析方法的主要表达工具及用于表示软件模型的一种图示方法。

90.静态视图

是因为它不描述与时间有关的系统行为，此种行为在其他视图中进行描述。静态视图主要是由类及类间相互关系构成，这些相互关系包括：关联、泛化和各种依赖关系，如使用 and 实现关系

91.Beta 测试

Beta 测试由软件的最终用户们在一个或多个客房场所进行。与 Alpha 测试不同，开发者通常不在 Beta 测试的现场，因 Beta 测试是软件在开发者不能控制的环境中的“真实”应用。用户 Beta 测试过程中遇到的一切问题（真实在或想像的），并且定期把这些问题报告给开发者。接收到在 Beta 测试期间报告的问题之后，开发者对软件产品进行必要的修改，并准备向全体客户发布最终的软件产品

92.极限编程

极限编程是一个轻量级的、灵巧的软件开发方法；同时它也是一个非常严谨和周密的方法。它的基础和价值观是交流、朴素、反馈和勇气；即，任何一个软件项目都可以从四个方面入手进行改善：加强交流；从简单做起；寻求反馈；勇于实事求是。XP 是一种近螺旋式的开发方法，它将复杂的开发过程分解为一个个相对比较简单的小周期；通过积极的交流、反馈以及其它一系列的方法，开发人员和客户可以非常清楚开发进度、变化、待解决的问题和潜在的困难等，并根据实际情况及时地调整开发过程

93.软件质量模型

软件质量模型是一种用于评估软件质量的模型，它包括过程模型、产品模型和外部特性模型等方面。常用的软件质量模型包括 ISO/IEC 9126、ISO/IEC 12207 等

94.业务流程图

业务流程图是一种描述系统内各单位、人员之间业务关系、作业顺序和管理信息流向的图表，利用它可以帮助分析人员找出业务流程中的不合理流向，它是物理模型。业务流程图主要是描述业务走向。业务流程图描述的是完整的业务流程，以业务处理过程为中心，一般没有数据的概念

95.动态视图

动态视图是软件架构中的一种视图，用于描述系统的动态行为。它通常包括活动图、状态图、时序图和协作图等

96.因果图

鱼骨图也称为因果分析图或石川图。它看上去有些象鱼骨，问题或缺陷（即后果）标在"鱼头"外。在鱼骨上长出鱼刺，上面按出现机会多寡列出产生生产问题的可能原因。鱼骨图有助于说明各个原因之间如何相互影响。它也能表现出各个可能的原因是如何随时间而依次出现的。这有助于着手解决问题

97.过程规范

过程规范是指在软件开发过程中，为了保证软件质量和开发效率，需要对软件开发过程进行规范化。常用的过程规范包括 ISO/IEC 12207 等

98.基于时间的缺陷模式（未找到答案）

是一种软件过程质量测试方法，是基于时间的缺陷到达模式，缺陷到达模式可以提供更多的信息，有时即使得到的整体缺陷率是一样的，但其质量差异可能较大，原因就是缺陷到达的时间不同，越多的缺陷到达越早，则测试过程质量就越好。

99.过程模型

所谓软件过程模型就是一种开发策略，这种策略针对软件工程的各个阶段提供了一套范形，使工程的进展达到预期的目的。对一个软件的开发无论其大小，我们都需要选择一个合适的软件过程模型，这种选择基于项目和应用的性质、采用的方法、需要的控制，以及要交付的产品的特点。一个错误模型的选择，将迷失我们的开发方向

100.领域分析

领域分析的目的是支持软件重用，它产生的是描述了特定领域中各系统共性的领域模型。

领域分析的过程由领域分析的前期准备、分析领域知识和形成重用成分三部分组成。

面向对象分析方法是领域分析的一个很好工具。

领域分析是指在软件开发过程中，对软件应用领域进行分析和研究，以便更好地理解用户需求和系统功能。

101. 类整体-部分结构图

如建立一个时钟类，这个类中的属性如秒、分钟、小时等，加上对属性的操作如秒计时，分钟计时，小时计时等，就是一个类的整体部分结构。

102. 决策过程

决策过程是从提出问题、确定目标开始，经过方案选优、作出决策、交付实施为止的全部过程。

103. Bug

计算机程序中出现的错误或故障，导致程序无法正常运行或产生意外的错误的结果。

104、性能测试

性能测试是通过测试工程模拟多种正常、峰会以及异常负载条件来对系统的各项性能指标进行测试。

105、构件图（组件图）

构件图是以代码构件为单位表示代码的物理结构，及构件之间的依赖关系。

106、部署图

部署图是表示系统中硬件和软件的物理结构，及各部分之间的连接方式。

107、需求

需求是指软件系统或产品中必须满足的功能、性能、质量和其他特定要求的描述。

108、需求分析

需求分析是指开发人员要准确理解用户的要求，进行细致的调查分析，将用户非形式的需求陈述转化为完整的需求定义。再由需求定义转化为相应的形式功能规约的过程。

需求分析也称为软件需求分析、系统需求分析或需求分析工程等，是开发人员经过深入细致的调研和分析，准确理解用户和项目的功能、性能、可靠性等具体要求，将用户非形式的需求表达转化为完整的需求定义，并由需求定义转换到相应的形式功能规约（需求规格说明书），从而确定系统必须做什么的过程。

109、内容耦合

内容耦合（Content Coupling）：一个模块直接使用另一个模块的内部数据或者控制流程时。比如当一个类直接调用访问另一个类的属性信息。

120、公共耦合

公共耦合（Common Coupling）：当两个或更多模块共享同一个全局数据结构或全局变量时，这些模块被认为具有公共耦合。公共环境包括全局变量、共享数据结构、或存储介质上的文件等。模块间通过访问这些共享的数据结构或全局变量进行交互。这种耦合方式比较紧密，因为模块间的共享数据使得它们的依赖关系更加显著。与数据耦合不同，公共耦合涉及的全局数据结构可能会被多个模块同时读写，增加了模块间的耦合度。

121、外部耦合

外部耦合（External Coupling）：当多个模块共享一个外部传来的数据结构或者全局变量时。如 I/O 将模块耦合到特定的设备、格式、通信协议上。

122、软件众包

众包（crowdsourcing）指的是一个公司或机构把过去由员工执行的工作任务，以自由自愿的形式外包给非特定的（而且通常是大型的）大众志愿者的做法。

123、软件质量

软件质量是指所有描述计算机软件优秀程度的特性的组合。通俗地说就是,软件满足各功能、性能需求,符合标准的程度,程度高则质量。IEEE 定义软件质量为“**与软件产品满足规定的和隐含的需求的能力有关的特征或特性的全体**”。

*扩展概念：

软件质量模型：ISO/IEC 9126 软件质量模型由 3 个层次组成：第一层是质量特性，第二层是质量子特性，第三层是度量指标。ISO9126 软件质量模型是评价软件质量的国际标准,由 6 个特性和 27 个子特性组成。ISO 9126 标准已经被 ISO/IEC

25010:2011 取代，后者是国际标准组织（ISO）目前关于软件工程-质量模型的标准。

124、软件质量保证

软件质量保证（SQA, Software Quality Assurance）是一种应用于整个软件过程的保护性活动。提高软件质量的途径包括提高过程的可见性、改进项目过程结构和在中间阶段进行检查。

软件质量保证是向用户及社会提供满意的高质量的产品，确保软件产品从诞生到消亡为止的所有阶段的质量的活动，即确定、达到和维护需要的软件质量而进行的所有有计划、有系统的管理活动。

125、无耦合

无耦合（No Coupling）：模块之间没有任何直接的关系，是最理想的状态，但在实际应用中很难完全实现。

126、偶然内聚

偶然内聚（Coincidental Cohesion）：模块内部各个部分之间没有明显的关联，仅仅是偶然放在一起，这是最低级别的内聚。通常会导致代码难以维护和理解。这里提一嘴，我认为代码生成器就是一种偶然内聚，只是因为其他模块都有执行若干不相干的语句，我们将之提取，并作为独立模块。

127、逻辑内聚

逻辑内聚（Logical Cohesion）：当一个模块中的元素被组织在一起，因为它们在逻辑上分类相似，通常通过一个控制语句（如 if-else 或 switch）来选择执行。一些不相干的功能被组织在同一个模块中，通过外部参数来控制实现哪个功能。简单记为，通过逻辑语句联系在一起的内聚。

128、时间内聚

时间内聚（Temporal Cohesion）：模块中的元素相关联，因为它们需要在同一时间段内被执行。顾名思义，就是单纯在时段上重合，也称为瞬时内聚，关键词就是同时执行。

129、过程内聚

过程内聚（Procedural Cohesion）：当模块中的元素协同工作执行一个任务序列，彼此之间按照特定顺序执行。例如：先写姓名 → 电话 → 家庭住址，先后顺序符合客户需求。关键词就是特定顺序执行。

130、统计质量管理

统计质量管理又称统计质量控制，是把数理统计原理用来控制生产过程和产品质量的质量管理方法。

131、顺序内聚

顺序内聚（Sequential Cohesion）：当一个模块内的操作必须按特定顺序执行，输出数据成为下一步操作的输入数据时。前一功能元素的输出就是下一功能元素的输入。例如：先计算员工的年龄再计算退休时间。例如某要完成获取订单信息的功能，前一个功能获取用户信息，后一个执行计算均价操作，显然该模块内两部分紧密关联。

过程内聚与顺序内聚的区别是：

顺序内聚中是数据流从一个处理单元流到另一个处理单元，而过程内聚是控制流从一个动作流向另一个动作。

132、功能内聚

功能内聚（Functional Cohesion）：当一个模块内的所有功能都是围绕一个单一的功能组织的，这是最高级别的内聚。ALL for ONE，所有处理元素只为完成某一个功能，缺一不可。

132、风险

风险就是在项目过程中有可能发生的某些意外事情，而且可能对项目产生巨大的负面影响甚至导致项目失败。风险是指在一定条件下和一定时间内可能发生的各种结果的变化程度。

133、风险管理

风险管理是对项目风险从识别到分析乃至采取应对措施等的一系列过程，它包括将积极因素的影响最大化和使消极因素的影响最小化两方面。

134、软件可维护性（可维护性）

软件可维护性是指软件能够被理解、校正、适应及增强功能的容易程度。可维护性可用 7 个质量特性来衡量,即可理解性、可测试性、可修改性、可靠性、可移植性、可使用性和效率。

135、需求评审

需求评审是软件开发过程中的重要环节。它的目的是确保需求文档的准确性、完整性和一致性，以及消除歧义。

136、需求管理

需求管理是一个对系统需求变更了解和控制的过程。需求管理的过程与其它需求工程过程相互关联的。

137、需求跟踪

需求跟踪是指建立需求与其前后两个方向相关元素之间关联关系的活动，其目的是建立与维护“需求来源-需求规约-设计文档-程序代码-测试用例”之间的一致性，确保所有的工作成果符合用户的需求。

138、RUP（统一过程）

RUP（Rational Unified Process，统一软件开发过程，统一软件过程是一个面向对象且基于网络的程序开发方法论。RUP 是一种迭代增量式的软件开发方法论，它强调将系统开发过程分为一系列的迭代周期，每个周期都包括了需求分析、设计、编码、测试和部署等活动。

139、功能需求

功能性需求（functional requirement）规定开发人员必须在产品中实现的软件功能，用户利用这些功能来完成任务,满足业务需求。n 例如：用户登录、订单查询、电子支付、论坛网站的发帖留言等。

140、结构化开发方法

结构化开发方法（structured development method），又被称为面向数据流的开发方法，或面向过程的开发方法，其基本思想是软件功能的分解和抽象，用系统工程的思想 and 工程化的方法，按用户至上的原则，结构化、模块化、自顶向下地对系统进行分析 and 设计的方法。

141、面向对象开发方法

面向对象开发方法（Object-Oriented Development Methodology）是一种将现实世界中的事物抽象成对象，通过对象之间的交互来实现程序功能的软件开发方法。其核心思想是参照人类认识系统的方法，通过对象、类、继承和多态等概念来设计和实现软件系统。

142、低代码

低代码通过可视化编程，具有不同经验、水平的开发人员可以通过图形化用户界面，使用拖拽组件和模型驱动的逻辑来创建网页和移动应用程序。低代码能够在合适的业务场景下实现大幅度的提效降本。

（简单：代码=可视化+少量或0代码+快速生成）。

低代码开发平台（LCDP）是无需编码（0代码）或通过少量代码就可以快速生成应用程序的开发平台。

143、过程改进

为了满足新的目标，过程拥有者对过程所执行的一系列改进活动。包括对已有某个组织内软件过程的标识、分析和改变。

过程改进是一种组织管理的方法，涉及对软件开发过程的优化和改进，旨在通过一系列活动、方法、实践及转换来提高工作流程效率、降低成本、提高用户满意度。

144、保护性活动

保护性活动是软件开发团队为了维护完整开发任务的进度、质量、更改和风险而遵循的一系列步骤或过程。

145、过程流

过程流描述了在执行顺序和时间上如何组织框架中的活动任务，执行顺序和时间的安排直接影响到里程碑、基线和交付物的设置。分为：线性过程流、迭代过程流、演化过程流。

146、V 模型

V 模型是瀑布模型的变体，编码（coding）构成了 V 点，左边是分析和设计，右边是测试和维护，它表达了测试与分析设计的对应相关，该模型更明确地说明了隐藏在瀑布描绘中的一些迭代和返工。

147、需求工程

需求工程是一个涵盖创建和维护系统需求文档所必需的一切活动的过程。

需求工程是指应用已证实有效的技术、方法进行需求分析，确定客户需求，帮助分析人员理解问题并定义目标系统的所有外部特征的一门学科。

需求工程包括需求开发和需求管理。

需求工程包括需求获取、分析建模、需求规约、需求验证等基本活动。

148、ISO9000

ISO9000 是一个关于质量管理体系的标准族，用于帮助组织确保满足客户和利益相关者的需求。

作用：指导组织建立和改进质量体系、质量体系评估

149、SPICE (ISO15504)

1993 年，由 ISO、IEC、JTC1 发起了 ISO15504 标准的制定项目，项目名称为“软件过程改进和能力测定 (Software Process Improvement and Capability dEtermination)”，常简称为 SPICE。

150、Six Sigma (6σ) 管理

6σ管理是通过减少波动，不断创新，达到缺陷逼近百万分之三点四的质量水平，以实现顾客满意和最大收益的系统科学。

151、模块

模块在程序中是数据说明、可执行语句等程序对象的集合,或者是单独命名和编址的元素,如高级语言中的过程、函数和子程序等。在软件的体系结构中,模块是可组合、分解和更换的单元。

152、模块化

模块化是指解决一个复杂问题时自顶向下逐层把系统划分成若干模块的过程,有多种属性,分别反映其内部特性。每个模块完成一个特定的子功能,所有的模块按某种方法组装起来,成为一个整体,完成整个系统所要求的功能。

153、继承性

继承是子类自动地共享基类中定义的数据和方法的机制。

继承性是子类自动共享父类数据结构和方法的机制,这是类之间的一种关系。在定义和实现一个类的时候,可以在一个已经存在的类的基础之上来进行,把这个已经存在的类所定义的内容作为自己的内容,并加入若干新的内容。

154、过程设计语言 (PDL)

也称程序描述语言,又称为伪码,是一种用于描述模块算法设计和处理细节的语言。

155、可移植性

可移植性指程序从一个计算机环境移植到另一个计算机环境的容易程度。

156、渐增式测试

渐增式是把未经过测试的模块逐个组装到已经过测试的模块上进行集成测试,每增加一个模块进行一次集成测试,重复此过程直至完成集成测试。

157、详细设计(过程设计)

详细设计也称过程设计,是在概要设计的基础上,设计出软件各模块内部的详细算法,并精确地表达这些算法。一般采用过程设计的表达工具给出适当的算法描述。常见的详细设计工具:自然语言描述、程序流程图、N-S流程图、判定树、判定表、PDL(伪代码)等。

158、软件概要设计(总体设计)

进入了设计阶段,要把软件“做什么”的逻辑模型变换为“怎么做”的物理模型,即着手实现软件的需求,并将设计的结果反映在“设计规格说明书”文档中,所以软件设计是一个把软件需求转换为软件表示的过程,最初这种表示只是描述了软件的总的体系结构,称为软件概要设计(总体设计)或结构设计。

159、程序设计风格

程序设计风格是指一个人编制程序时所表现出来的特点、习惯、思路等。

160、软件开发环境

软件开发环境指在计算机的基本软件的基础上,为了支持软件的开发而提供的一组工具软件系统。

161、错误推测法

在测试程序时,人们可能根据经验或直觉推测程序中可能存在的各种错误,从而有针对性地编写检查这些错误的测试用例的方法。

162、软件工程过程

软件工程过程规定了获取、供应、开发、操作和维护时,要实施的过程、活动和任务。

163、PAD(问题分析图)

PAD 是指问题分析图,是一种算法描述工具,它是一种由左往右展开的二维型树结构。

164、增量提交

增量提交是指在项目开发周期内,以一定的时间间隔增量方式向用户提交 工作软件及相应文档。

165、技术可行性

技术可行性对要开发项目的功能、性能、限制条件进行分析,确定在现有的资源条件 下,技术风险有多大,项目是否能实现。

166、系统流程图

系统流程图是描绘物理系统的传统工具,它用图形符号来表示系统中的各个元素,例 如人工处理、数据处理、数据库、文件、设备等。它表达了系统中各个元素之间 的信息流动的情况。

167、结构化设计

结构化设计是根据模块独立性准则、 软件结构准则将数据流图转换为软件的体系结构,用软件结构图来建立系统的物理模型,实现系统的概要设计。

168、喷泉模型

喷泉模型是一种以用户需求为动力,以对象作为驱动力的模型,适合于面向 对象的开发方法。它克服了瀑布模型不支持软件重用和多项开发活动集成的局限性。喷泉模型使开发过程具有迭代性和无间隙性。系统某些部分常常重复工作多次,相关功能在每次迭代中随之加入演化的系统。无间隙是指在分析、设计、实现等开发活动之间不存在明显的边界。

169、风险分析

风险分析实际上就是贯穿在软件工程上的一系列风险管理步骤,其中包括 风险识别、风险估计、风险管理策略、风险解决和风险监督,它能让人们主动 “攻击”风险。

170、经济可行性

对组织的经济状况和投资能力进行分析，对系统建设，运行和维护费用进行估算，对系统建成后可能取得的社会及经济效益进行估计。

171、面向对象设计

面向对象设计是把分析阶段得到的需求转变成符合成本和质量要求的、抽象的系统实现方案的过程。或者说，面向对象设计就是用面向对象观点建立求解域模型的过程。

172、判定条件覆盖

判定条件覆盖指设计足够的测试用例，使得判定表达式中的每个条件的所有可能取值至少出现一次，并使每个判定表达式所有可能的结果也至少出现一次。

173、版本控制

版本控制是一种用于管理和追踪软件开发过程中代码变更的工具。通过版本控制系统，开发团队能够协同合作，记录和管理代码的修改历史，并可以回溯到之前的版本。常见的版本控制工具包括 Git 和 SVN 等。

174、软件复用

将已有软件的各种有关知识用于建立新的软件，以缩减软件开发和维护的花费。软件复用是提高软件生产力和质量的一种重要技术。早期的软件复用主要是代码级复用，被复用的知识专指程序，后来扩大到包括领域知识、开发经验、设计决定、体系结构、需求、设计、代码和文档等一切有关方面。

软件复用颗粒度包含：代码和设计拷贝、源代码复用、设计和软件体系结构复用、应用程序生成器的使用、领域特定的软件体系结构的复用等。

175、测试用例

为某个特殊目标而编制的一组测试输入、执行条件以及预期结果，以便测试某个程序路径或核实是否满足某个特定需求。

176、信息隐蔽

将每个程序的成分隐蔽或封装在一个单一的设计模块中,定义每一个模块时尽可能少地显露其内部的处理。

177、McCabe 度量法

由 Thomas McCabe 提出的一种基于程序控制流的复杂性度量方法。

178、软件调试

软件测试成功后，根据错误迹象确定错误的原因和准确位置，并加以改正。

179、软件测试

软件测试是使用人工或自动的手段来运行或测定某个软件系统的过程，其目的在于检验它是否满足规定的要求或弄清

预期结果与实际结果之间的差别。从是否关心软件内部结构和具体实现的角度划分，测试方法主要有白盒测试和黑盒测试。

三、简答题（100 题）

1.简述软件安全性与软件可靠性的异同。

相同点：软件安全性与软件可靠性均与软件失效有关。

不同点：软件可靠性使用统计分析的方法来确定软件失效发生的可能性,而失效的发生未必导致灾难或灾祸.软件安全则考察失效会导致灾难发生的条件.

2.简述软件开发的可行性研究的目的和内容。

可行性研究是指对一个计划、项目或想法进行详尽的调查和评估，以确定其是否可行或可实施。

可行性研究的目的在于确定项目是否有实施的必要性和是否具备实施的条件（如技术、资源、法律等）。

内容涵盖技术可行性（能否实现功能）、经济可行性（成本与收益是否合理）、社会可行性（是否符合道德和社会影响）、法律可行性（是否合规）。

作用：对于确保项目的顺利实施和投资回报有着至关重要的作用

3.什么是面向数据流的设计，数据流包括哪几种？

面向数据流的设计方法把信息流映射成软件结构，信息流的类型决定了映射的方法，根据系统的数据流进行的设计，又称结构化设计

数据流包括：变换流、事务流

1. 变换流

根据基本系统模型，信息通常以外部世界的形式进入软件系统，经过处理以后再以外部世界的形式离开系统。

2. 事务流

基本系统模型意味着变换流，因此，原则上所有信息流都可以归结为这一类。但是当数据流是以事务为中心的，也就是说，数据沿输入通路到达一个处理 T，这个处理根据输入数据的类型在若干个动作序列中选出一个来执行。这类数据流应该划为一类特殊的数据流，称为事务流。

4.什么时候软件测试结束，为什么？

软件测试的结束标准通常是根据项目的需求和目标来确定的。以下是一些通用的测试结束标准：

1.所有测试用例都已执行：确保所有计划的测试用例都被执行，包括功能测试、性能测试、安全测试等各种类型的测试。

2.所有缺陷已修复：所有已识别的缺陷都被记录、分类和追踪，并且经过验证的已经被修复。

3.测试覆盖率满足要求：根据项目的要求，确保测试覆盖了系统的各个方面，包括功能、性能、安全等。

4.性能满足要求：如果项目有性能要求，确保系统在负载和性能方面达到了这些要求。

5.安全测试通过：如果项目涉及安全性要求，确保系统在安全性方面通过了测试，并没有明显的漏洞或弱点。

6.用户验收测试通过：如果有用户验收测试，确保系统满足用户的期望和需求。

7.测试报告和文档已准备完毕：确保所有必要的测试文档，包括测试计划、测试用例、测试报告等已经准备完毕。

8.测试团队和利益相关者已确认：确保测试团队和利益相关者都已经确认测试的结束，并且对测试结果满意。

9.上线准备就绪：确保系统已准备好上线，包括备份和恢复计划、数据迁移、培训和支持等。

10.风险评估已完成：对项目的风险进行评估，确保已经采取措施来降低或管理这些风险。

5.什么是对象，对象有哪几种形式？

对象是面向对象编程中的基本概念，它是对现实世界中的实体、事物或概念的抽象表示。对象具有以下几种形式：

类：类是对象的模板，它定义了对象的属性和方法，但并不占用实际的空间。

实例：实例是类的具体化，它根据类的定义创建了对象，并占用实际的空间。

属性：属性是对象的特征，它描述了对象的状态或数据。

方法：方法是对象的行为，它描述了对象可以做什么或如何改变自己的状态。

链：链是对象之间的关系，它描述了对象如何相互影响或通信。

6.什么是 McCabe 复杂性度量，如何计算？

是利用程序模块的程序图中环路的个数，来计算程序的复杂性的。为此，也称为环路复杂度算法。

计算方法：

$$V(G)=m-n+p$$

$V(G)$ 是有向图 G 中环路数， m 是图 G 中弧数， n 是图 G 中结点数， p 是图 G 中强连通分量个数。

7.软件项目管理的主要方面是什么？

- 1) 软件项目计划与组织
- 2) 软件项目成本管理
- 3) 软件项目进度控制
- 4) 软件质量保证
- 5) 软件配置管理
- 6) 生成软件项目管理文档

8.怎样完成从数据流图到程序结构图的转换？

从以下 4 个步骤进行：

- 1.重画数据流图
- 2.区分有效（逻辑）输入、有效（逻辑）输出和中心变换部分
- 3.进行一级分解，设计上层模块
- 4.进行二级分解，设计中、下层模块

9.如何估算程序中的错误？

可用下面的估算方法来计算：

- 1)用 Shooman 模型估算 ET
- 2)用植入法估算固有错误总数
- 3)用最小二乘法估算固有错误

10.什么是类，类与对象间是什么关系？

类是对一类事物的抽象描述，它定义了事物的属性和方法，但并不占用实际的空间。对象是类的具体化，它根据类的定义创建了事物，并占用实际的空间。类与对象之间的关系可以总结为以下几点：

类是对象的模板，对象是类的实例。例如，人是一个类，小韩、小石等是人的对象。

类描述了对象的共同特征，对象表示了类的个体差异。例如，人有名字、性别、年龄等属性，小韩、小石等有不同名字、性别、年龄等值。

类可以创建多个对象，对象只属于一个类。例如，人可以创建小韩、小石等多个对象，小韩、小石等只属于人这个类。

11.软件的缺陷为什么在软件开发和维护过程中会扩大？

软件的缺陷是指软件产品与需求或规格不一致的地方，它会影响软件的功能、性能、可靠性等。软件缺陷在软件开发和维护过程中会扩大的原因有以下几点：

需求不清晰或频繁变更。如果需求没有明确地定义或者在开发过程中不断地修改，会导致软件产品与需求之间的差距越来越大，从而产生更多的缺陷。

设计不合理或编码错误。如果软件的设计没有考虑到各种情况或者与需求不匹配，或者编码过程中出现逻辑错误或语法错误，会导致软件产品无法正常运行或出现异常，从而产生更多的缺陷。

测试不充分或环境差异。如果软件的测试没有覆盖到所有的功能、场景或边界条件，或者测试环境与实际运行环境有差异，会导致软件产品在实际使用中出现问题，从而产生更多的缺陷。

12.成本估算方法有哪几种？

成本估算方法 就方法论而言，有两种基本的成本估算方法：自顶向下和自底向上。

这两种方法都要求采用某种方法做出估算,可以利用的估算方法有：

- 1、工作分解结构方法（WBS）
- 2、专家判断
- 3、基于问题的估算
- 4、基于经验模型的估算

13.面向数据流的设计方法包含那些步骤？

- 1) 确定数据流的类型：变换流还是事务流
- 2) 划定流界
- 3) 将数据流图转换为软件结构
- 4) 通过设计复审和启发式策略精化所得到软件结构

14.什么叫测试用例，如何设计测试用例

测试用例（Test Case）是为某个特殊目标而编制的一组**测试输入、执行条件以及预期结果**，以便测试某个程序路径或核实是否满足某个特定需求。

测试用例的设计方法有多种，比如：

等价类划分法。把输入数据或操作条件分成有效等价类和无效等价类，从每个等价类中选取代表性的值作为测试数据。

边界值分析法。在等价类划分的基础上，针对输入数据或操作条件的边界值或极限值进行测试。

错误推测法。根据经验或直觉，推测可能出现错误的地方，设计相应的测试用例。

判定表法。当一个界面中有多个控件，不同控件的组合会得到不同的结果时，用表格的形式列出所有可能的组合和对应的结果。

因果图法。当输入条件或输出条件的组合比较多时，用图形的方式分析判定表，简化测试用例的设计。

正交实验法。当输入条件或输出条件有多个因素和水平时，用数学概率学知识，设计最经济的实验路径。

场景法。模拟用户操作场景，测试多个功能组合。

流程图法。用流程图的方式展现基于用户使用场景的测试步骤。

15.面向对象方法特征有哪些？

封装性、继承性、多态性、抽象

封装。把数据和方法封装在内部，构成一个整体功能，只提供某个方法给外面访问。

继承。继承是解决代码的复用，是类和类之间的关系，使得子类具备父类中得方法。

多态。多态是指同一个方法在不同的对象上有不同的表现形式或行为。

抽象。抽象是将有关事物的共性归纳、集中的过程，忽略不重要的细节。

16.什么是 CMM 关键过程域，举例说明其含义。

关键过程域是指一系列相互关联的操作活动，这些活动反映了一个软件组织改进过程时必须集中精力改进的几个方面。换句话说，关键过程域标识了达到某个成熟度等级时所必须满足的条件。

如软件成熟度模型中第 4 级（管理级）有 2 个关键过程域，主要的任务是为软件过程和软件产品建立一种可以理解的定量的方式。

17.影响软件成本的主要因素有哪些具体方面？

软件的规模、复杂度、开发工具、开发人员的业务素质、软件工程管理水平和开发工作环境等。

18.画数据流图原则是什么？

- (1) 明确系统界面。
- (2) 自顶向下逐层扩展。
- (3) 合理布局。

(4) 数据流图绘制过程，就是系统的逻辑模型的形成过程，必须始终与用户密切接触，详细讨论，不断修改，也要和其他系统建设者共同商讨一求一致意见。

19.什么叫基本路径测试，如何确定基本路径？

实际程序中路径是个很庞大的数字，所有路径覆盖是不现实的，测试中把路径数压缩在一定范围内，称为基本路径测试。

在程序控制流图的基础上，通过分析控制构造的环路复杂性测度，导出基本可执行路径的基本集合。

20.简述 USE CASE 的作用。**测试用例：**

在不展现一个系统或子系统内部结构的情况下，对系统或子系统的某个连贯的功能单元的定义和描述。其实 Use Case 就是对系统功能的描述而已，不过一个 Use Case 描述的是整个系统功能的一部分，这一部分一定要是在逻辑上相对完整的功能流程。

测试用例是为某个特殊目标而编制的一组测试输入、执行条件以及预期结果，以便测试某个程序路径或核实是否满足某个特定需求。

测试用例的作用：

- 1，通过一个用例来证明被测软件的某功能符合需求说明书中规定的要求，可以通过设计正反两方面的测试用例来验证
- 2，可以保证一个软件被测试的有效性，使测试人员知道哪些些功能以被测，哪些功能还需要测试，从而避免漏测，重复测，提高测试效率
- 3，指导测试的工作，保证测试是有计划的实施，而不是随意性的测试
- 4，为公司留下财富，为后期软件维护提高帮助，为公司新人进来提供指导，在测试的时候可以尽量把人为因素的影响减少到最小。保障软件测试质量的稳定
- 5，可以作为评估测试结果的，为编写测试报告提供依据。
- 6，分析缺陷的标准，通过收集缺陷，对比测试用例和缺陷数据库，分析确证是漏测还是缺陷复现。漏测反映了测试用例的不完善，应立即补充相应测试用例，最终达到逐步完善软件质量。而已有相应测试用例，则反映实施测试或变更处理存在问题。

21.软件质量的特性有哪些，怎样度量？

功能性、可靠性、易用性、效率、可维护性、可移植性。

通过产品质量、过程中质量和软件维护质量度量等三种方法来度量。

22.什么是 COCOMO 成本估算模型？

是一种利用经验模型进行成本估算的方法

COCOMO，英文全称为 constructive cost model，中文为构造性成本模型。它是一种精确、易于使用的，基于模型的成本估算方法，最早由勃姆（Boehm）于 1981 年提出。从本质上说是一种参数化的项目估算方法，参数建模是把项目的某些特征作为参数，通过建立一个数字模型预测项目成本（类似于居住面积作为参数计算的整体的住房成本）。

23.数据流图作用是什么？

DFD 是**描述数据处理的工具**。数据流图从数据传递和加工的角度，以图形的方式刻画数据流从输入到输出的移动变换过程。从而便于需求分析。

24.简述回溯测试和作用。

回溯法采用试错的思想，它尝试分步的去解决一个问题。在分步解决问题的过程中，当它通过尝试发现现有的分步答案不能得到有效的正确的解答的时候，它将取消上一步甚至是上几步的计算，再通过其它的可能的分步解答再次尝试寻找问题的答案。回溯法通常用最简单的递归方法来实现，在反复重复上述的步骤后可能出现两种情况：

- 1、找到一个可能存在的正确的答案
 - 2、在尝试了所有可能的分步方法后宣告该问题没有答案
- 作用：
- 1、找出满足约束条件的所有解。

25.面向对象设计涉及那几个主要活动？

分析活动、设计活动、编码活动

26.Boehm 和 McCall 等人从哪些方面来确定软件质量，他们有什么异同？

软件质量就是“软件与明确的和隐含的定義的需求相一致的程度”。具体地说，软件质量是软件符合明确叙述的功能和性能需求、文档中明确描述的开发标准、以及所有专业开发的软件都应具有的隐含特征的程度。影响软件质量的主要因素，这些因素是从管理角度对软件质量的度量。可划分为三组，分别反应用户在使用软件产品时的三种观点。正确性、健壮性、效率、完整性、可用性、风险（产品运行）；可理解性、可维修性、灵活性、可测试性（产品修改）；可移植性、可再用性、互运行性（产品转移）。

McCall 等人将质量模型分为三层因素、衡量准则、度量,并对软件质量因素进行了研究,认为软件质量是正确性、可靠性、效率等构成的函数,而正确性、可靠性、效率等被称为软件质量因素,或软件质量特征,它表现了系统可见的行为化特征。每一因素又由一些准则来衡量,而准则是跟软件产品和设计相关的质量特征的属性。

Boehm 模型是由 Boehm 等在 1978 年提出来的质量模型,在表达质量特征的层次性上它与 McCall 模型是非常类似的。不过,它是基于更为广泛的一系列质量特征,它将这些特征最终合并成 19 个标准。Boehm 提出的概念的成功之处在于它**包含了硬件性能的特征**,这在 McCall 模型中是没有的。但是,其中与 McCall 模型类似的问题依然存在。

27.什么是软件的逆向工程和再工程？

软件逆向工程是通过反汇编和调试等手段分析计算机程序的二进制可执行代码，从而获得程序的算法细节和实现原理的技术

软件再工程是对既存软件系统进行调查，**并将其重构为新形式代码的开发过程**。

28.简述数据字典的作用

数据字典是存储有关数据的来源、说明、与其他数据的关系、用途和格式等信息的数据库。**数据字典的作用是：**

- 提供系统的逻辑模型，与数据流图配合使用。
- 规范数据的定义和命名，保证数据的一致性和完整性。
- 帮助开发人员和用户了解系统的数据结构和流程。
- 方便管理和维护下拉框等表单元素的选项。

29.纠错技术中的归纳法和演绎法的中心思想是什么？

1.归纳法，指的是从许多个别事例中获得一个较具概括性的规则。这种方法主要是从收集到的既有资料，加以抽丝剥茧地分析，最后得以做出一个概括性的结论。演绎法，则与归纳法相反，是从既有的普遍性结论或一般性事理，推导出个别性结论的一种方法。由较大范围，逐步缩小到所需的特定范围。

2.归纳法是从特殊到一般，优点是能体现众多事物的根本规律，且能体现事物的共性。缺点是容易犯不完全归纳的毛病。演绎法是从一般到特殊，优点是由定义根本规律等出发一步步递推，逻辑严密结论可靠，且能体现事物的特性。缺点是缩小了范围，使根本规律的作用得不到充分的展现。归纳法和演绎法在应用上并不矛盾，有些问题可采用前者，有些则采用后者。而更多情况，将两者结合着应用，则能收到更好的效果。

30.面向对象设计如何体现抽象信息、隐藏和模块化这三个概念的？

通过类的引入，封装,把数据结构和操作这些数据的方法紧密地结合在一起所构成的模块.对象的封装性实现隐藏。

31.影响软件质量的因素分哪两大类？（或影响软件质量的因素有哪些？）*

- (1) 可直接度量的因素：如单时间内千行代码中所产生的错误数
- (2) 不可直接度量的因素：如可用性可维护性等

软件质量因素集中在产生的三个重要方面：

- 软件的操作特性
- 软件承受变化的能力
- 对新环境的适应性

32.如何衡量软件成本，各表示什么含义？ *

成本估算和成本管理是软件项目的核心任务之一。在制定项目计划时，就必须对项目需要的人力及其他资源、项目持续时间和项目成本做出估算。如果新项目和以往的项目类似，估算可以参考以前的成本费用。

软件工程中，软件成本的衡量方法主要有以下几种：

- 功能点法：根据软件的功能复杂度和规模，计算软件的功能点数，再乘以每个功能点的成本，得到软件的总成本。
- COCOMO 法：根据软件的类型、规模、开发难度等因素，使用数学模型估算软件的开发时间和人力，再乘以每人每月的成本，得到软件的总成本。
- 专家判断法：根据专家的经验 and 知识，对软件的开发过程和资源进行评估，给出软件的总成本。
- 类比法：根据与当前项目相似的历史项目的数据，对软件的开发过程和资源进行比较和调整，得到软件的总成本。

33.需求分析的任务是什么？ *

需求分析是将用户需求陈述转化为完整的需求定义，继而形成规范化的需求规格说明的过程。其基本任务是准确定义新系统的功能，主要任务有：

- (1)需求分析是借助于当前系统导出目标系统的逻辑模型，解决目标系统做什么；
- (2)深入描述软件的功能和性能，确定软件设计的限制和软件同其它系统元素的接口细节，定义软件的其它有效性需求；
- (3)软件需求规格说明还为评价软件质量提供依据；

34.什么是因果图，用于何处？ *

因果图是一种表示因果关系的图形工具，用于分析和解决问题。

因果图是根据分析时对各种条件可能产生的结果的一种对照分析法。

因果图方法是根据 输入与输出之间的因果关系来设计用例的。

其测试用例的基本依据可以是分析阶段的决策表

根据因果关系建立因果关系图

建立决策表获得用例

35.举例说明什么是类的整体部分结构？ *

如建立一个时钟类，这个类中的属性如秒、分钟、小时等，加上对属性的操作如秒计时，分钟计时，小时计时等，就是一个类的整体部分结构。

36.软件质量的含义是什么？

软件质量是软件特性的综合，指软件满足规定或潜在用户需求的能力。

- **内部质量**：指软件产品本身的质量，如代码的可读性、可维护性、可测试性等，它可以通过静态分析和代码审查等方法来评估。
- **外部质量**：指软件产品在运行时的质量，如功能性、可靠性、效率、可用性等，它可以通过动态测试和用户反馈等方法来评估。
- **使用质量**：指软件产品在特定环境和条件下的质量，如满足性、安全性、适应性等，它可以通过用户调查和市场分析等方法来评估。
- **过程质量**：指软件开发过程中遵循的标准和规范的质量，如计划、管理、监控、改进等，它可以通过过程评估和过程改进等方法来评估。

37.如何安排软件开发进度，应该考虑哪些问题？

软件工程中，软件开发进度的安排是一个重要的管理活动，应该考虑以下几个问题：

- **软件开发的目标和范围**：明确软件要实现的功能、性能、质量等需求，以及软件的交付时间和成本等约束。
- **软件开发的流程和阶段**：选择合适的软件开发模型，如瀑布模型、迭代模型、敏捷模型等，划分软件开发的主要阶段，如需求分析、设计、编程、测试、部署等，以及每个阶段的输入、输出、标准和评审等。
- **软件开发的资源和人员**：评估软件开发所需的硬件、软件、人力等资源，分配和协调项目团队的角色和职责，如项目经理、分析师、设计师、程序员、测试员等，以及他们之间的沟通和协作方式。
- **软件开发的风险和变更**：识别和分析软件开发过程中可能遇到的风险，如需求变更、技术难题、人员流失等，制定风险应对策略和变更控制机制，以及风险和变更的跟踪和报告方式。

38.什么是结构化分析，有什么特点？

是面向数据流进行需求的分析，遵循自顶向下、逐步求精原则。

主要特点是快速、自然和方便。成功率较高，发展较为成熟；简单、易掌握，适应于瀑布模型；特别适合于数据处理领域中的应用，对规模大的项目，特别复杂的应用不太适应。难于解决软件重用问题，难于适应需求的变化。

39.什么是边界值分析，与等价类划分有什么不同？

边界值分析法就是对输入或输出的边界值进行测试的一种黑盒测试方法。

- (1)、边界值分析不是从某等价类中随便挑一个作为代表，而是使这个等价类的每个边界都要作为测试条件。
- (2)、边界值分析不仅考虑输入条件，还要考虑输出空间产生的测试情况。

40.举例说明什么是类的组装结构关系

如建立一个时钟类，这个类中的属性如秒、分钟、小时等，加上对属性的操作如秒计时，分钟计时，小时计时等，就构成一个类的组装结构关系。

41.原型开发由哪些步骤？

原型开发是一种快速开发软件的方法，主要由以下几个步骤组成：

- **确认用户基本需求**，需求分析师与用户合作完成，可以不完整，易于描述，大型系统应该形成需求文档。
- **设计系统初始原型**，实现一个可运行的系统，原则如下：
 - ☐集成原则：利用现有的系统和原型构成，需要原型工具。
 - ☐最小原则：只实现最基本的功能，不考虑细节和优化。

⑦可变原则：保持系统的灵活性和可修改性，方便后续改进。

- 评估和修改原型，将原型展示给用户，收集用户的反馈和建议，根据用户的需求进行修改和完善，直到用户满意为止。
- 转换为目标系统，将原型转换为正式的软件系统，需要考虑系统的稳定性、可靠性、安全性、效率等方面。

42.什么是软件配置管理，有什么作用？

简单而言就是**管理软件的变化**，它应用于整个软件工程过程，通常由相应的工具、过程和方法学组成。

实施有效的软件配置管理可以解决以下软件开发中的常见问题：

- 1、开发人员未经授权修改代码或文档；
- 2、人员流动造成企业的软件核心技术泄密；
- 3、找不到某个文件的历史版本；
- 4、无法重现历史版本；
- 4、无法重新编译某个历史版本，使维护工作十分困难；
- 5、“合版本”时，开发冻结，造成进度延误；
- 6、软件系统复杂，编译速度慢，造成进度延误；
- 7、因一些特性无法按期完成而影响整个项目的进度或导致整个项目失败；
- 8、已修复的 Bug 在新版本中出现；
- 9、配置管理制度难于实施；
- 10、分处异地的开发团队难于协同，可能会造成重复工作，并导致系统集成困难；

43.原型开发技术的目的，特点是什么？

解决用户与开发者之间的鸿沟，以原型(软件产品的样品)为共同语言，实现用户与开发者双向沟通。

容易地确定系统的性能，确认各项主要系统服务的可应用性，确认系统设计的可行性，确认系统作为产品的结果。

软件开发人员和用户之间的理解偏差在功能展现时显露出来在原型设计中发现需求的不完善和不一致可迅速地展现一个简单的应用系统通过原型系统可以导出系统需求原型系统可以支持用户培训和系统测试。

44.简述在确定等价划分测试用例设计中遵循的原则。

等价划分测试用例设计是一种软件测试方法，它将软件的输入或输出域划分为若干个等价类，每个等价类代表了相同的功能或错误行为。然后从每个等价类中选取一个或几个代表性的值作为测试用例，以此来减少测试用例的数量，提高测试效率。

等价划分测试用例设计遵循以下原则：

- 有效等价类原则：有效等价类是指符合软件规格说明的输入或输出域，从有效等价类中选取的测试用例应该使软件正常运行。
- 无效等价类原则：无效等价类是指不符合软件规格说明的输入或输出域，从无效等价类中选取的测试用例应该使软件出现错误或异常。
- 单一故障假设原则：单一故障假设是指在一个测试用例中只包含一个错误，这样可以更容易地定位和修复错误。

45.领域分析的目标是什么，依据是什么？

领域分析的目标是理解和描述一个特定的问题域，包括它的概念、术语、规则、实体、关系、行为、过程等。领域分析的依据是问题域的需求和特征，以及相关的文献、资料、专家、用户等。领域分析可以帮助提高软件开发的效率和质量，以及促进软件复用和维护。

46.原型开发包括那几类，目的是什么？

- 1) 抛弃式原型开发是弄清需求和为风险评估提供额外的信息。
- 2) 演化式原型开发先给出一个系统的最初实现，让用户去使用和评价，不断进行细化和完善，经过多次反复形成最后完善的应用系统。
- 3) 递增式原型开发减少修改的范围，实现易于控制和管理。

47.基线在配置管理中有什么作用?

基线是软件文档或源码的一个稳定版本,它是进一步开发的基础。基线在配置管理中的作用是为开发软件提供了一个定点和快照,可以方便比较、评审、交接、检查、修复、发布等

48.简述总体设计的一般过程?

总体设计又称为概要设计或初步设计。总体设计过程通常由两个主要阶段组成:

系统设计:确定系统的具体实现方案。结构设计:确定软件结构。

9个步骤:

设想供选择的方案;选取合理的方案;推荐最佳方案;功能分解;设计软件结构;设计数据库;制定测试计划;书写文档;审查和复审

49.什么是软件测试中逻辑覆盖,一般软件测试至少应保证哪些覆盖?

是以程序内部的逻辑结构为基础的测试技术,是通过对程序逻辑结构的遍历实现程序的覆盖,这一方法要求测试人员对程序的逻辑结构有清楚的了解。至少应保证语句覆盖。

50.类的开发有几个途径,如何进行类的开发?

一是新建基类,二是通过类间的继承来进行类的开发。

51.原型具有什么特点?

- 1) 遵循了人们认识事物的客观规律,易于掌握和接受
- 2) 将模拟的手段引入系统分析的初始阶段,沟通了人们(用户和开发人员)的思想,缩短了用户和系统分析人员之间的距离,解决了结构化方法中最难于解决的一环。强调用户参与、描述、运行、沟通。
- 3) 充分利用最新的软件工具,摆脱了传统的方法,使系统开发的时间、费用大大地减少,效率、技术等方面都大大地提高。强调软件工具支持。

52.软件维护的基本内容是什么?(软件维护的类型)

软件维护的基本内容是指根据需求变化或硬件环境的变化对应用程序进行部分或全部的修改。软件维护一般分为以下四种类型:

- **改正性维护**:是指改正在系统开发阶段已发生而系统测试阶段尚未发现的错误。
- **适应性维护**:是指使应用软件适应信息技术变化和管理需求变化而进行的修改。
- **完善性维护**:是指为扩充功能和改善性能而进行的修改,主要是指对已有的软件系统增加一些在系统分析和设计阶段中没有规定的功能与性能特征。
- **预防性维护**:是为了改进应用软件的可靠性和可维护性,为了适应未来的软硬件环境的变化,应主动增加预防性的新的功能,以使应用系统适应各类变化而不被淘汰。

53.解释体系结构中深度、宽度、扇出、扇入对软件的影响。

体系结构中的深度、宽度、扇出、扇入是指**结构图**的一些特征,它们对软件的影响如下:

- **深度**:指结构图控制的层次,即模块的层数。深度越大,说明模块之间的调用关系越复杂,软件的可维护性和可测试性越低。
- **宽度**:指一层中最大的模块个数。宽度越大,说明模块之间的并行关系越多,软件的可理解性和可维护性越低。
- **扇出**:指一个模块直接下属模块的个数。扇出越大,说明模块之间的耦合度越高,软件的可复用性和可测试性越低。
- **扇入**:指一个模块直接上属模块的个数。扇入越大,说明模块之间的内聚度越高,软件的可复用性和可维护性越高。
- **设计原则**:低扇出高扇入,扇入大表示模块的重用性高,利用率高.扇出大表示模块的复杂度高.所以要高扇入低

扇出。

54.为什么说在确认测试阶段发现需求理解的错误，要修改就要付出更大的代价？

在确认测试阶段发现需求理解的错误，要修改就要付出更大的代价，是因为这样会导致以下问题：

- **需求变更：**如果需求理解错误，就需要重新修改需求文档，重新评审需求，重新制定开发计划，重新分配开发任务，重新编写代码，重新测试代码，重新部署上线。这些工作都会消耗大量的时间、人力和资源。
- **质量风险：**如果需求理解错误，就可能导致开发出来的产品不符合客户的期望和需求，或者存在严重的缺陷和漏洞。这些问题会影响产品的质量、性能、安全性和用户体验，甚至可能造成客户的损失和投诉。
- **信誉损失：**如果需求理解错误，就可能导致产品不能按时交付，或者交付的产品不满足客户的要求。这些情况会损害开发团队和公司的信誉和形象，降低客户的满意度和忠诚度，影响未来的合作机会。

因此，在确认测试阶段发现需求理解的错误，要修改就要付出更大的代价。所以，在开发过程中，应该尽量避免需求理解错误，通过有效的沟通、评审、文档、原型等方式，保证测试人员、开发人员和客户对需求的理解一致。

55.什么是 OOA 建模语言，它应该包括哪些方面？

OOA 建模语言是指用于进行面向对象分析（Object-Oriented Analysis）的语言，它可以描述系统的功能、结构、行为和约束等方面。OOA 建模语言应该包括以下方面：

- **类图：**用于描述系统中的类及其属性、方法和关系，如泛化、实现、关联、聚合、组合和依赖等。
- **用例图：**用于描述系统的功能需求，以及参与者和用例之间的关系，如包含、扩展和泛化等。
- **状态图：**用于描述系统中某个对象的状态变化和触发条件，以及状态之间的转移关系。
- **活动图：**用于描述系统中的业务流程或算法，以及活动之间的控制和数据流关系。
- **序列图：**用于描述系统中对象之间的交互过程，以及消息的发送和接收顺序。
- **协作图：**用于描述系统中对象之间的交互过程，以及对象的结构和位置关系。
- **组件图：**用于描述系统中的组件及其接口和依赖关系，以及组件之间的通信方式。
- **部署图：**用于描述系统中的物理结构，包括节点、设备、连接器和部署规范等。

56. 软件开发中，为什么要进行阶段评审？

进行阶段评审的主要原因包括以下几点：

确保项目达到既定目标和标准：阶段评审的主要目的是确保项目在每个阶段都能够达到既定的目标和标准。通过评审，可以及时发现和解决问题，保证项目的顺利进行。

提高项目质量和效率：阶段评审可以有效地发现项目中存在的问题，提出改进建议，为项目的后续工作提供有力支持。同时，评审工作也能够提高团队成员的工作积极性和责任感，促进团队的协作和沟通，提升项目的整体质量和效率。

风险管理：阶段评审关注产品或项目的整体开发进度、当前状态及潜在风险管理。通过评审，可以及时发现潜在的风险，并采取相应的措施进行管理，确保项目按计划顺利进行。

57.软件维护分几类，每类的内容是什么？

软件维护是指在软件交付后，对软件进行修改和更新，以适应用户需求的变化，修复软件缺陷，提高软件性能和质量等。

软件维护一般分为以下几类：

- **更正性维护：**更正性维护是指对软件中存在的错误或缺陷进行修复，使软件符合规格说明。
- **适应性维护：**适应性维护是指对软件进行修改，以适应外部环境的变化，如硬件、操作系统、协议等。
- **完善性维护：**完善性维护是指对软件进行修改，以增加或改进软件的功能、性能、可用性等，满足用户的新需求或期望。
- **预防性维护：**预防性维护是指对软件进行修改，以预防未来可能出现的问题，提高软件的可维护性和可扩展性。

58.什么叫模块化，为什么要划分模块？

模块化是一种软件设计和开发的方法，它将软件系统划分为若干个相对独立的、功能完整的、可复用的模块，每个

模块负责一定的功能，模块之间通过接口进行通信和协作。

模块化的目的主要有以下几个：

- 提高软件的可理解性和可维护性，使软件结构清晰，便于分析、设计、编码、测试和修改。
- 提高软件的可复用性和可扩展性，使软件可以重用已有的模块，或者增加新的模块，以适应新的需求或功能。
- 提高软件的可靠性和安全性，使软件可以隔离和控制模块之间的错误或异常，防止错误或异常的传播和影响。
- 提高软件的开发效率和协作效率，使软件可以分工合作，实现并行开发和集成。

59.比较在组装测试中，自顶向下与自底向上、深度优先与宽度优先的优缺点。

自顶向下

优点：

- 1) 较早地验证主要的控制和判断点。
- 2) 可以首先实现和验证一个完整的软件功能，增强信心；
- 3) 减少了测试驱动程序开发和维护的费用；
- 4) 可以和开发设计工作一起并行执行集成测试，能够灵活的适应目标环境；
- 5) 容易进行故障隔离和错误定位。

缺点：桩模块的开发和维护费用大。

自底向上

优点

- 1) 减少了桩模块的工作量；
- 2) 容易对错误进行定位。

缺点

- 1) 直到最后一个模块加进去之后才能看到整个系统的框架；
- 2) 只有到测试过程的后期才能发现时序问题和资源竞争问题；
- 3) 驱动模块的设计工作量大；
- 4) 不能被及时发现高层模块设计上的错误。

自底向上的组装测试方案是工程实践中最常用的测试方法。相关技术也较为成熟。它的优点很明显：管理方便、测试人员能较好地锁定软件故障所在位置。但它对于某些开发模式不适用，如使用 XP 开发方法，它会要求测试人员在全部软件单元实现之前完成核心软件部件的组装测试。尽管如此，自底向上的组装测试方法仍不失为一个可供参考的组装测试方案。

自顶向下集成的优点在于能尽早地对程序的主要控制和决策机制进行检验，因此较早地发现错误。缺点是在测试较高层模块时，低层处理采用桩模块替代，不能反映真实情况，重要数据不能及时回送到上层模块，因此测试并不充分。

自底向上集成方法不用桩模块，测试用例的设计亦相对简单，但缺点是程序最后一个模块加入时才具有整体形象。它与自顶向综合测试方法优缺点正好相反。

60.简述 RUP (Rational Unified Process) 的基本内容

RUP (Rational Unified Process) 是一种面向对象且基于网络的程序开发方法论，由 Rational 公司提出，是一种软件开发过程框架，基于面向对象符号体系给出了有关软件开发过程组织及实施的指导

迭代式开发。RUP 主张在软件开发的早期阶段就想完全、准确的捕获用户的需求几乎是不可能的，因此采用迭代式开发，将软件生命周期分解为多个迭代周期，每个迭代周期都包含需求分析、设计、实现和测试等活动。

用例驱动。RUP 以用例为核心，用用例来描述系统的功能需求，用用例来驱动系统的分析、设计、实现和测试。

以架构为中心。RUP 认为系统架构是软件开发过程中最重要的工件，它决定了系统的质量属性和可维护性，因此要在早期阶段就确定系统架构，并在后续阶段不断地验证和完善。

可配置。RUP 是一个通用的过程模板，包含了很多开发指南、制品、开发过程所涉及到的角色说明，由于它非常庞大所以对具体的开发机构和项目，用 RUP 时还要做裁剪，也就是要对 RUP 进行配置。

61.简述软件生命周期模型在软件开发中的作用。

软件生命周期是软件工程的一个重要的概念。把整个软件生命周期划分为若干个较小的阶段,每个阶段都有相对独立的任务和完成任务的步骤和方法,然后逐步完成各个阶段的任务,这有利于软件开发过程的组织和管理,从而降低了整

个软件开发过程的困难程度,从而使规模庞大、结构复杂和管理复杂的软件开发变得容易控制和管理。

62.软件维护有哪些特点?

软件维护是指在软件交付后,对软件进行修改和更新,以适应用户需求的变化,修复软件缺陷,提高软件性能和质量等。软件维护有以下几个特点:

- 软件维护是一个持续的过程,不是一个一次性的事件,软件维护的周期通常比软件开发的周期长。
- 软件维护是一个复杂的过程,涉及到多方面的因素,如用户、开发者、管理者、环境等,需要进行有效的沟通和协调。
- 软件维护是一个成本高昂的过程,据统计,软件维护的成本占软件生命周期成本的 60%~80%。
- 软件维护是一个风险较大的过程,每次修改都可能引入新的错误或影响原有的功能,需要进行充分的测试和验证。

63.模块化的三个重要特征是什么,阐明各自的作用。

- 1.相对独立性,可以对模块单独进行设计、制造、调试、修改和存储,这便于由不同的专业化企业分别进行生产;
- 2.互换性,模块接口部位的结构、尺寸和参数标准化,容易实现模块间的互换,从而使模块满足更大数量的不同产品的需要;
- 3.通用性,有利于实现横系列、纵系列产品间的模块的通用,实现跨系列产品间的模块的通用。

64.单元测试时,为什么需要开发驱动模块和桩模块?

这是因为在开发设计的初级阶段,完成一个单元的开发任务后,这个单元相关联上面、下面的模块尚未开发出来,为了要测试所开发单元,所以要驱动模块和桩模块

65.什么是 Use Cases Model, 主要作用是什么?

是系统既定功能及系统环境的模型

用例模型最重要的作用是将系统行为传达给客户或最终用户。

66.什么是需求工程?

需求工程是指应用已证实有效的技术、方法进行需求分析,确定客户需求,帮助分析人员理解问题并定义目标系统的所有外部特征的一门学科。

它通过合适的工具和记号系统地描述待开发系统及其行为特征和相关约束,形成需求文档,并对用户不断变化的需求演进给予支持。RE 可分为系统需求工程(如果是针对由软硬件共同组成的整个系统)和软件需求工程(如果仅是专门针对纯软件部分)。软件需求工程是一门分析并记录软件需求的学科,它把系统需求分解成一些主要的子系统和任务,把这些子系统或任务分配给软件,并通过一系列重复的分析、设计、比较研究、原型开发过程把这些系统需求转换成软件的需求描述和一些性能参数。

需求工程是一个不断反复的需求定义、文档记录、需求演进的过程,并最终在验证的基础上冻结需求。80 年代,HerbKrasner 定义了需求工程的五阶段生命周期:需求定义和分析、需求决策、形成需求规格、需求实现与验证、需求演进管理。近来,MatthiasJarke 和 KlausPohl 提出了三阶段周期的说法:获取、表示和验证。

67.简述软件的维护过程。

软件维护的过程一般包括以下几个步骤:

- 接收和分析维护请求,维护人员收到用户或管理者提出的维护请求,对请求进行分析,确定维护的类型、范围、优先级、可行性等。
- 制定和评估维护方案,维护人员根据维护请求,制定一个或多个维护方案,包括维护的目标、方法、工具、资源、时间、成本等,并对方案进行评估和选择。
- 执行和测试维护方案,维护人员按照选定的维护方案,对软件进行修改和更新,并对修改后的软件进行测试和验证,确保软件的正确性和完整性。
- 交付和评价维护结果,维护人员将修改后的软件交付给用户或管理者,并收集用户或管理者对维护结果的反

馈和评价，以及对维护过程的改进意见。

68.简述层次方框图与软件结构图的异同点。

相同：形式类似。

不同：

- 1,层次图描绘软件的层次结构,层次方框图描绘的是数据结构。
- 2,层次图的方框表示模块或子模块。层次方框图的方框表示数据结构整体或其子集。
- 3,层次图的连线表示调用关系，层次方框图表示组成关系。

69.简述黑盒子测试和白盒子测试及其适用性。

黑盒子测试是一种把程序看做一个黑盒子，完全不考虑程序内部结构和处理过程的测试方法。黑盒子测试是对程序的接口进行测试，它只检验程序功能是否能够要预定那样正常的工作。

黑盒子测试力图发现以下问题：

- 1) 功能是否不正确或者不符合要求
- 2) 界面是否错误
- 3) 数据结构错误或外部数据库访问错误
- 4) 性能不符合

白盒测试也称结构测试或逻辑驱动测试，它是按照程序内部的结构测试程序，通过测试来检测产品内部动作是否按照设计规格说明书的规定正常进行，检验程序中的每条通路是否都能按预定要求正确工作。这一方法是把测试对象看作一个打开的盒子，测试人员依据程序内部逻辑结构相关信息，设计或选择测试用例，对程序所有逻辑路径进行测试，通过在不同点检查程序的状态，确定实际的状态是否与预期的状态一致。

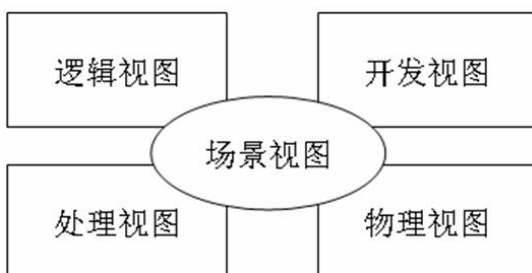
对于白盒子和黑盒子测试，他们两者是不能互相替代的，白盒子测试主要是早阶段进行的，黑盒子测试时后阶段进行的。

70.什么是 RUP (Rational Unified Process) 的 4+1 视图？

RUP (Rational Unified Process, 统一软件开发过程，统一软件过程是一个面向对象且基于网络的程序开发方法论。

要开发出用户满意的软件并不是件容易的事，软件架构师必须全面把握各种各样的需求、权衡需求之间有可能的矛盾之处，从不同视图进行架构设计，来分门别类地将不同需求一一满足

Philippe Kruchten 提出的 4+1 视图方法



该方法的不同架构视图承载不同的架构设计决策，支持不同的目标和用途：

逻辑视图：当采用面向对象的设计方法时，逻辑视图即对象模型。对象模型表示了静态的、结构化的系统数据性质，描述了系统的静态结构，它是从客观世界实体的对象关系角度来描述，表现了对对象的相互关系。

开发视图：描述软件在开发环境下的静态组织。

处理视图：描述系统的并发和同步方面的设计。

物理视图：描述软件如何映射到硬件，反映系统在分布方面的设计。

71.简述软件分析员在系统分析中的任务和作用。

系统分析员 (system analyst) 是对大型、复杂的信息系统建设任务中，承担分析、设计和领导实施的领军人物。要做好与客户之间的关系，同时对客户的需求要正确的理解，要选择合适的开发技术，同时做好与客户间沟通交流，学会说服对方。

工作内容：

- 1、参与用户需求调研、负责系统体系结构、功能、性能的分析和总体设计工作；
- 2、负责项目的开发流程管理，进行项目的计划、管理、跟进工作；
- 3、参与并指导开发工程师完成系统详细设计和开发工作，解决相应业务、技术难题；
- 4、参与编制用户手册、协助客户的系统软件、硬件平台的安装实施工作；
- 5、制定项目文档格式，编写项目管理规范要求的相关文档

作用：

系统分析工作是解决一个问题的的工作，目标是将一个对计算机应用系统的需求转化成实际的物理实现，

72.软件的可维护性与哪些因素有关，如何提高软件的可维护性？

主要有 5 个：1 可理解性 2 可测试性 3 可修改性 4 可移植性 5 可重用性

方法：

1. 建立明确的软件质量目标。
2. 采用现代化的开发方法。
3. 建立明确的质量保证审查。
4. 选择可维护的语言。
5. 改进程序员的文档。

73.事务型软件结构图有什么特点，原因是什么？

事务型软件结构图是一种软件结构图，它描述了软件系统中的事务处理过程，即软件系统如何响应用户或外部事件的输入，执行相应的业务逻辑，产生相应的输出或反馈。

事务型软件结构图有以下几个特点：

- 事务型软件结构图是一种自顶向下的结构图，它从整体到局部，逐层展示了软件系统的功能结构和数据流。
- 事务型软件结构图是一种动态的结构图，它强调了软件系统的行为和状态变化，而不是静态的组成和关系。
- 事务型软件结构图是一种抽象的结构图，它忽略了软件系统的具体实现细节，只关注软件系统的功能和数据。

事务型软件结构图之所以具有这些特点，主要是因为：

事务型软件系统通常是面向用户或外部事件的软件系统，它需要提供清晰和完整的功能视图和数据视图，以便于用户或外部事件的理解和使用。

事务型软件系统通常是动态和复杂的软件系统，它需要表达软件系统的运行逻辑和状态转换，以便于软件系统的分析和设计。

事务型软件系统通常是高层次和跨平台的软件系统，它需要屏蔽软件系统的实现细节和平台差异，以便于软件系统的移植和维护。

74.单元测试、组装测试和确认测试各自主要的目标是什么，相互有什么关系？

单元测试 (unit testing)，是指对软件中的最小可测试单元进行检查和验证。

组装测试 (也叫集成测试，联合测试) 是单元测试的逻辑扩展。在单元测试的基础上，将所有模块按照设计要求 (如根据结构图) 组装成为子系统或系统，进行集成测试。

确认测试的目的是向未来的用户表明系统能够像预定要求那样工作。经集成测试后，已经按照设计把所有的模块组装成一个完整的软件系统，接口错误也已经基本排除了，接着就应该进一步验证软件的有效性，这就是确认测试的任务，即软件的功能和性能如同用户所合理期待的那样。

单元测试是在开发时对小的模块的功能进行测试，集成测试是在几个模块集成的时候对模块之间的接口、集成之后的模块的功能进行测试。这两个测试都需要对软件内部结构比较熟悉，所以通常使用白盒测试的方法。

而确认测试的目的在于测试软件产品是否符合最初定义的用户需求，它不关心内部结果，只关心能否实现功能，也就是相当于把软件当做一个黑盒子，看输入数据能否有正确的输出。

75.Coad/Yourdon 的 OOA 模型包括那几个方面?

Coad 和 Yourdon 方法可分为面向对象分析 (OOA) 和面向对象设计 (OOD) 两部分。在 OOA 中, **建立了概念模型, 由类与对象、属性、服务、结构和主题等 5 个分析层次组成。**

- (1) 类与对象。从问题域、文字出发, 寻找并且标识类与对象。
- (2) 属性。确定对象信息及其之间的关系。可分为原子概念层的单个数据, 和类结构中的公有属性与特定属性。
- (3) 服务。标识消息连接和所有服务说明。
- (4) 结构。标识类层次结构, 确定类之间的整体部分结构与通用特定结构。
- (5) 主题。主题是比结构更高层次的模块, 它与相关类在一起, 控制着系统的复杂度。

在 Coad 和 Yourdon 方法的系统设计阶段也采用了 OOA 阶段的 5 个层次来描述以下设计模型的 4 个部分。

- (1) 问题域。
- (2) 用户界面。包括问题域与用户、系统外部和专用设备、磁盘文件和数据管理界面等。
- (3) 任务管理。包括任务的运行和任务的交互管理等。
- (4) 数据库管理。包括文件管理和数据库管理等。

76.面向对象生存期模型与传统的生存期模型有什么区别?

存期模型是一种描述软件开发过程的方法, 它将软件开发过程划分为若干个阶段, 每个阶段有明确的目标、输入、输出、活动和标准。

面向对象生存期模型是一种基于面向对象思想的生存期模型, 它强调软件系统的对象、类、继承、多态等概念, 以及软件系统的可重用性和可扩展性。

传统生存期模型是一种基于结构化思想的生存期模型, 它强调软件系统的功能、数据、流程等概念, 以及软件系统的可靠性和可维护性。

面向对象生存期模型与传统生存期模型有以下几个区别:

- 面向对象生存期模型更加灵活和迭代, 它允许在任何阶段进行修改和反馈, 而传统生存期模型更加严格和线性, 它要求按照固定的顺序进行开发。
- 面向对象生存期模型更加注重软件系统的设计和实现, 它使用类图、用例图、状态图等图形化的工具进行建模和编码, 而传统生存期模型更加注重软件系统的分析和测试, 它使用数据流图、判定表、判定树等符号化的工具进行规约和验证。
- 面向对象生存期模型更加适合开发复杂和动态的软件系统, 它可以利用对象、类、继承、多态等机制进行抽象和封装, 提高软件系统的可重用性和可扩展性, 而传统生存期模型更加适合开发简单和稳定的软件系统, 它可以利用功能、数据、流程等机制进行分解和组合, 提高软件系统的可靠性和可维护性。

77.软件工程的文档分哪两大类, 主要作用是什么?

软件工程的文档一般分为两大类:

- 软件开发文档, 是指在软件开发过程中产生的文档, 如需求规约、设计说明、测试计划、用户手册等, 它们主要用于描述软件系统的需求、设计、测试、使用等方面的信息, 以便于软件开发人员、管理人员、用户等进行沟通和协作。
- 软件维护文档, 是指在软件维护过程中产生的文档, 如维护请求、维护方案、维护报告等, 它们主要用于记录软件系统的修改和更新的原因、方法、结果等信息, 以便于软件维护人员、管理人员、用户等进行分析和评价。

78.详细设计的任务是什么?

详细设计是软件工程中软件开发的一个步骤, 就是对概要设计的一个细化, 就是详细设计每个模块实现算法, 所需的局部结构。

详细设计包括业务对象设计、功能逻辑设计、数据库设计和界面设计等工作。详细设计是系统实现的依据, 需要考虑所有的设计细节。

详细设计的主要任务是设计每个模块的实现算法、所需的局部数据结构。详细设计的目标有两个: 实现模块功能的算法要逻辑上正确和算法描述要简明易懂。

79.为什么把软件测试的目标定义为只是发现错误?

软件测试的目标是发现错误,是因为:

- 软件测试是一种验证软件系统是否符合需求和设计的活动,它通过执行软件系统或其部分,观察软件系统的输出或行为,与预期的输出或行为进行比较,从而发现软件系统中存在的错误或缺陷。
- 软件测试不能保证软件系统是完全正确或完全满足用户需求的,它只能证明软件系统存在错误或不满足用户需求,而不能证明软件系统不存在错误或完全满足用户需求。因此,软件测试的目标不是证明软件系统的正确性或质量,而是发现软件系统的错误或问题。
- 软件测试是一种有限的活动,它受到时间、资源、技术等因素的限制,它不能覆盖所有可能的输入、输出、状态、环境等情况,它只能选择一部分有代表性或重要性的测试用例进行测试。因此,软件测试的目标不是检查软件系统的所有方面,而是尽可能多地发现软件系统的重要或严重的错误或缺陷。

软件测试是对软件计划、软件设计和软件编码进行纠错的活动,测试的目的是找出整个软件开发周期中各个阶段的错误,分析错误的性质和位置并加以纠正。纠正的过程包括对文档和代码的修改,找错的活动被称为测试,而纠错的过程被称为调试。

- (1)测试是为了发现程序中的错误而执行程序的过程。
- (2)好的测试方案是极有可能发现迄今尚未发现的尽可能多的错误的测试。
- (3)成功的测试是发现了迄今尚未发现的错误的测试。

80.什么是对象关系模型和对象行为模型,有什么不同?

对象关系模型和对象行为模型是两种面向对象的建模方法,它们分别用于描述软件系统中的对象之间的静态结构和动态行为。

对象关系模型是一种用于描述软件系统中的对象之间的静态结构的模型,它主要包括以下几个方面:

- 对象,是指软件系统中具有一定属性和行为的实体,如人、车、订单等。
- 类,是指具有相同属性和行为的一组对象的抽象,如学生类、汽车类、订单类等。
- 属性,是指对象或类所具有的特征或状态,如姓名、颜色、数量等。
- 方法,是指对象或类所具有的行为或功能,如学习、驾驶、下单等。
- 关系,是指对象或类之间的联系或依赖,如继承、关联、聚合等。

对象关系模型通常使用类图来表示,类图是一种图形化的工具,它用矩形框表示类,用连线表示关系,用文字表示属性和方法。

对象行为模型是一种用于描述软件系统中的对象之间的动态行为的模型,它主要包括以下几个方面:

- 事件,是指软件系统中发生的事情或变化,如按键、点击、输入等。事件是某个特定时刻所发生的事情,它是对引起对象从一种状态转换到另一种状态的现实世界中的事件的抽象。
- 消息,是指软件系统中对象之间传递的信息或请求,如调用方法、返回值等。
- 状态,是指软件系统中对象或类在某一时刻的情况或条件,如开启、关闭、就绪等。
- 活动,是指软件系统中对象或类在某一状态下执行的操作或动作,如初始化、计算、输出等。
- 转移,是指软件系统中对象或类从一个状态转变到另一个状态的过程或条件,如满足条件、触发事件等。

对象行为模型通常使用用例图、状态图、活动图等来表示,它们都是一种图形化的工具,它们用圆形框表示事件或状态,用箭头表示消息或转移,用文字表示活动或条件。

81.软件工程是如何克服软件危机的?

软件工程是一门应用数学、计算机科学、管理学等多学科知识和方法,以系统化、规范化、可度量的方式开发和维护软件的工程学科。

软件危机是指在软件开发和维护过程中出现的一系列问题,如软件需求不明确、软件设计不合理、软件质量不可靠、软件成本超支、软件进度延误等。

软件工程是为了克服软件危机而产生的,它通过以下几个方面来解决软件危机:

- 软件工程引入了生存期模型,将软件开发和维护过程划分为若干个阶段,每个阶段有明确的目标、输入、输出、活动和标准,以便于对软件过程进行管理和控制。

- 软件工程采用了面向对象的思想，将软件系统视为由对象、类、继承、多态等构成的实体，以便于对软件系统进行抽象和封装，提高软件系统的可重用性和可扩展性。
- 软件工程使用了图形化的工具，如类图、用例图、状态图等，以便于对软件系统进行建模和编码，提高软件系统的可理解性和可维护性。
- 软件工程实施了质量保证，通过规范的文档、测试、评审等手段，以便于对软件系统进行验证和评价，提高软件系统的可靠性和质量。

82.软件开发主要由哪些方面风险，如何进行软件风险分析？

软件开发的危险是指在软件开发过程中可能出现的不利的或不确定的情况，如需求变更、技术难题、人员流失、资源不足等，它们可能导致软件系统的功能、质量、成本、进度等方面的损失或偏差。

软件风险分析是指对软件开发过程中可能出现的风险进行识别、评估、控制和监测的活动，它主要包括以下几个步骤：

- 风险识别，是指通过收集信息、分析文档、访谈人员等方式，找出软件开发过程中可能存在的风险，并将其记录在风险清单中。
- 风险评估，是指通过定性或定量的方法，对风险清单中的每个风险进行概率和影响的分析，并将其分类为高、中、低等级别。
- 风险控制，是指针对不同级别的风险，制定相应的应对策略，如规避、转移、减轻、接受等，并将其记录在风险计划中。
- 风险监测，是指通过定期的检查、报告、审查等方式，跟踪风险清单和风险计划中的每个风险的状态和变化，并根据实际情况进行调整和更新。

83.程序的编码风格主要体现在哪几个方面？

程序的编码风格即为程序设计风格或编程风格，其主要作用是使无论是程序本人还是其他人，都能比较容易的阅读、理解及修改源代码；

主要表现在四个方面：源程序文档化、数据说明方法，表达式和语句结构及输入/输出方法；

84.简述软件测试中桩模块和驱动模块的作用。

桩模块（Stub）是指模拟被测试的模块所调用的模块，而不是软件产品的组成的部分。主模块作为驱动模块，与之直接相连的模块用桩模块代替。

桩模块的作用：在集成测试前要为被测模块编制一些模拟其下级模块功能的“替身”模块，以代替被测模块的接口，接受或传递被测模块的数据，这些专供测试用的“假”模块称为被测模块的桩模块。

传统的单元测试术语（unit testing terminology），包括了驱动模块（driver）和桩模块（stub）。驱动模块是用来模拟被测试模块的上一级模块，相当于被测模块的主程序。它接收数据，将相关数据传送给被测模块，启用被测模块，并打印出相应的结果。

作用：驱动模块的目的很单纯，就是为了访问类库的属性和方法，来检测类库的功能是否正确；

85.如何从需求文档中识别类和对象

从需求文档中识别类和对象的方法有以下几种：

- 词法分析法，是指通过分析需求文档中的名词、动词、形容词等词汇，找出可能是类或对象的候选词，如人、车、订单等是名词，可能是类或对象；学习、驾驶、下单等是动词，可能是方法；年龄、颜色、数量等是形容词，可能是属性。
- 语义分析法，是指通过分析需求文档中的语义关系，找出可能是类或对象的候选词，如具有相同属性和行为的一组实体，可能是类；具有独立存在和标识的实体，可能是对象；具有包含或组成的关系的实体，可能是聚合或组合；具有一般和特殊的关系的实体，可能是继承或泛化。
- 模板分析法，是指通过使用一些预定义的问题模板，对需求文档中的每个候选词进行提问和回答，找出可能是类或对象的候选词，如“它是什么？”、“它能做什么？”、“它有什么？”、“它属于什么？”等。

86.软件危机主要有哪些表现?

软件危机的表现有以下几种:

- 对软件开发成本和进度的估计常常很不准确,导致软件项目超支和延期。
- 软件产品的质量往往靠不住,导致软件系统出现故障和漏洞。
- 用户对已完成的软件系统不满意,导致软件系统无法满足用户的需求和期望。
- 软件系统的维护成本高昂,导致软件系统难以适应环境的变化和技术的更新需求增长得不到满足

软件危机是指在软件开发中遇到的问题找不到解决方法,使问题积累起来,形成了尖锐的矛盾。

其表现包括软件开发进度难以控制、软件质量难以保证、软件成本不断提高等。

产生的原因主要是软件规模的扩大和软件复杂性的增加,导致传统的软件开发方法无法满足需求。

87.什么是软件维护的副作用,如何防止软件维护的副作用?

软件维护的副作用主要是在修改原有错误时会引入新的错误。

预防的措施主要是程序模块化,信息隐蔽,高内聚、低耦合等。

面向对象分析设计中使用封装、抽象、继承等办法。具体的包括:

- 1、按模块把修改分组;
- 2、自顶向下地分析所修改模块的顺序;自底向上修改有关模块。
- 3、每次修改一个模块;牵涉到全局性的基础模块改动必须仔细分析。
- 4、对于每个修改了的模块,在安排修改下一个模块之前,要确定这个修改的副作用,主要是关注修改后对调用这个模块的其他模块的功能影响。

88.好的软件体系结构设计应遵循哪些原则?

软件体系结构设计的原则有以下几种:

- 抽象原则,是指将软件系统分解为若干个抽象的组件和连接器,以便于对软件系统进行高层次的描述和理解。
- 模块化原则,是指将软件系统划分为若干个相对独立的模块,以便于对软件系统进行分工和协作。
- 封装原则,是指将软件系统中的每个组件或模块的内部细节隐藏起来,只暴露出必要的接口和协议,以便于对软件系统进行保护和隔离。
- 开放原则,是指将软件系统设计成可以与其他软件系统进行交互和集成,以便于对软件系统进行扩展和复用。
- 低耦合高内聚原则,是指将软件系统中的组件或模块之间的依赖关系尽量减少,而将组件或模块内部的相关性尽量增强,以便于对软件系统进行修改和测试。

89.简述渐增式测试方法与非渐增式测试方法的优劣。

渐增式测试是先从单个模块开始测试,然后每次将测试后的一个模块添加到系统中来进行下一个测试的方法。

非渐增式测试,是指先独立地测试每一模块,然后将所有这些模块连接到一起运行;一下子把所有模块放在一起,并把庞大的程序作为一个整体来测试,测试者面对的情况十分复杂。

非渐增式与渐增式两种测试的区别

非渐增式测试方法需要编写的测试用例较多,工作量较大;渐增式测试方法开销小。

渐增式测试方法发现模块间接口错误早;而非渐增式测试方法晚。

非渐增式测试方法发现错误,较难诊断;而使用渐增式测试方法,如果发生错误则往往和最近加进来的那个模块有关。

渐增式测试方法测试更彻底

渐增式测试方法需要较多的机器时间

使用非渐增式测试方法,可以并行测试。

90.UML 中有哪些动态建模的工具,各表达什么内容?

UML 中有以下几种动态建模的工具,各表达如下内容:

- 用例图,是指描述软件系统与外部参与者之间的交互和功能需求的图。
- 时序图,是指描述软件系统中对象之间按时间顺序进行的消息交换的图。
- 协作图,是指描述软件系统中对象之间的结构和消息交换的图。
- 状态图,是指描述软件系统中对象或类在不同状态下的行为和转换的图。
- 活动图,是指描述软件系统中的业务流程或算法逻辑的图。

91.软件技术审查和管理复审(评审)的作用是什么?

1、软件技术审查由一组评审者按照规范的步骤对软件需求、设计、代码或其他技术文档进行仔细地检查,以找出和消除其中的缺陷。

作用:

- (1) 发现软件在功能、逻辑、实现上的错误;
- (2) 验证软件符合它的需求规格;
- (3) 确认软件符合预先定义的开发规范和标准;
- (4) 保证软件在统一的模式下进行开发;
- (5) 便于项目管理。

2、管理复审是指系统地评价鉴定全部管理工作的一种控制方法,它侧重于管理决策方面的审核,包括对组织中关于计划、工程技术、生产、营销、人事、会计以及财务方面的审核。

管理复审的主要任务是,在软件生命周期的每个阶段结束时,对工程项目的成本、实际花费的经费、投资回收的前景、项目的进度等经济因素从管理角度进行审查。从而找出在成本、进度、质量上存在的问题,为后续改进提供问题基础。

简版回答:

软件技术审查和管理审查是软件质量保证的两种重要活动。软件技术审查是对软件开发过程中的各种文档、设计、代码等进行系统性、结构化的检查,以发现和消除潜在的缺陷,提高软件质量和可靠性。

管理审查是对软件项目的总体状况、成本、进度、风险等进行评估和监控,以便及时调整项目计划,保证项目目标的实现。

92.软件的可维护性与那些软件质量特性有关?

软件可维护性的定义:软件能够被理解、校正、适应及增强功能的容易程度。

软件可维护性可用下面七个质量特性来衡量,即可理解性、可测试性、可修改性、可靠性、可移植性、可使用性和效率。对于不同类型的维护,这七种特性的侧重点也是不相同。

93.什么是模块独立性,如何衡量模块独立性?

模块独立性指每个模块只完成系统要求的独立的子功能,并且与其他模块的联系最少且接口简单。

模块独立性是指模块内部各部分及模块间的关系的一种衡量标准,由内聚和耦合来度量。

- 1.内聚:衡量模块内部各部分间的功能相关性。高内聚表示模块内元素紧密协作完成单一功能。
- 2.耦合:衡量模块间的依赖关系。低耦合表示模块间联系简单,修改不影响其他模块。

两者结合,内聚越高、耦合越低,模块独立性越强,系统更易维护和扩展。

94.软件测试有几种方法,每种方法的特点是什么?

1、按是否查看程序内部结构分为:

- (1)、黑盒测试 (black-box testing): 只关心输入和输出的结果
- (2)、白盒测试 (white-box testing): 去研究里面的源代码和程序结构

2、按是否运行程序分为:

(1)、静态测试 (static testing): 是指不实际运行被测软件,而只是静态地检查程序代码、界面或文档可能存在的错误的过程。

(2)、动态测试 (dynamic testing),是指实际运行被测程序,输入相应的测试数据,检查输出结果和预期结果是否相符的过程

3、按阶段划分：

(1) 单元测试 (unit testing)，是指对软件中的最小可测试单元进行检查和验证。

(2) 集成测试 (integration testing)，是单元测试的下一阶段，是指将通过测试的单元模块组装成系统或子系统，再进行测试，重点测试不同模块的接口部门。集成测试就是用来检查各个单元模块结合到一起能否协同配合，正常运行。

(3) 系统测试 (system testing)，指的是将整个软件系统看做一个整体进行测试，包括对功能、性能，以及软件所运行的软硬件环境进行测试。系统测试的主要依据是《系统需求规格说明书》文档。

(4) 验收测试 (acceptance testing)，指的是在系统测试的后期，以用户测试为主，或有测试人员等质量保障人员共同参与的测试，它也是软件正式交给用户使用的最后一道工序。

其他还有回归测试、冒烟测试、随机测试

其中黑盒测试包括功能测试和性能测试；

功能测试有：逻辑功能测试、界面测试、易用性测试、安装测试、兼容测试；

性能测试有：一般性能测试、稳定性测试、压力测试、负载测试

95.UML 中的接口类有什么特点，举例说明。

接口描述了一组实现这一接口的分类器所必须共同遵守的合同，这种合同包括结构上的也包括行为上的，此外，接口是不能实例化的。接口实现则是一种依赖关系，其描述了接口与分类器之间的一种特殊的实现关系。

图 1 示例了接口的棒棒糖表示方法，其中“棒棒糖”表示与之相连的类提供相应的接口，也就是说相关联的类实现了这一接口。

从图中我们可以看出，ProximitySensor 类实现了 ISensor 接口，而 TheftAlarm 类则需要一个 ISensor 接口。由于两个类一个提供，而另一个需要，因此我们可以将其连接在一起，如图 2 所示。



图 1 接口的棒棒糖表示法

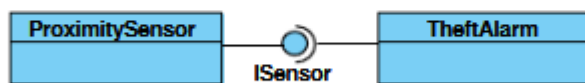


图 2 接供和需要接口的图形可以连接在一起

96.常见的软件开发模型有那些，各有什么特点？（简述螺旋模型、瀑布模型、演化模型、增量模型各自的特点及它们之间的主要区别）

常见软件开发模型有：瀑布模型、演化模型、增量模型、螺旋模型、快速原型、V 模型、喷泉模型等。

瀑布模型：瀑布模型是一种经典的软件开发方法论，它将软件开发生命周期划分为一系列顺序的阶段，就像瀑布一样，水流从一个阶段依次流向下一个阶段，每个阶段都有明确的输入和输出。

瀑布模型的特点是：

- (1) 规定了阶段之间自上而下、相互衔接的固定次序，上一阶段的成果作为下一阶段的输入
- (2) 每一个阶段完成之后，必须对阶段性成果进行评审，确认之后才可进入下一阶段。

演化模型：演化模型主要针对事先不能完整定义需求的软件项目，此模型可以减少风险。演化模型全部系统是增量开发,增量提交。

增量模型：增量模型在软件开发过程中把软件产品划分为一系列的增量构件,分别进行设计、编程、集成和测试。每个构件由多个相互作用的模块构成,并且能够完成特定的功能。使用增量模型的软件体系结构必须是开放的,加入新构件的过程必须简单方便,新的增量构件不得破坏已经开发出来的产品。因此,增量模型是一种非整体开发模型,灵活性好。

快速原型模型：快速原型模型的第一步是建造一个快速原型，实现客户或未来的用户与系统的交互，用户或客户对原型进行评价，进一步细化待开发软件的需求。通过逐步调整原型使其满足客户的要求，开发人员可以确定客户的真正需求是什么；第二步则在第一步的基础上开发客户满意的软件产品。

螺旋模型：螺旋模型，它将瀑布模型和演化模型结合起来，强调了其他模型所忽视的风险分析，特别适合于大型复杂的系统。螺旋模型由风险驱动，强调可选方案和约束条件从而支持软件的重用，有助于将软件质量作为特殊目标融

入产品开发之中。

V 模型：瀑布模型的变体，编码（coding）构成了 V 点，左边是分析和设计，右边是测试和维护，它表达了测试与分析和设计的对应相关，该模型更明确地说明了隐藏在瀑布描绘中的一些迭代和返工。V 模型的特点：

1. **结构清晰：**V 模型将软件测试的各个阶段和开发阶段进行对应，使整个测试过程的结构清晰可见。
2. **并行开展：**V 模型强调测试活动与开发活动的并行进行，以尽早发现和修复软件缺陷，提高软件质量。
3. **强调验证与验证：**V 模型注重对需求的验证和设计的验证，确保软件满足用户需求和设计要求。
4. **易于管理和控制：**V 模型的测试活动与开发活动一一对应，使项目管理和控制更加方便。

喷泉模型：喷泉模型是典型的面向对象生命周期模型，是一种以用户需求为动力，以对象作为驱动力的模型。该模型克服了瀑布模型不支持软件重用和多项开发活动集成的局限性。“喷泉”一词本身体现了迭代和无间隙特性。迭代意味着模型中的开发活动常常需要重复多次，在迭代过程中不断地完善软件系统；无间隙是指在开发活动之间不存在明显的边界。特点：适应变化的迭代开发、增量交付、适合需求不明确与频率变化、适合技术探索型和以客户为中心项目等。

97.CMMI 与 ISO9000 有什么异同

1、CMMI 是指“能力成熟度模型集成”，其英文全称为 Capability Maturity Model Intergration，简称 CMMI。它是对于软件组织在定义、实施、度量、控制和改善其软件过程的实践中各个发展阶段的描述。CMMI 的核心是把软件开发视为一个过程，并根据这一原则对软件开发和维护进行过程监控和研究，以使其更加科学化、标准化、使企业能够更好地实现商业目标。

2、ISO9000 是指质量管理体系标准，它不是指一个标准，而是一种标准的统称。

ISO 与 CMM 的相同：

(1)ISO9000 质量体系与 CMMI，都共同着眼于质量和过程管理，两者都为了解决同样的问题。从一方面说他们是相互联系、相互补充的。

(2)两者都吸收了现代质量管理理论，都以“过程思维”为指导。

(3)ISO9001 中的质量要素都可以对应到 CMMI 中关键过程区域特征上，而 CMMI 在生产过程中的管理重点，又弥补了 ISO9001 在微观管理上的不足。

ISO 与 CMM 的不同：

(1) ISO9001 确定一个质量体系的最少需求，而 CMMI 模型更在注重持续过程改进。

(2) 而且，ISO9001 只建立了一个可接受水平，而 CMMI 是一个具有五个水平的评估工具。

(3) ISO9001 标准面向合同环境，站在用户立场，要求对质量要素进行控制，规定了质量管理体系的最低标准。而 CMMI 标准则强调软件开发过程的成熟度，即内部过程的不断改进和提高。

(4) 在形式上，ISO9001 审核只有“通过”和“不通过”两个结论，而 CMMI 评定则是一个动态过程，软件组织在通过低级别评估后，可根据高级别的要求确定改进方略。

98.什么是耦合，如何衡量模块的耦合度？

耦合是程序结构中各个模块之间相互关联的度量。

它取决于各个模块之间的接口的复杂程度、调用模块的方式以及哪些信息通过接口

99.软件测试分几种类型，主要解决什么问题？

参照 94 题

100.简要说明 RUP 中阶段、核心 workflow 和迭代的关系。

RUP (Rational Unified Process, 统一软件开发过程, 统一软件过程)是一个面向对象且基于网络的程序开发方法论。

RUP 中的软件生命周期在时间上被分解为四个顺序的阶段，分别是：初始阶段 (Inception)、细化阶段 (Elaboration)、构造阶段 (Construction) 和交付阶段 (Transition)。每个阶段结束于一个主要的里程碑 (Major Milestones)；每个阶段本质上是两个里程碑之间的时间跨度。在每个阶段的结尾执行一次评估以确定这个阶段的目标是否已经满足。如果评

估结果令人满意的话，可以允许项目进入下一个阶段。

RUP 中有 9 个核心工作流，分为 6 个核心过程工作流（Core Process Workflows）和 3 个核心支持工作流（Core Supporting Workflows）。尽管 6 个核心过程工作流可能使人想起传统瀑布模型中的几个阶段，但应注意迭代过程中的阶段是完全不同的，这些工作流在整个生命周期中一次又一次被访问。9 个核心工作流在项目中轮流被使用，在每一次迭代中以不同的重点和强度重复。

101. 简述一下里程碑的作用？（在项目中设置里程碑有什么好处?）

里程碑一般是项目中完成阶段性工作的标志，标志着上一阶段的结束、下一阶段的开始。里程碑定义了当前阶段完成的标注和下一新阶段启动的条件和前提。

里程碑的作用：

- (1) **明确目标与方向**：里程碑帮助团队明确项目的关键目标和方向，确保所有成员对项目的重要阶段有共同的理解和期待。
- (2) **监控进度**：通过定期检查里程碑的达成情况，项目经理可以有效监控项目的整体进度，及时发现并解决潜在问题。
- (3) **风险控制**：里程碑作为项目的关键节点，有助于识别和评估项目风险，提前采取措施减少不确定性和负面影响。
- (4) **激励团队**：达成里程碑是团队努力的结果，可以增强团队士气，提高成员的工作动力和满意度。
- (5) **沟通工具**：里程碑是向项目相关方报告项目状态的有力工具，有助于保持透明度，建立信任。

102. 测试结束的标准是什么？

软件测试的结束标准通常是根据项目的需求和目标来确定的。以下是一些通用的测试结束标准：

1. 所有测试用例都已执行：确保所有计划的测试用例都被执行，包括功能测试、性能测试、安全测试等各种类型的测试。
2. 所有缺陷已修复：所有已识别的缺陷都被记录、分类和追踪，并且经过验证的已经被修复。
3. 测试覆盖率满足要求：根据项目的要求，确保测试覆盖了系统的各个方面，包括功能、性能、安全等。
4. 性能满足要求：如果项目有性能要求，确保系统在负载和性能方面达到了这些要求。
5. 安全测试通过：如果项目涉及安全性要求，确保系统在安全性方面通过了测试，并没有明显的漏洞或弱点。
6. 用户验收测试通过：如果有用户验收测试，确保系统满足用户的期望和需求。
7. 测试报告和文档已准备完毕：确保所有必要的测试文档，包括测试计划、测试用例、测试报告等已经准备完毕。
8. 测试团队和利益相关者已确认：确保测试团队和利益相关者都已经确认测试的结束，并且对测试结果满意。
9. 上线准备就绪：确保系统已准备好上线，包括备份和恢复计划、数据迁移、培训和支持等。
10. 风险评估已完成：对项目的风险进行评估，确保已经采取措施来降低或管理这些风险。

简版：

- (1) 测试目标达成
- (2) 测试计划的实施
- (3) 质量标准的达成
- (4) 缺陷的修复
- (5) 风险评估
- (6) 客户满意度
- (7) 法规和合规性
- (8) 性能要求满足
- (9) 文档和报告
- (10) 决策和批准

总之，测试结束标准是一个综合考虑项目目标、质量、风险和客户需求的过程。

103. 比较原型模型和螺旋模型的优缺点。

原型模型是一个用于验证设计方案或解决方案的工具，可以帮助设计师、开发人员和客户更好地理解 and 沟通需求和解决方案。

原型模型的优点是快速创建原型，与客户进行交互和反馈，以便更好地理解 and 确认需求。它容易适应需求的变化，有利用开发与培训的同步，费用低、开发周期短且对用户更友好。

原型模型也有一些缺点，客户与开发者对原型理解不同，准确的原型设计比较困难，不利于开发人员的创新。

螺旋模型则是一种迭代和风险驱动的软件开发模型，它将软件开发分为四个阶段：计划、风险分析、工程实现和评审。在每个迭代中，开发人员需要根据风险管理计划来确定下一步的工作。

螺旋模型的优点是灵活性高，可以在项目的各个阶段进行变更，客户始终参与每个阶段的开发，保证了项目不偏离正确方法以及项目的可按性。

螺旋模型的缺点：它需要具有相当丰富的风险评估经验和专门知识，在风险较大的项目开发中，如果未能及时标识风险，势必造成重大损失；会增加开发成本，延迟提交时间。

104、软件开发过程与软件过程的核心区别？

软件开发过程与软件过程的核心区别在于范围与侧重点：

(1)范围不同

软件过程：覆盖软件全生命周期（需求、开发、测试、部署、维护、退役），包含技术活动（如编码）和非技术活动（如项目管理、质量保障）。

软件开发过程：仅聚焦开发阶段（需求到交付），如需求分析、设计、编码、测试。

(2)侧重点不同

软件过程：强调整体流程管理（如 CMMI、ISO 标准），包含组织级规范、维护和支持。

软件开发过程：专注技术实现方法论（如瀑布模型、敏捷开发）。

总结：

软件过程是宏观框架，包含开发、维护等所有环节；软件开发过程是其子集，仅解决“如何构建软件”。

（简明版：软件过程 = 全生命周期管理；软件开发过程 = 构建阶段的方法。）

105、为什么没有一个单一的、全包容的程序复杂度度量方法？

程序复杂度难以通过单一度量方法全面评估，主要原因如下：

(1) 多维复杂性

程序复杂度涉及多个维度，如结构复杂度（控制流、数据流）、认知复杂度（代码可读性）、维护复杂度（扩展性）等。不同维度需不同指标衡量，例如：

圈复杂度（Cyclomatic Complexity）侧重控制流分支；

Halstead 复杂度关注操作符与操作数数量；**代码行数（LOC）**仅反映规模而非逻辑难度。

(2)应用场景的多样性

不同领域对复杂度的定义不同：

嵌入式系统关注实时性和资源占用；

分布式系统聚焦通信与同步逻辑；

业务系统重视可维护性与需求变更适应性。

单一指标无法满足所有场景的核心需求。

(3)语言与范式的差异

编程语言特性（如函数式、面向对象）和架构风格（微服务、单体）直接影响复杂度表现。例如：

函数式编程的递归和高阶函数复杂度难以用传统面向对象指标衡量；

微服务架构的分布式事务复杂度远超单体应用。

(4)主观性与上下文依赖

复杂度感知受开发者经验、团队规范及业务目标影响：

性能优化代码可能牺牲可读性，但对特定场景至关重要；

设计模式可能增加结构复杂度，却提升扩展性。

统一标准无法平衡此类权衡。

(5)度量工具的局限性

现有工具（如静态分析器）通常聚焦技术指标，但忽略：

团队协作成本（如代码评审难度）；

业务逻辑的抽象合理性（如领域模型清晰度）。

总结

程序复杂度本质上是多维度、多视角的综合问题，需结合具体场景、语言特性和团队目标，通过多种指标互补（如圈复杂度+认知复杂度+架构耦合度）进行合理评估，而非依赖单一“万能”方法。

软件复杂性度量是一个复杂的问题，因为软件复杂性是一个多维度的概念，包括代码行数、代码结构、代码质量、代码可读性、代码可维护性等多个方面。不同的软件项目可能会有不同的复杂性特征，因此很难单一对软件复杂性进行度量。

106、软件开发成本由哪些组成成分？

- (1)人员成本: 包括开发人员、测试人员、项目经理等人员的工资、福利、培训等费用。
- (2)硬件成本: 包括服务器、存储设备、网络设备等硬件设备的购置、维护、更新等费用。
- (3)软件成本: 包括开发工具、测试工具、数据库软件等软件的购置、维护、更新等费用。
- (4)外包成本: 如果公司选择外包开发, 那么外包公司的服务费用也是软件开发成本的一部分。
- (5)其他成本: 包括办公场地租金、水电费、差旅费等其他费用。

107、软件测试的策略有哪些？

(1) 单元测试

单元测试 (unit testig), 是指对软件中的最小可测试单元进行检查和验证。

(2) 集成测试

集成测试 (integratio testig), 也叫组装测试或联合测试。在单元测试的基础上, 将所有模块按照设计要求 (如根据结构图) 组装成为子系统或系统, 进行集成测试。

(3) 系统测试

系统测试 (system testig), 是对整个系统的测试, 将硬件、软件、操作人员看作一个整体, 检验它是否有不符合系统说明书的地方。

(4) 确认测试

确认测试 (cofirmatio testig), 又称有效性测试, 是对通过集成测试的软件在模拟的环境下进行的有效性测试, 目的是要表明软件是可以工作的, 并且符合“软件需求说明书”中规定的全部功能和性能要求。

108、结构化分析和结构化设计的鸿沟。

结构化分析和结构化设计是一种面向数据流的传统软件开发方法。分析阶段的主要模型是数据流程图, 设计阶段的主要模型是软件模块结构图, 数据流程图和软件模块结构图之间需要进行转换。不同的人转换出的模块图可能不同, 有很大的随意性。在结构化分析中, 模块之间的关系被局限于信息流, 这限制了对模块之间众多关系的表达, 也无法体现模块和模块之间其他的众多关系, 包含各种各样的结构、行为、依赖、包含、继承、关联关系等等。而面向对象编程则能够有效解决这些问题。

109、面向对象的测试和传统测试的区别？

面向对象软件抛弃了传统的开发模式, 对每个开发阶段都有不同于传统软件测试的要求和结果, 已经不可能用功能细化的观点来检测面向对象分析和设计的结果。

因为:

- a) 测试的对象不同: 传统软件测试的对象是面向过程的软件, 一般用结构化方法构建; 面向对象测试的对象是面向对象软件, 采用面向对象的概念和原则, 用面向对象的方法构建。
- b) 测试的基本单位不同: 前者是模块; 面向对象测试的基本单元是类和对象。
- c) 测试的方法和策略不同: 传统软件测试采用白盒测试, 黑盒测试, 路径覆盖等方法; 面向对象测试不仅吸纳了传统测试方法, 也采用各种类测试等方法, 而且集成测试和系统测试的方法和策略也很不相同。

110、变换型数据流图和事务型数据流图的区别

变换型数据流图和事务型数据流图是两种不同的数据流图类型。

变换型数据流图是一种线性结构，由输入、变换和输出三部分组成，事务型数据流图则呈束状，即一束数据流平行流入或流出，可能同时有几个事务要求处理。

事务型数据流图的特点是将它的输入流分离成许多发散的数据流，形成许多加工路径，并根据输入的值选择其中一个路径来执行。

这两种典型的结构分别可以通过“变换分析”和“事务分析”技术，导出相应的初始模块结构图。

变换分析和事务分析技术的思想是首先设计出顶层模块，然后自顶向下、逐步求精，最后得到一个满足数据流图所表示的系统模块初始结构图，即系统的物理模型。

111、高质量软件需求应遵循哪些原则？

清晰性原则：需求应该清晰、明确、易于理解和解释。

完整性原则：需求应该涵盖所有的功能和性能要求。

一致性原则：需求应该与其他文档和规范保持一致。

可验证性原则：需求应该是可验证的，以便能够检查它们是否得到满足。

可追踪性原则：需求应该是可追踪的，以便能够跟踪它们的来源和演变历史。

112、变换型和事务型数据流图各有什么特点？

变换型数据流图和事务型数据流图是两种不同的数据流图类型。

变换型数据流图是一种线性结构，由输入、变换和输出三部分组成，事务型数据流图则呈束状，即一束数据流平行流入或流出，可能同时有几个事务要求处理。

事务型数据流图的特点是将它的输入流分离成许多发散的数据流，形成许多加工路径，并根据输入的值选择其中一个路径来执行。

113、软件生命周期作用

软件生存周期是指从软件需求提出到报废的整个过程，包括问题定义、可行性分析、总体描述、系统设计、编码、调试和测试、验收与运行、维护升级到废弃等阶段。

软件生存周期的作用：

- (1)确立了软件开发和演绎中各阶段的次序限制以及各阶段或机动的准则。
- (2)确立开发过程所遵守的规定和限制。
- (3)便于各种活动的协调、便于各种人员的有效通信、有利于活动重用、有利于活动管理。
- (4)规范软件开发过程，提高软件开发效率和质量，降低开发成本。

114、软件生命周期模型与软件开发的关系

软件生命周期是指软件从需求提出到软件废弃的整个过程，包括需求分析、设计、编码、测试和维护等阶段。

常见的生命周期模型有瀑布模型、快速原型模型、增量模型、螺旋模型等

软件开发则是提软件生命周期中的编码阶段。

软件开发是软件生命周期的模型的一部分，它是整个生命周期中的一个重要环节。在软件开发过程中，开发人员需要遵循软件生命周期模型的各个阶段，以确保软件质量和项目进度。

115、软件质量控制与软件质量保证之间有何区别？

①软件质量保证：软件质量保证是一种预防性的活动，旨在确保软件开发过程中遵循适当的流程、标准和规范，以提高最终软件产品的质量。

质量保证关注的是整个软件开发过程中的质量管理，包括规划、组织、实施和监控各项活动，以确保软件开发团队按照正确的方式进行工作，从而最大程度地降低缺陷和问题的出现。

②软件质量控制：软件质量控制是一种检测性的活动，旨在检查和验证软件产品是否符合预定的质量标准和要求。质量控制主要关注的是已经开发完成的软件产品，通过执行测试、审查和检查等活动，发现和修复软件中存在的缺陷和问题，以确保软件产品的质量符合预期。

质量保证与质量控制的区别

软件质量保证与质量控制的区别如下表所示：

项目	质量保证 (QA)	质量控制(QC)
定义	质量保证是一组活动，可确保始终保持软件开发过程中使用的过程质量。	QC是一组用于检测已开发软件中的缺陷的活动。
关注重点	QA的重点是通过关注流程来防止开发软件中的缺陷。	QC的重点是通过关注测试过程来识别开发软件中的缺陷。
如何做	建立高质量的管理系统，并定期审核开发软件的操作是否符合要求。	通过使用开发软件中的测试技术和工具来检测和消除质量问题元素。
为什么做	质量保证通过使用包括文档在内的系统活动来确保质量问题的预防。	QC通过使用流程和技术来实现和维护高质量的软件，从而确保识别和消除缺陷。
面向	质量保证是面向流程的。	QC是面向产品的。
过程类型	质量保证是一个积极主动的过程。它关注改善开发；在测试期间不会出现缺陷。	QC是一种反应过程，因为它涉及在产品开发之后和产品发布之前识别缺陷。
责任	开发团队的每个成员都负责QA	只有特定的测试团队负责QC
示例	验证软件流程。	检验软件功能和流程。

116、软件开发中如何进行风险分析？

软件工程项目的开发也存在各种各样的风险，有些风险甚至是灾难性的。

软件工程的风险分析主要包括：风险识别、风险评估和风险管理。

1、风险标识

风险可以分为项目风险、技术风险和商业风险三类。

由于项目在预算、进度、人力、资源、顾客和需求等方面的原因对软件项目产生的不良影响称为项目风险。

软件在设计、实现、接口、验证和维护过程中可能发生的潜在问题，如规格说明的二义性、采用陈旧或尚不成熟的技术等等，对软件项目带来的危害称技术风险。

开发了一个没人需要的优质软件，或推销部门不知如何销售这一软件产品，或开发的产品不符合公司的产品销售战略，等等，称为商业风险。

风险识别方法：

(1) **头脑风暴法**：头脑风暴法是一种广泛使用的创新技术，可以用于识别潜在的软件项目风险。该方法通过集体讨论和思考来产生大量想法，并对这些想法进行分类和筛选。

(2) **专家访谈法**：专家访谈法是一种通过与相关领域专家交流来识别潜在风险的方法。通过与专家交流，可以了解到他们对潜在问题的看法和经验，并提供有关如何应对这些问题的建议。

(3) **检查清单法**：检查清单法是一种基于已知经验或历史数据来识别潜在风险的方法。该方法通过检查清单来确定可能出现的风险，并对这些风险进行评估和分类。

(4) **场景分析法**：场景分析法是一种通过模拟可能发生的事件来识别潜在风险的方法。该方法通过制定不同的场景，模拟不同的情况，以便确定可能出现的问题和障碍。

(5) **故事板法**：故事板法是一种通过绘制故事板来识别潜在风险的方法。该方法将项目中可能出现的情况绘制成图像，以便更好地理解 and 评估潜在风险。

2、风险评估

在识别潜在风险后，需要对这些风险进行评估。评估包括确定每个潜在风险发生的概率、影响程度和优先级。

(1) **概率评估**：概率评估是指确定每个潜在风险发生的可能性。可以使用历史数据、专家意见或其他可用信息来确定每个潜在风险发生的概率。

(2) **影响评估**：影响评估是指确定每个潜在风险发生时对项目目标实现造成的影响程度。可以使用定量或定性方法来评估影响程度。

(3) **优先级评估**：优先级评估是指确定每个潜在风险的优先级。可以使用概率和影响程度来确定每个潜在风险的优先级。

3、风险管理

在识别和评估风险后，需要采取措施来处理这些风险。可以采用以下几种方法来管理软件项目中的风险。

(1) **避免**：避免是指通过采取措施来消除潜在风险或减少其发生的可能性。例如，可以选择使用已经得到验证的技术或工具，以减少出现问题的可能性。

(2) **转移**：转移是指将潜在风险转移到其他人或组织。例如，可以购买保险来减轻一些财务方面的风险。

(3) **缓解**：缓解是指通过采取措施来减轻潜在风险造成的影响。例如，可以制定备份计划，在出现故障时能够快速恢复数据。

(4) **接受**：接受是指认识到某些潜在风险无法避免或缓解，并决定承担其造成的影响。例如，可能无法避免某些自然灾害造成的影响。

总结：软件项目中的风险是不可避免的，但可以通过识别、评估和管理来减少其对项目目标实现造成的影响。通过使用多种方法来识别和评估风险，并采取适当的措施来处理这些风险，可以帮助软件项目成功完成。

117、对于软件企业，CMMI 模型有什么作用？

CMMI 全称是 Capability Maturity Model Integration，即能力成熟度模型集成（也有称为：软件能力成熟度集成模型）CMMI 资质主要是帮助软件企业对软件工程过程进行管理和改进，增强开发与改进能力，从而能按时地、不超预算地开发出高质量的软件。

第一：**能保证软件开发的质量与进度**，能对“杂乱无章、无序管理”的项目开发过程进行规范。

第二：**有利于成本控制**。因为质量有所保证，浪费在修改、解决客户的抱怨方面的成本会降低很多。现在绝大多数情况是缺少规范制度，只是求快。项目完成后，要花很多时间修修补补，费用很容易失控。

第三：**有助于提高软件开发者的职业素养**。每一个具体参与其中的员工，无论是项目经理，还是工程师，甚至一些高层管理人的做事方法逐渐变得标准化、规范化。

第四：**能够解决人员流动所带来的问题**。公司通过过程改进，建立了财富库以共享经验，而不是单纯依靠某些人员。

第五：**有利于提升公司和员工绩效管理水平，以持续改进效益**。通过度量和分析开发过程和产品，建立公司的效率指标。

第六：**政策扶持**，根据各省市每年政策，对实施 CMMI 企业会有一定金额的政策补贴，部分地区最高可拿 50 万。

118、简述原型构造在软件开发中的作用。

软件原型是一个软件产品的工作模型，它在软件开发过程中用于测试和演示软件的功能和界面，帮助开发者和用户更好地理解软件并进行决策。

软件原型在软件开发中的作用

(1) **需求验证**：通过构建原型，开发团队和客户可以更直观地看到软件将如何工作，从而更容易发现和解决需求中的模糊性、不一致性或遗漏。

(2) **促进沟通**：原型作为一种可视化工具，有助于减少技术术语和误解，促进开发团队、客户和其他利益相关者之间的有效沟通。

(3) **风险降低**：早期发现潜在的问题和障碍，通过原型进行验证和测试，可以降低项目后期的风险和成本。

提升用户体验：用户界面原型可以让用户参与到软件的设计过程中，提供宝贵的反馈，从而帮助开发团队创建更符合用户需求和期望的产品。

(4) **促进团队协作**：原型可以作为团队成员之间的共同参考点，帮助大家达成共识，明确目标和方向。

支持迭代开发：在敏捷开发方法中，原型常常作为迭代周期的一部分，用于快速展示和测试新的功能或设计思路。

综上所述，软件原型在软件开发中扮演着至关重要的角色，它不仅是验证需求和设计的有效工具，也是促进沟通、降低风险、提升用户体验和推动团队协作的重要手段。

119、软件的概要设计与软件的详细设计之间有什么联系和区别？

概要设计和详细设计是软件开发中两个非常重要的步骤。

概要设计是指在软件开发过程中，对软件的整体结构、组件、模块和交互进行规划和设计的过程。

详细设计是在概要设计阶段之后的一阶段，是对概要设计中所确定的结构的具体实现。

概要设计和详细设计两者之间的区别在于粒度的不同。

概要设计主要是指对系统整体的规划，主要关注整个系统的结构和模块关系；

详细设计主要是对概要设计中所确定的系统结构进行更为具体的实现，主要关注系统 具体实现的内容，涉及到实际编码细节和算法实现等。

两者之间的联系：

在于概要设计是详细设计的基础，概要设计中所规划的结构和模块关系就是为详细设计所服务的；而详细设计则需要依靠概要设计中的设计思路和规划，合理地实现整个系统的功能。

120、说明为什么在需求工程过程中区分用户需求和系统需求是重要的。

- (1) 用户和系统需求之间存在根本的区别这意味着他们应该分开考虑。用户需求是为了描述系统的功能,从用户的角度来看,这是用户理解这些需求的基本要素。他们应该用自然语言表达,可以不是很详细地表达,允许一些实现的灵活性。参与过程的人必须能够理解用户的环境和应用领域。
- (2) 系统需求比用户需求更详细并且旨在成为可能是部分的系统的精确规范系统合同。他们也可能被用在外包的开发,开发团队需要一个完整的规定应该开发什么。系统需求是在用户需求建立后开发的。

121、业务需求、用户需求、功能需求之间有何区别？

软件需求包括三个不同的层次——业务需求、用户需求和功能需求，当然也包括非功能需求。

业务需求：描述组织或客户高层次的目标。业务需求就是系统目标，它必须是业务导向、可度量、合理、可行的。业务需求是需求定义的产物；

用户需求：描述的是用户的目标，或用户要求系统必须能完成的任务。用户需求是需求捕获的结果；

功能需求：规定开发人员必须在产品中实现的软件功能，用户利用这些功能来完成任务，满足业务需求。

三者之间区别：

- (1) 业务需求：公司想如何解决用户的问题，如何满足用户的欲望，并将利益最大化。
- (2) 用户需求。用户需要在应用系统中实现什么东西，为实现这个目标，需要用户提供的全部详细的业务说明，业务流程，表格样式等。
- (3) 功能需求。将用户需求归类分解为计算机可以实现的子系统和功能模块，用设计语言描述和解释用户的需求，以达到可以指导程序设计的目的。

122、衡量模块独立性的两个标准是什么？它们表示的含义是什么？

(1) 衡量模块独立的两个标准

衡量模块的独立性的标准是两个定性的度量标准：耦合性和内聚性。耦合性与内聚性是模块独立性的两个定性标准，将软件系统划分模块时，**尽量做到高内聚低耦合，提高模块的独立性**，为设计高质量的软件结构奠定基础。

(2) 衡量模块独立的两个标准的含义：

①耦合性：也称块间联系，指软件系统结构中各模块间相互联系紧密程度的一种度量。模块之间联系越紧密，其耦合性就越强，模块的独立性则越差。模块间耦合高低取决于模块间接口的复杂性、调用的方式及传递的信息。

耦合的类型，它们由高到低分别是：内容耦合、公共耦合、外部耦合、控制耦合、标记耦合和数据耦合等。降耦方法：使用单向关联或依赖关系，通过印记耦合、通过接口、或通过数据耦合等。

- ① 内容耦合：当一个模块直接修改或操作另一个模块的数据，或者直接转入另一个模块时，就发生了内容耦合。这是最强的耦合类型，模块独立性最弱。
- ② 公共耦合：两个以上的模块共同引用一个全局数据项就称为公共耦合。这种耦合的紧密程度取决于模块对公共数据的存取方式。
- ③ 控制耦合：一个模块在界面上传递一个信号（如开关值、标志量等）控制另一个模块，接收信号的模块的动作根据信号值进行调整。
- ④ 标记耦合：模块间通过参数传递复杂的内部数据结构，这种数据结构的变化将使相关的模块发生变化。
- ⑤ 数据耦合：模块间通过参数传递基本类型的数据，这是最佳的耦合方式，因为模块之间的依赖仅限于参数传递。
- ⑥ 非直接耦合：两个模块之间没有直接关系，它们之间的联系完全是通过主模块的控制和调用来实现的。这是最弱的耦合类型，模块独立性最强。

②内聚性：又称块内联系，指模块的功能强度的度量，即一个模块内部各个元素彼此结合的紧密程度的度量。若一个模块内各元素（语句之间、程序段之间）联系得越紧密，则它的内聚性就越高。

内聚类型从低到高：偶然内聚、逻辑内聚、时间内聚、过程内聚、通信内聚（信息内聚）、顺序内聚、和功能内聚。

内聚按紧密程度从低到高：

- ① 偶然内聚：模块内部各个部分之间没有明显的关联，仅仅是偶然放在一起，这是最低级别的内聚。
- ② 逻辑内聚：模块中的元素被组织在一起，因为它们在逻辑上分类相似，通常通过一个控制语句（如 if-else 或 switch）来选择执行。
- ③ 时间内聚：模块中的元素相关联，因为它们需要在同一时间段内被执行。
- ④ 过程内聚：模块中的元素协同工作执行一个任务序列，彼此之间按照特定顺序执行。
- ⑤ 通信内聚（信息内聚）：模块中的所有功能都操作相同的数据结构，即模块内各个组成部分都使用相同的数据或产生相同的数据结构。
- ⑥ 顺序内聚：模块内的操作必须按特定顺序执行，输出数据成为下一步操作的输入数据。
- ⑦ 功能内聚：模块内所有元素都紧密围绕一个单一的功能组织，共同完成一个特定的任务，这是最高级别的内聚。

123、简述数据流图(DFD)与流程图(Flow Chart)的异同点。

数据流图(DFD)与流程图(Flow Chart)的相同：两者都是描述数据的流动情况。

数据流图(DFD)与流程图(Flow Chart)区别：

- (1) 数据流图 (data flow diagram , DFD)，是 SA 方法中用于表示系统逻辑模型的一种工具，它以图形的方式描绘数据在系统中流动和处理的过程，由于它只反映系统必须完成的逻辑功能，所以它是一种功能模型，是从数据的角度来描述一个系统的；而流程图则是从对数据加工的角度来描述系统的；
- (2) 数据流图中的箭头是数据流，而流程图中的箭头则是控制流，它表达的是程序执行的次序；
- (3) 数据流图适合于宏观地分析一个组织业务概况，而程序流程图只适合于描述系统中某个加工的执行细节。
- (4) 数据流程图应该重点描述了数据加工的过程，主要是模块内部，数据流图则是描述模块之间的关系。

124、简述软件的特点。

- 1) 软件是一种**逻辑实体**，而不是具体的物理实体，因而它具有抽象性。
- 2) 软件的生成与硬件不同，在软件的开发过程中**没有明显的制造过程**。
- 3) 在软件的运行和使用期间，**没有硬件那样的机械磨损、老化问题**。
- 4) 软件的开发和运行常常受到计算机系统的限制，对计算机系统有着不同程度的**依赖性**。
- 5) 软件开发至今尚未完全摆脱**手工的开发方式**。
- 6) 软件的**复杂性**
- 7) 软件**成本相当昂贵**
- 8) **社会性**

125、简述 CMMI 的五个级别及其特征。

CMMI 一共分五个级别，一级最低，五级最高，一般企业初次认证 CMMI 从三级开始。

- 1、CMMI 一级，**已完成级（初始级）**。在完成级水平上，企业对项目的目标与要做的努力很清晰。项目的目标得以实现。一般来说，公司的初始阶段就是一级，所以一级不需要认证。
- 2、CMMI 二级，**已管理级**。在管理级水平上，企业在项目实施上能够遵守既定的计划与流程，有资源准备，权责到人，对相关的项目实施人员有相应的培训，对整个流程有监测与控制，并与上级单位对项目与流程进行审查。二级认证要求企业有一套简单的流程，一般规范的企业基本都可以达到。
- 3、CMMI 三级，**已定义级**。在定义级水平上，企业不仅仅能够对项目的实施有一整套的管理措施，并保障项目的完成；企业能够根据自身的情况以及自己的标准流程，将这套管理体系与流程予以制度化。三级认证有一套完整的流程，所以对一般企业来说，三级是软件企业最好的选择。
- 4、CMMI 四级，**量化管理级**。量化管理级分析对软件工程和产品质量的详细度量数据，对软件过程和产品都有定量的理解与控制，管理有一个做出结论的客观依据。
- 5、CMMI 五级，**优化级**。优化管理级的量化反馈和先进的新思想、新技术促使过程持续不断改进。企业不仅能够通过信息手段与数字化手段来实现对项目的管理，而且能够充分利用信息资料，对企业在项目实施的过程中可能出现的次品予以预防。能够主动地改善流程，运用新技术，实现流程的优化。

126、需求工程包括哪些基本活动？各项基本活动的主要任务是什么？

需求工程过程包括如下主要活动：

(1)获取需求。深入实际，在充分理解用户需求的基础上，获取足够多的问题领域的知识，积极与用户交流，捕捉、分析和修订用户对目标系统的需求，并提炼出符合解决领域问题的用户需求。需求获取的方法一般有问卷法、面谈法、数据采集法、用例法、情景实例法以及基于目标的方法等。

(2)需求分析与建模。对已获取的需求进行分析和提炼，进行抽象描述，建立目标系统的概念模型，需求概念模型的要求包括实现的独立性：不模拟数据的表示和内部组织等；需求模拟技术又分为企业模拟、功能需求模拟和非功能需求模拟等。进一步对所建立的模型（原型）进行分析。需求模型的表现形式有自然语言、半形式化（如图、表、结构化英语等）和形式化表示等三种。

(3)需求规格说明。对需求模型进行精确的、形式化的描述，为计算机系统的实现提供基础。

(4)确认需求。以需求规格说明为基础输入，通过符号执行、模拟或快速原型等方法，分析和验证需求规格说明的正确性和可行性，确保需求说明准确、完整地表达系统的主要特性，就是对需求规格说明与用户达成一致。其主要任务是冲突求解，包括定义冲突和冲突求解两方面。常用的冲突求解方法有：协商、竞争、仲裁、强制、教育等，其中有些只能用人因素去控制。

(5)需求管理。在整个需求工程过程中，贯穿了需求管理活动。**需求管理主要包括跟踪和管理需求变化**，支持系统的需求演进。由于客户的需要总是不断（连续）增长的，但一般的软件开发又总是落后于客户需求的增长，如何管理需求的进化（变化）就成为软件管理的首要问题。对于传统的变化管理过程来说，其基本成分包括软件配置、软件基线和变化审查小组。当前的发展是软件家族法，即产品线方法。多视点方法也是管理需求变化的一种新方法，它可以用于管理不一致性，并进行关于变化的推理。进化需求是十分必要的。

127、

四、论述题（20 题）

1.论述面向对象方法对传统方法的优势和问题

答：面向对象方法(Object-Oriented Method)是一种把面向对象的思想应用于软件开发过程中，指导开发活动的系统方法，简称 OO (Object-Oriented)方法，是建立在“对象”概念基础上的方法学。对比传统方法，它的优点是：所开发的程序是面向对象程序，直接描述客观世界的对象及其相互关系；具有模块性、继承性和类比性；**可广泛支持对象的重用**；十分有利于开发过程的质量控制；**极大地减小了系统维护的复杂程度**。缺点是：面对大型系统时，若缺乏整体系统设计划分，易造成系统结构不合理、各部分关系失调等问题；**只能在现有业务基础上进行分类整理，不能从科学管理角度进行理顺和优化**；需要一定的软件支持环境。

2.什么是面向对象开发过程，讨论各阶段任务和要点

答：面向对象开发过程就是指利用面向对象方法进行的软件开发的完整流程，主要分为四个阶段：分析、设计、编程、测试。面向对象分析包括需求分析和需求模型化两个部分。其主要作用是明确用户的需求，并用标准化的面向对象模型来规范地表达这一需求，最后形成面向对象的分析模型。它是系统设计的依据。面向对象设计的任务是把分析阶段得到的问题域需求转变成符合要求的、抽象的系统实现方案，主要分为两个阶段：**系统设计和对象设计**。要点设计针对问题域子系统、人-机交互子系统、任务管理子系统、数据管理子系统、类中的服务和关联。编程和测试则分别是由开发人员编写程序具体代码并在系统代码开发完成后对系统整体性能和功能进行测试。

3.用覆盖的观点讨论面向对象的软件测试策略

答：软件测试覆盖包括分支覆盖，语句覆盖、条件覆盖等。

语句覆盖：它要求被测程序的每一可执行语句在测试中尽可能都检验过；

分支覆盖：要求程序中所有判定的分支尽可能得到检验；

条件覆盖：当判定式中含有多个条件时，要求每个条件的取值均得到检验；

还有其他的覆盖方式则是将这三种覆盖进行组合，使得测试覆盖更加全面，可以覆盖到程序中所有可能的路径。

由此可以看出只有对程序内部十分了解才能进行适度有效的覆盖测试。但是贯穿在程序内部的逻辑存在着不确定性和无穷性，尤其对于大规模复杂软件。因此我们不能穷举所有的逻辑路径，即使穷举也未必会带来好运（穷举不能查出程序逻辑规则错误，不能查出数据相关错误，不能查出程序遗漏的路径）。所以正确使用覆盖（白盒）测试，就要先从代码分析入手，根据不同的代码逻辑规则、语句执行情况，选用适合的覆盖方法。任何一个高效的测试用例，都是针对具体测试场景的。逻辑测试不是片面的测试正确的结果或是测试错误的结果，而是尽可能全面地覆盖每一个逻辑路径。这样才能使得测试的工作量与准确性达到一个合理的平衡。

4.结合软件工程要素，论述面向对象方法的思想

答：软件工程的方法（开发方法）、工具（支持方法的工具）、过程（管理过程）称为软件工程的三要素。方法支撑过程和工具；过程和工具促进方法学的研究。

面向对象方法（Object--Oriented Method, OOM）是一种运用对象、类、消息传递、继承、封装、聚合、多态性等概念来构造软件系统的软件开发方法。

它的优点是：所开发的程序是面向对象程序，直接描述客观世界的对象及其相互关系；具有模块性、继承性和类比性；可广泛支持对象的重用；十分有利于开发过程的质量控制；极大地减小了系统维护的复杂程度。

缺点是：面对大型系统时，若缺乏整体系统设计划分，易造成系统结构不合理、各部分关系失调等问题；只能在现有业务基础上进行分类整理，较难从科学管理角度进行理顺和优化；需要一定的软件支持环境。

因此，通过三要素的关系可以看出，面向对象方法决定了它需要一定的开发工具和严格合理的管理过程来支持，同时，在这两个条件得到满足的情况下，面向对象的方法将能够极大的方便、优化工程的开发以及管理维护。

5.论述需求工程过程，说明各阶段关系

答：需求工程过程包括需求开发和需求管理两个部分，需求开发的一般过程分为需求获取、需求分析、编写需求规格说明书（SRS）、需求验证四个阶段；

需求管理则主要包括需求基线的建立、需求变更控制以及需求跟踪等活动。需求获取是通过与用户的交流，对现有系统的观察及对任务进行分析，从而开发、捕获和修订用户的需求；需求分析是通过之前需求获取所得到的信息，为最终用户所看到的系统建立一个概念模型，作为对需求的抽象描述；由需求模型构件生成精确的形式化的描述，即需求规格说明书，它将作为用户和开发者之间的一致协议；

需求分析的结果应该通过评审、测试等手段验证它的正确性、完整性和一致性，这就是需求验证。

贯穿于整个过程中，需求管理中最基本的任务则是明确需求，并使所有相关人员达成共识；建立需求跟踪能力联系链，确保所有用户需求被正确地应用，并且在需求发生变更时，能够完全地控制其影响范围，始终保持产品与需求的一致性。

6.什么是软件可维护性，讨论软件工程与可维护性关系

答：软件可维护性即维护人员对该软件进行维护的难易程度，具体包括理解、改正、改动和改进该软件的难易程度。软件工程是一门研究用工程化方法构建和维护有效的、实用的和高质量的软件的学科。

软件工程的目标是：在给定成本、进度的前提下，开发出具有适用性、有效性、可修改性、可靠性、可理解性、可维护性、可重用性、可移植性、可追踪性、可互操作性和满足用户需求的软件产品。软件交付使用后，需要能够对它进行修改，以改正潜伏的错误，改进性能和其它属性，使软件产品适应环境的变化等。软件维护费用在软件开发费用中占有很大的比重。因此，可维护性是软件工程中一项十分重要的目标。

7.论述软件工程中的文档类型和作用。*

答：按照文档产生和使用的范围，软件文档大致可分为三类：开发文档、管理文档、用户文档。

一、开发文档

定义：开发文档是描述软件技术实现细节的文档，供开发、测试和维护人员使用，贯穿需求、设计、编码、测试等阶段。

1. 需求规格说明书（SRS）

作用：

明确系统功能、性能、接口等需求，作为设计与验收的基准。

通过用例、数据流图等形式描述用户需求，避免开发偏差。

示例内容：用户场景、非功能性需求（如并发量、响应时间）、数据字典。

2. 系统设计文档 (SDD)

作用：

定义系统架构（如分层架构、微服务）、模块划分及交互逻辑。

指导开发团队分工协作，确保技术方案一致性。

示例内容：架构图、模块接口定义、数据库 ER 图、API 规范。

3. 详细设计说明书

作用：

细化模块内部逻辑，指导具体编码实现。

提供算法描述、类图、流程图等，降低编码复杂度。

示例内容：类属性与方法定义、伪代码、输入输出规则。

4. 测试计划与用例

作用：

规划测试策略（单元测试、集成测试、性能测试等），确保需求覆盖率。

提供可执行的测试场景，验证系统是否符合预期。

示例内容：测试环境配置、测试用例（正向/反向场景）、缺陷追踪流程。

5. 技术白皮书

作用：

解释核心技术的实现原理（如加密算法、分布式架构），供技术评审或外部合作参考。

示例内容：技术选型依据、性能优化方案、第三方库依赖说明。

二、管理文档

定义：管理文档用于项目规划、资源协调和过程控制，服务于项目经理、质量保证团队及利益相关者。

1. 项目计划书

作用：

明确项目目标、里程碑、资源分配（人力、预算、时间）。

作为项目执行的基准，指导团队分工与进度跟踪。

示例内容：WBS（工作分解结构）、甘特图、风险管理计划。

2. 进度报告

作用：

定期同步项目进展，暴露延期风险或资源瓶颈。

向管理层汇报关键成果和下一步计划。

示例内容：已完成任务、未完成任务及原因、调整后的时间表。

3. 变更控制文档

作用：

记录需求或设计的变更请求，评估影响并跟踪执行。

防止无序变更导致项目失控。

示例内容：变更申请单、影响分析（成本、进度）、变更审批记录。

4. 质量保证计划 (QAP)

作用：

制定质量标准（如代码覆盖率、缺陷密度），确保交付质量。

定义代码审查、测试覆盖等质量控制流程。

示例内容：质量指标、审计计划、验收标准。

5. 风险管理计划

作用：

识别潜在风险（如技术难点、第三方依赖延迟），制定应对策略。

降低项目失败概率。

示例内容：风险清单（概率/影响矩阵）、应急预案、风险责任人。

三、用户文档

定义：用户文档面向最终用户或运维人员，帮助其理解、使用和维护系统。

1. 用户手册

作用：

提供系统操作指南（如功能使用步骤、界面说明），降低用户学习成本。

常见形式：PDF 手册、在线帮助页面。

示例内容：登录流程、功能截图、常见问题解答（FAQ）。

2. 安装部署指南

作用：

指导运维人员完成系统安装、环境配置（如服务器、数据库）。

确保系统正确部署并运行。

示例内容：软硬件要求、配置参数说明、故障排查步骤。

3. 维护手册

作用：

描述系统维护流程（如数据备份、日志清理）、故障恢复方法。

延长系统生命周期，保障稳定性。

示例内容：日常维护任务清单、错误代码解释、联系支持渠道。

4. 发布说明（Release Notes）

作用：

通告版本更新内容（新增功能、修复缺陷），帮助用户了解升级影响。

提供升级步骤和兼容性说明。

示例内容：版本号、更新日志、已知问题、升级注意事项。

5. 培训材料

作用：

辅助用户或运维人员快速掌握系统核心功能。

常见形式：PPT 课件、操作视频、模拟演练脚本。

示例内容：培训大纲、实操案例、考核试题。

四、文档的核心价值

知识传递：沉淀技术方案与管理经验，减少人员流动带来的信息丢失。

过程可控：通过规范化文档确保开发流程透明、可追溯。

质量保障：明确需求与验收标准，降低交付偏差风险。

协作基础：为跨团队、跨角色沟通提供统一依据。

通过系统化的文档管理，软件工程活动得以从“人治”转向“法治”，显著提升项目的可预测性和成功率。

8. 论述需求工程的过程和方法，讨论其有效性

答：需求工程包括需求开发和管理，而需求开发又包括这几个过程：需求获取，需求分析，需求规格说明和需求验证。

在需求开发之前，还需要有一个知识培训的过程，需求工程也是一个项目工程，因此也包括了项目的管理。

对于这些过程，有以下方法可以采用。

1、知识培训：需求分析员培训、用户培训、开发人员培训；

2、需求获取：项目范围确定、用户确定、用例确定、系统事件和响应、获取方法、检查完善；

3、需求分析：绘制关联图、原型开发、需求优先顺序别、需求建模、数据字典创建、子系统。

在知识培训中，通过对三方的培训，可以使分析员提高沟通能力，理解并掌握需求工程技能，使用户明白需求重要性，能够准确描述需求，使开发人员对用户应用有所了解。

通过需求获取中的方法，可以明确项目目标、确定用户群，通过用例迅速明确项目功能需求，这些可以通过会议讨论，观察用户工作过程，问答式对话，诱发式需求诱导等实现。

而在需求分析阶段则是对用户的需求获取之后的一个粗加工过程，需要对需求进行推敲和润色以使所有涉众都能准确理解需求。分析过程首先需要对需求进行检查，以保证需求的正确性和完备性，然后将高层需求分解成具体的细节，创建开发原型，完成需求从需求获取人员到开发人员的过渡。

9.讨论软件测试与软件可靠性的关系，如何计算可靠性

答：软件测试的目的是发现软件错误，排除软件缺陷，提升软件可靠性。软件可靠性可定义为在规定的条件下，在规定的时间内，软件不引起系统失效的概率或在规定的时期内，在所述条件下程序执行所要求的功能的能力。

据此，可以在测试时间内，对软件各个模块进行测试统计，得到每个功能块的可靠性，功能串联时可靠性 $R=R_1 \times R_2 \times \dots \times R_N$ ，并联时 $R=1 - (1-R_1)(1-R_2)\dots(1-R_N)$ 。混联则综合两式进行计算。

10.什么是软件可测试性，讨论软件工程与软件可测试性

答：软件的可测试性是指软件**发现故障并隔离、定位其故障的能力特性**，以及在一定的时间和成本前提下，进行测试设计、测试执行的能力。软件可测试性就是一个计算机程序能够被测试的容易程度。软件工程是一门研究用工程化方法构建和维护有效的、实用的和高质量的软件的学科。

软件工程的目的是：在给定成本、进度的前提下，开发出具有适用性、有效性、可修改性、可靠性、可理解性、可维护性、可重用性、可移植性、可追踪性、可互操作性和满足用户需求的软件产品。

而软件在完成后，必须保证其能够防止因概念、设计和结构等方面的不完善造成的软件系统失效，具有挽回因操作不当造成软件系统失效的能力，也就是要达成软件可靠性。软件测试正是保证软件可靠性的一项非常重要的技术保障。因此，在软件工程中，必须在软件设计时就考虑好软件的可测试性，这样才能在软件完成后通过测试，调整改进软件的不足，满足其可靠性要求

11.论述软件测试的基本任务，方法和策略及其在现代软件工程的地位

测试的目的（来源老师 PPT）：

- 1) 测试是为了证明程序有错，而不是证明程序无错误；
- 2) 一个好的测试用例在于能够发现至今未发现的错误；
- 3) 一个成功的测试是发现了至今未发现的错误的测试。

测试的方法：

- 1) 测试方法概述
- 2) 白盒测试方法
- 3) 黑盒测试方法
- 4) 其他测试方法

测试的策略：

按阶段进行测试是一种基本的测试策略

- 1) 单元测试
- 2) 集成测试
- 3) 系统测试
- 4) 确认测试

软件测试在软件工程中的地位：覆盖整个生命周期。从需求阶段已介入，编写测试计划、测试用例、测试总结等。

12.论述软件测试与软件质量的关系

软件质量是指软件产品的特性可以满足用户的功能、性能需求的能力。软件过程是人们通常所说的软件生命周期中的活动，一般包括软件需求分析、软件设计、软件编码、软件测试、交付、安装和软件维护。随着软件过程的开始，软件质量也逐渐建立起来。软件过程的优劣决定了软件质量的高低，好的过程是高效高质量的前提。人员和过程是决定软件质量的关键因素。高质量的人员和好的过程应该得到好的产品。

软件系统的开发包括一系列生产活动，其中由人带来的错误因素非常多，错误可能出现在程序的最初需求分析阶段，设计目标可能是错误的或描述不完整，也可能在后期的设计和开发阶段，因为人员之间的交流不够，交流上有误解或者根本不进行交流，所以尽管人们在开发软件的过程中使用了许多保证软件质量的方法和技术，单开发出的软件中还会隐藏许多错误和缺陷。可见，只有通过严格的软件测试，才能很好的提高软件质量，而软件质量并不是依靠软件测试来保证的，软件的质量要靠不断的提高技术水平和改进软件开发过程来保证，软件测试只是一种有效的提高软件质量的技术手段，而不是软件质量的安全网。

软件测试能够找出软件缺陷，确保软件产品满足需求。但是测试不是质量保证，二者并不等同。测试可以查找错误

并进行修改,从而提高软件产品的质量。软件测试避免错误以求高质量,并且还有其他方面的措施以保证质量问题,如软件质量保证。

正规的软件测试系统主要包括:制定测试计划、测试设计、实施测试、建立和更新测试文档。而软件质量保证的工作主要为:制定软件质量要求、组织正式度量、软件测试管理、对软件的变更进行控制、对软件质量进行度量、对软件质量情况及时记录和报告。软件质量保证的职能是向管理层提供正确的可行信息,从而促进和辅助设计流程的改进。软件质量保证的职能还包括监督测试流程,这样测试工作就可以被客观地审查和评估,同时也有助于测试流程的改进。二者的不同之处在于软件质量保证工作侧重对软件开发流程中的各个过程进行管理与控制,杜绝软件缺陷的产生。而测试则是对已产生的软件缺陷进行修复。

13.讨论不同软件过程模型与软件开发的关系

常见的过程模型有瀑布模型、快速原型模型、增量模型、螺旋模型、喷泉模型等。

软件过程是为了获得高质量软件所需要完成的一系列任务的框架,它规定了完成各项任务的工作步骤。通常使用生命周期模型简洁地描述软件过程。生命周期模型规定了把生命周期划分成哪些阶段及各个阶段的执行顺序,因此,也称为过程模型。常见的过程模型有瀑布模型、快速原型模型、增量模型、螺旋模型、喷泉模型等。

1.瀑布模型

这个特点有两重含义:

1.必须等前一阶段的工作完成之后,才能开始后一阶段的工作;

2.前一阶段的输出文档就是后一阶段的输入文档,因此,只有前一阶段的输出文档正确,后一阶段的工作才能获得正确的结果。

瀑布模型每个阶段都应坚持两个重要做法:

1.每个阶段都必须完成规定的文档,没有交出合格的文档就是没有完成该阶段的任务。完整、准确的合格文档是软件开发时期各类人员之间相互通信的媒介,也是运行时期对软件进行维护的重要依据。

2.每个阶段结束前都要对所完成的文档进行评审,以便迟早发现问题,改正错误。事实上越是早期阶段犯下的错误,暴露出来的时间就越晚,排除故障改正错误所需付出的代价也越高。因此,及时审查,是保证软件质量,降低软件成本的重要措施。

可以说瀑布模型是由文档驱动的。这个事实也是它的一个缺点,在可运行的软件产品交付给用户之前,用户只能通过文档来了解产品是什么样的。瀑布模型历史悠久、广为人知的,它的优势在于它是规范的、文档驱动的方法;这种模型的问题是,最终开发出的产品可能并不是用户真正需要的。

(1) 传统的瀑布模型: (2) 实际的瀑布模型:

2.快速原型模型

所谓快速原型是快速建立起来的可以在计算机上运行的程序,它所能完成的功能往往是最终产品能完成的功能的一个子集。快速原型的本质是“快速”,开发人员应该尽可能快地建造出原型系统,以加速软件开发过程,节约软件开发成本。原型的作用是获知用户的真正需求,一旦需求确定了,原型系统将被抛弃。

快速原型模型正是为了克服瀑布模型的缺点而提出来的。它通过快速构建一个可在计算机上运行的原型系统,让用户试用原型系统并收集用户反馈意见的办法,获取用户的真实需求。

3.增量模型

增量模型也称为渐增模型,使用增量模型开发软件时,把软件产品作为一系列的增量构件来设计、编码、集成和测试。每个构件由多个相互作用的模块构成,并且能够完成特定的功能。使用增量模型时,第一个增量构件往往实现软件的基本需求,提供最核心的功能。

优点:

1.能在较短的时间内向用户提交可完成部分工作的产品。

2.逐步增加产品功能可以使用户有充裕的时间学习和适应新产品,从而减少一个全新的软件可能给客户组织带来的冲击。

增量模型具有可在软件开发的早期阶段使投资获得明显回报和较易维护的优点,但是,要求软件具有开放的结构是使用这种模型时固有的困难。

4.螺旋模型

螺旋模型的基本思想就是,使用原型及其他方法来尽量降低风险。理解这种模型的一个简便方法,是把它看作每个阶段之前都增加了风险分析过程的快速原型模型。

螺旋模型主要适用于内部开发的大规模软件项目。如果进行风险分析的费用接近整个项目的经费预算,则风险分

析是不可行的。事实上项目越大，风险也越大，因此进行风险分析的必要性也越大。此外只有内部开发的项目，才能在风险过大时方便中止项目。

螺旋模型的主要优势在于，它是风险驱动，但是，这也可能是它的一个弱点。除非软件开发人员具有丰富的风险评估经验和这方面的专门知识，否则将出现真正的风险：当项目实际上正在走向灾难时，开发人员可能还认为一切正常。

风险驱动的螺旋模型适用于内部开发的大型软件项目，但是，只有在开发人员具有风险分析和排除风险的经验及专门知识时，使用这种模型才会获得成功。

(1) 简化的螺旋模型 (2) 完整的螺旋模型

4. 喷泉模型

喷泉模型对软件复用和生存周期中多项开发活动的集成提供了支持，以面向对象的软件开发方法为基础，它适合面向对象的开发方法。它克服了瀑布模型不支持软件重用和多项开发活动集成的局限性。喷泉模型使开发过程具有迭代性和无间隙性。系统某个部分常常重复工作多次，相关功能在每次迭代中随之加入演化的系统。无间隙是指在分析、设计和实现等开发活动之间不存在明显的边界。

按照在软件生命周期过程中应完成的任务的性质，在概念上可以把软件生命周期划分成定义、可行性研究、需求分析、总体设计、详细设计、编码和单元测试、综合测试以及运行维护等 8 个阶段。实际从事软件开发工作时，软件规模、种类、开发环境及使用的技术方法等因素，都影响各阶段的划分。

软件过程是为了获得高质量的软件产品所需要完成的一系列任务的框架，它规定了完成各项任务的工作步骤。由于没有适用所有软件项目的任务集合，科学、有效的软件过程应该定义一组适合所承担的项目特点的任务集合。通常使用软件过程模型简洁地描述软件过程，它规定了把软件生命周期划分成的阶段及各个阶段的顺序。

14. 针对敏捷方法的十二条核心实践，结合软件工程问题论述其有效性

PPT 参考答案：

1. On-Site Customer (现场客户)：客户是 Team 成员，在开发现场和开发人员一起工作。
2. 计划项目 (Planning Game)
3. 频繁地小规模发布软件 (Small Releases)
4. 简单设计 (Simple Design)
5. 测试驱动开发 (Test Driven Development)
6. 持续集成 (Continuous Integration)
7. 集体拥有代码 (Collective Code Ownership)
8. 编程规范 (Coding Standards)
9. 重构 (Refactoring)
10. System Metaphor (系统隐喻)
11. Pair Programming (结对编程)
12. 平稳的工作效率 (Sustainable Pace)

敏捷开发中有 12 条原则，它们是敏捷实践区别于重型过程的特征所在。

1. 我们最优先要做的是通过尽早的、持续的交付有价值的软件来使客户满意。
2. 即使到了开发的后期，也欢迎改变需求。敏捷过程利用变化来为客户创造竞争优势。
3. 经常性的交付可以工作的软件，交付的间隔可以从几周到几个月，交付的时间间隔越短越好。
4. 在整个项目开发期间，业务人员和开发人员必须天天都在一起工作。
5. 围绕被激励起来的人来构建项目。给他们提供所需要的环境和支持，并且信任他们能够完成工作。
6. 在团队内部，最有效果并且富有效率的传递信息的方法，就是面对面的交谈。
7. 工作的软件是首要进度度量标准。
8. 敏捷过程提可持续的开发速度。责任人、开发者和用户应该能够保持一个长期的、恒定的开发速度。
9. 不断地关注优秀的技能和好的设计会增强敏捷能力。
10. 简单——使未完成的工作最大化的艺术——是根本的。
11. 最好的构架、需求和设计出自自组织的团队。
12. 每隔一定时间，团队会在如何才能更有效地工作方面进行反省，然后相应地对自己的行为进行调整。

15. 针对软件危机，论述传统软件工程方法和敏捷方法的解决思路

PPT 参考：

软件危机出现于 20 世纪 60 年代末，软件危机是指在计算机软件的开发和维护过程中遇到的一系列严重问题。比如：

- 1) 软件开发的成本和进度难以准确估计，延迟交付甚至取消项目的现象屡见不鲜
- 2) 软件存在着错误多、性能低、不可靠、不安全等质量问题
- 3) 软件维护极其困难，而且很难适应不断变化的用户需求

为了解决“软件危机”，许多计算机和软件科学家参照技术过程的一般模式提出了软件生存期概念（Life cycle）及其瀑布模型（Waterfall Model）。计算机软件的生存周期描述了软件孕育、诞生、成长、成熟、衰亡的生存过程：

制定计划、需求分析、软件设计、程序编制、软件测试、运行维护。

但是，传统软件工程方法推行多年以来，人们对于软件危机的“恐惧”仍没有丝毫减弱，相反随着软件系统的急速膨胀而增强，表现为：对软件开发成本和进度的估计常常不准确，开发成本超出预算，实际进度比预定计划一再拖延的现象并不罕见；用户对“已完成”系统不满意的现象也经常发生；软件产品的质量往往靠不住，Bug 一大堆，补丁一个接一个等等。于是，无论是产业界还是理论界，都开始对传统软件工程思想产生怀疑，甚至背叛，于是以“人”为核心的敏捷开发方法就是在这样的背景下产生的，它背叛了传统软件工程中以“过程”为核心，把设计和开发尽可能分开，尽量弱化“人”在整个工程中地位的思想。

敏捷开发实际上包括了许多优秀的软件开发习惯。首先，这种方法改变了软件测试的流程，在编写代码前进行测试，减少了开发风险；此外，通过这种方法，可以对软件进行持续集成，即每个小时都在集成，任何部件间的冲突都可以随时解决；这种方法的“动态规划”和“重构”做法，意味着开发者可以对软件架构进行持续改进，可以根据用户的需求随时进行改进，而利用传统的软件开发方法则很难对软件的架构进行调整，同时也会造成成本的大幅增加。

16. 论述过程、方法和工具在软件工程实践中的关系

软件过程、方法和工具是软件工程的三个核心要素。

其中，软件过程是指在软件开发的整个生命周期中，按照一定的规范和流程进行的一系列活动，包括需求分析、设计、编码、测试、维护等。

软件方法是指在软件开发过程中，采用的一种或多种技术和方法，如面向对象、结构化分析与设计等。软件方法是为软件开发提供了“如何做”的技术。

而软件工具则是指在软件开发过程中使用的各种辅助工具。软件过程将各个技术层次结合在一起，这三者之间相互依存，相互作用，共同构成了软件工程的核心体系，使得合理及时地开发计算机软件成为可能。

17. 针对不同软件方法，论述项目管理要点

软件研究人员也在不断探索新的软件开发方法。至今已形成了八类软件开发方法。

一、Parnas 方法

此方法在 1972 年提出的。主要针对软件在可维护性和可靠性方面存在着严重问题，主要提出了信息隐蔽原则：在概要设计时列出将来可能发生变化的因素，并在模块划分时将这些因素放到个别模块的内部。第二条原则是在软件设计时应应对可能发生的种种意外故障采取措施。软件是很脆弱的，很可能因为一个微小的错误而引发严重的事故，所以必须加强防范。。

二、SASA 方法

1978 年，提出了结构化方法，即 SASD 方法，也可称为面向功能的软件开发方法或面向数据流的软件开发方法，首先用结构化分析（SA）对软件进行需求分析，然后用结构化设计（SD）方法进行总体设计，最后是结构化编程（SP）。

三、面向数据结构的软件开发方法

1、Jackson 方法：从目标系统的输入、输出数据结构入手，导出程序框架结构，再补充其它细节，就可得到完整的程序结构图。此方法有时也称为面向数据结构的软件设计方法。

2、Warnier 方法

与 Jackson 方法类似。差别有三点：一是它们使用的图形工具不同，分别使用 Warnier 图和 Jackson 图；另一个差别是使用的伪码不同；最主要的差别是在构造程序框架时，Warnier 方法仅考虑输入数据结构，而 Jackson 方法不仅考虑输入数据结构，而且还考虑输出数据结构。

四、问题分析法

基本思想是：考虑到输入、输出数据结构，指导系统的分解，在系统分析指导下逐步综合。这一方法的具体步骤是：从输入、输出数据结构导出基本处理框；分析这些处理框之间的先后关系；按先后关系逐步综合处理框，直到画

出整个系统的 PAD 图。PAM 方法的另一个优点是使用 PAD 图。这是一种二维树形结构图，是到目前为止最好的详细设计表示方法之一，远远优于 NS 图 and PDL 语言。

五、面向对象的软件开发方法

随着 OOP（面向对象编程）向 OOD（面向对象设计）和 OOA（面向对象分析）的发展，最终形成面向对象的软件开发方法 OMT(Object Modelling Technique)。这是一种自底向上和自顶向下相结合的方法，而且它以对象建模为基础，从而不仅考虑了输入、输出数据结构，实际上也包含了所有对象的数据结构。

六、可视化开发方法

用户界面在软件系统中所占的比例也越来越大，有的甚至高达 60~70%。产生这一问题的原因是图形界面元素的生成很不方便。可视化开发就是在可视开发工具提供的图形用户界面上，通过操作界面元素，诸如菜单、按钮、对话框、编辑框、单选框、复选框、列表框和滚动条等，由可视开发工具自动生成应用软件。

七、ICASE

提高人类的劳动生产率，提高生产的自动化程度，一直是人类坚持不懈的追求目标。软件开发也不例外。

八、软件重用和组件连接

软件重用 (Reuse) 又称软件复用或软件再用。软件人工制品可以是源代码片断、子系统的设计结构、模块的详细设计、文档和某一方面的规范说明等。所以软件重用是利用已有的软件成份来构造新的软件。它可以大大减少软件开发所需的费用和时间，且有利于提高软件的可维护性和可靠性。

18. 论述软件项目特点和管理要点

软件项目的特点

复杂性：软件项目通常由多个模块和组件组成，这些模块和组件之间存在复杂的相互作用关系。软件项目的开发和管理需要高度的技术和专业知识。

不确定性：软件项目的需求和规格说明通常是不完整、不准确和不一致的。此外，软件项目的开发过程中可能会出现各种问题的挑战，例如技术难题、人员变动、进度延误等。

可变性：软件项目的需求和规格说明可能会随着时间的推移而发生变化。因此软件项目管理需要具备灵活性和适应性，以便及时应对变化。

风险性：软件项目的开发和管理涉及到多种风险，例如技术风险、进度风险、成本风险等。因此，软件项目管理需要具体风险管理能力，以便及时识别、评估和应对各种风险。

软件项目管理中，需要遵循一定的原则和方法，以确保软件项目的成功。具体而言，软件项目管理需要考虑以下要点：

计划：制定详细的计划，并根据实际情况进行调整

进度控制：监控项目的进度，并及时采取措施解决问题

人员管理：合理分配人力资源，并保证团队协作

质量控制：确保软件产品符合质量标准，并及时修改缺陷

风险管理：识别、评估和应对各种风险

19. 论述软件估算的困难和解决方法

软件成本估算是对于将要开发的或正在开发的软件项目所需要的工作量和工作进度作出预测，从而产生出一组在接受误差范围内的近似规划。

成本估算的困难：

1) 需求信息的复杂性：与其他有些传统项目不同，信息系统要满足的人的主观需要。由于人的复杂性，给信息系统带来了无数的难以确定的因素。而且，随着项目的进展，许多具体情况的明确，项目的成本估算也会相应的有所变化。

2) 开发技术和工具的不断变化：开发工具软件的不断升级，技术方案不断更新，这些技术的进步让信息系统项目可以提供功能越来越强，但是给信息系统项目的成本估算带来困难。

3) 缺乏类似项目估算数据可供参考：有效的项目成本估算是建立在大量的同类项目的成本估算的基础上的。没有大量的同类项目的经验，信息系统项目的成本估算也就非常困难了。

4) 缺乏专业和富有经验的人才。

5) 信息系统研发人员技术能力的差异：不同人员的不同态度、经验和能力都会造成不同人员的截然不同的效率，

这也给信息系统的成本估算带来了困难。

6.) 管理层的压力与误解。

成本估算的方法：

1) 工作分解结构方法 (WBS)

2) 专家判断

3) 类比估算

4) 其他估算方法：德尔菲法(Delphi technique)、COCOMO 模型、特征点(feature point)、对象点(object point)、3-D 功能点(3-D function points, 数据、功能、控制)、Bang 度量(DeMarco's bang metric)、模糊逻辑(fuzzy logic)、标准构件法(standard component)等

5) 基于问题的估算

20.什么是软件性能，论述软件开发中如何提高软件的性能

软件的性能是软件的一种非功能特性，它关注的不是软件是否能够完成特定的功能，而是在完成该功能时展示出来的及时性。由于感受软件性能的主体是人，不同的人对于同样的软件能有不同的主观感受，而且不同的人对于软件性能关心的视角也不同。包括系统响应时间和应用延迟时间、吞吐量、并发用户数、资源利用率。

21、论述软件开发中的不确定性及应对方法？

软件开发中的不确定性来源

1.需求不确定性：客户需求模糊、频繁变更或隐性需求未被识别。

案例：某电商项目初期仅要求“支持在线支付”，开发中客户突然要求集成加密货币支付，导致架构需重构。

2. 技术不确定性：新技术选型风险（如性能不足）、第三方服务不可靠（如 API 接口变动）。

案例：团队采用某开源框架开发，版本升级后出现兼容性问题，修复耗时两周。

3. 过程不确定性：任务估算偏差、沟通低效、团队协作问题。

案例：瀑布模型中，测试阶段发现需求理解错误，返工成本高昂。

4. 环境不确定性：政策法规变化（如数据隐私法）、市场竞争压力或突发危机（如疫情导致远程协作）。

案例：某医疗软件因 GDPR 法规调整，需紧急重构数据存储逻辑。

研发项目中的不确定性和复杂性是常见的挑战，需要采取一系列的策略和方法来应对。下面从需求管理、团队合作、资源规划、风险管理和沟通协调等方面分别进行讨论。

一、需求管理

1.清晰的需求定义：确保项目团队对于项目的目标和要求有清晰一致的理解，减少后期变更和不确定性带来的影响。

2.迭代开发：采用敏捷开发方法，将项目拆分成多个迭代周期，每个周期内集中处理一部分需求，以降低整体不确定性和复杂性的影响。

3.需求优先级排序：根据项目的关键性和风险程度，对需求进行优先级排序，先解决高优先级的需求，减少不确定性对项目进度的影响。

二、团队合作

1.构建高效团队：选择合适的团队成员，具备专业知识和沟通能力，能够快速适应变化和解决问题。

2.跨部门协作：加强与其他团队和部门的协作，及时沟通和共享信息，减少信息孤岛和决策不一致的风险。

3.良好的沟通与协调：建立良好的沟通机制，保持团队成员之间的密切合作和信息的畅通流动，及时解决问题。

三、资源规划

1.合理的资源调配：根据项目的不确定性和复杂性的变化，灵活调整人力、物力和财力的分配，保证项目顺利进行。

2.资源约束管理：在资源有限的情况下，合理优化资源利用效率，减少资源的浪费和冲突，提高项目的成功率。

3.预留资源缓冲：考虑到可能出现的不确定性和复杂性，适当预留一部分资源作为缓冲，以应对项目过程中的意外情况。

四、风险管理

1.风险识别与评估：及早识别和评估项目中的风险，制定相应的应对措施，降低不确定性和复杂性的影响。

2.风险监控与控制：定期进行风险管理的检查和评估，及时调整项目计划和资源配置，控制风险的扩大。

3.持续学习与改进：根据项目过程中的不确定性和复杂性的经验教训，总结经验，提炼方法论，为未来的项目提供参考和改进。

五、沟通协调

- 1.定期项目会议：定期召开项目会议，分享项目进展，解决问题，及时调整项目方向和策略，保持团队的信息共享和协作。
 - 2.有效沟通渠道：建立多样化的沟通渠道，包括面对面、在线会议、邮件等，保持及时和有效的沟通和信息流通。
 - 3.情绪管理和冲突解决：及时处理团队成员之间的冲突和压力，保持团队合作和项目稳定性。
- 总结而言，研发项目中的不确定性和复杂性需要通过需求管理、团队合作、资源规划、风险管理和沟通协调等方法来应对。在实践过程中，灵活应用合适的策略和方法，能够更好地应对项目中的变化和挑战，提高项目的成功率和交付质量。

22、对于人的健康来讲“预防为主”比较好理解，软件工程也主张适当提高预防成本来降低整体的质量成本，对于软件来讲如何体现预防为主的思想？

一、引言

在当今的信息化时代，软件已经成为企业和个人日常生活中不可或缺的一部分。软件项目的质量不仅关系到用户的体验和满意度，还直接影响企业的生存和发展。因此，预防软件项目质量问题至关重要。本篇文档将详细介绍软件项目质量预防措施，帮助企业和开发团队提高软件质量，降低项目风险。

二、软件项目质量概述

软件项目质量是指软件产品满足用户需求和期望的程度。软件项目的质量取决于多个因素，包括需求分析、设计、编码、测试、部署等。在软件开发过程中，任何一个环节的失误都可能导致软件项目质量的下降。

三、软件项目质量预防措施

1.需求分析阶段

需求分析是软件项目的基础，准确的需求分析是保证软件项目质量的关键。在需求分析阶段，应采取以下预防措施：

- (1) 充分了解用户需求和期望，包括功能需求、性能需求、安全需求等。
- (2) 与用户进行充分沟通，确保对用户需求的理解无误。
- (3) 对需求进行详细的分析和评估，确保需求的合理性和可行性。
- (4) 制定需求变更应对策略，以应对可能的需求变更。

2.设计阶段

设计阶段是软件开发过程中至关重要的环节，良好的设计可以避免后期开发过程中的错误和返工。在设计阶段，应采取以下预防措施：

- (1) 根据需求分析结果，制定相应的设计文档和架构。
- (2) 遵循设计原则和最佳实践，确保设计的合理性和可维护性。
- (3) 进行设计评审，邀请领域专家和经验丰富的开发人员对设计进行评估和改进。
- (4) 在设计阶段引入技术预研，以便更好地选择适合的技术和工具。

3.编码阶段

编码阶段是实现软件设计的关键环节，也是最容易出现错误的环节之一。在编码阶段，应采取以下预防措施：

- (1) 遵循编码规范和标准，确保代码的可读性和可维护性。
- (2) 引入代码审查机制，对代码进行审核和把关。
- (3) 采用适当的开发方法和工具，提高代码质量和生产效率。
- (4) 重视代码测试和调试，确保代码的正确性和稳定性。

4.测试阶段

测试是保证软件项目质量的必要环节。在测试阶段，应采取以下预防措施：

- (1) 制定详细的测试计划和用例，确保测试的全面性和有效性。
- (2) 采用自动化测试工具和框架，提高测试效率和准确性。
- (3) 进行性能测试、安全测试等多种类型的测试，确保软件项目的各个方面的质量。
- (4) 及时处理和报告测试中发现的问题，进行缺陷跟踪和管理。

5.部署阶段

部署是将软件项目发布到生产环境的关键环节。在部署阶段，应采取以下预防措施：

- (1) 制定详细的部署计划和流程，确保部署的顺利进行。
- (2) 遵循配置管理和版本控制原则，避免出现配置混乱和版本冲突等问题。
- (3) 进行生产环境的监控和维护，确保软件的稳定性和安全性。
- (4) 及时收集用户反馈和意见，以便对软件项目进行持续改进和优化。

6. 项目管理阶段

项目管理是保证软件项目顺利实施的关键环节。在项目管理阶段，应采取以下预防措施：

- (1) 制定合理的项目计划和时间表，确保项目按期完成和质量达标。
- (2) 采用敏捷开发方法或传统瀑布模型等方法进行项目管理，确保项目管理的有效性和灵活性。
- (3) 进行项目风险管理，及时识别和应对潜在的项目风险。
- (4) 加强团队沟通和协作，提高团队凝聚力和工作效率。

23、论述度量对软件工程的作用及存在的困难。

软件度量是指通过定量的测量方法来评估软件产品、过程和项目的属性或特征。它是软件工程中不可或缺的一部分，能够帮助开发者和管理者更好地了解软件的质量、进度和成本等方面的指标。本文将介绍软件度量的概念、分类和应用，并探讨其在软件工程中的重要性。

软件度量可以按不同的角度进行分类，常见的分类方式有以下几种：

1. 功能度量：评估软件的功能性能，如用户需求的满足程度、功能是否完备等。
2. 成本度量：衡量软件开发和维护的成本，包括人力、时间、物力等方面的评估。
3. 质量度量：评价软件的质量特性，如可靠性、可维护性、可测试性等。
4. 复杂度度量：评估软件的复杂性，包括代码行数、函数调用关系、模块依赖关系等。
5. 效率度量：评估软件的运行效率和资源利用率，如响应时间、内存占用等。

软件度量在软件工程中具有广泛的应用，以下是几个常见的应用领域：

1. 项目管理：通过对软件开发过程进行度量，可以评估项目的进度、风险和成本等，帮助项目经理制定合理的计划和管理策略。
2. 质量控制：通过度量软件的质量特性，可以及时发现和解决潜在的质量问题，提高软件的可靠性和稳定性。
3. 产品评估：通过对软件功能、性能和用户满意度等方面的度量，可以评估产品的优劣和市场竞争能力。
4. 过程改进：通过度量软件开发过程中的各种指标，可以找出优化空间，改进开发方法和流程，提高软件开发效率。
5. 决策支持：通过对软件相关指标的度量，可以为管理层提供决策依据，帮助其制定战略和战术决策。

软件度量在软件工程中的重要性不可忽视。它不仅提供了对软件产品和过程的量化评估，还能够帮助开发团队和管理者发现问题、改进流程、提高质量和效率。通过精确的度量数据，可以更好地管理软件项目，减少风险，并为决策提供科学依据。

软件度量中遇到的困难主要包括以下几个方面：

- (1) 需求不清晰或变动频繁：在很多项目中，需求的不明确或频繁变动是影响测试质量和效率的一个常见痛点。如果在项目初期，需求没有得到充分确认或经常变动，测试人员难以根据准确需求来制定测试计划，容易导致测试用例设计不充分、测试执行无效等问题。
- (2) 测试环境不稳定或配置不一致：测试环境与生产环境的不一致，或者测试环境的不稳定，是许多测试项目中的隐形痛点。测试人员无法在与真实生产环境相似的环境中执行测试，导致测试结果不准确、测试周期延长。
- (3) 测试覆盖不足：时间紧迫和资源不足是测试团队面临的常见难题。在这种情况下，测试的覆盖面会被压缩，导致很多边界条件、异常情况得不到验证，从而埋下了隐患。
- (4) 开发与测试之间的沟通障碍：开发和测试团队之间的沟通不畅，常常导致工作效率低下、重复劳动，甚至项目延误。测试人员和开发人员在需求理解、Bug 反馈、版本更新等方面的沟通不及时，往往会导致测试遗漏、开发任务重复等问题。
- (5) 战略目标与测量程序的对齐问题：在软件工程组织中，战略目标的制定与测量程序的设计需要高度一致。研究中发现，学生在理解如何将战略目标与测量程序正确对齐方面存在困难。
- (6) 测量程序的效率问题：许多学生在设计指标时遇到了时间管理上的困难，导致需要的时间是其他团队的两倍。这主要是因为他们没有将测量结果分类和组织，影响了指标设计和获取结果的方法的理解。
- (7) 测量信息的效用问题：研究还发现学生在理解方法信息的用途方面存在明显不足，尤其是缺乏对战略管理的了解。这影响了他们对方法目的的理解，特别是将战略目标与指标对齐的能力。

24、针对提供软件产品和提供信息服务的这两类不同软件开发项目，讨论项目管理的特点。

软件产品开发的项目管理具有以下特点：

- 1、复杂性：软件项目管理具有高度的复杂性，涉及众多干系人、多变的需求、快速的技术更新和质量保障的挑战。项

目涉及开发团队内部的不同角色，如程序员、测试人员和架构师，他们各自有不同的专业技能和工作重点。客户需求可能频繁变动，技术更新迅速，质量保障也需要严格的测试流程。

2、迭代性：迭代性是软件项目管理的另一个重要特点，特别是敏捷开发方法的广泛应用。迭代性强调反馈和调整，有助于分散风险和快速响应市场变化。敏捷开发方法如 Scrum 和 Kanban 帮助团队快速适应需求变化，提高项目的灵活性和适应性。

3、依赖性：软件项目管理依赖于多种技术和工具，需要在不同的环境中运行。项目经理需要协调多个团队和部门，确保每个部分都能顺利进行。技术复杂性要求团队成员具备不同的技能和经验，项目经理需要提供必要的培训和资源支持。

4、协作性：软件项目通常需要跨职能团队的协作，包括开发、测试、运维等多个部门。高效的团队协作工具如项目管理软件可以帮助团队成员高效沟通和协作。

成本控制：项目管理还需要关注人力、硬件设备和软件工具的成本。合理的资源分配和成本控制策略对于项目的成功至关重要。

应对这些特点的策略和方法包括：

- (1) 采用敏捷开发方法：如 Scrum 和 Kanban，快速响应需求变化，提高项目的灵活性和适应性。
- (2) 使用项目管理软件：如 PingCode 和 Worktile，提供强大的团队协作功能，帮助团队成员高效沟通和协作。
- (3) 定期技术培训和学习：确保团队成员能够及时掌握新技术，保持项目的前沿性。
- (4) 严格的测试流程：包括单元测试、集成测试和系统测试，确保软件的质量和稳定性。

信息项目管理是指针对信息系统或信息技术相关项目的管理方法和技术。它与传统项目管理有许多相似之处，同时还具有一些独特的特点。

1. 复杂性：信息项目常常非常复杂，包括大量的技术、需求、资源和风险因素。信息系统往往与组织的各个部门和业务关联密切，涉及到大量的数据和流程。因此，信息项目管理需要对复杂性进行全面的分析和处理。

2. 技术驱动：信息项目通常是以信息技术为核心的，技术驱动是其独特的特点之一。在信息项目管理中，需要合理应用各种技术工具和方法，包括软件开发、数据库管理、网络技术等。同时，由于技术的快速更新和变化，信息项目管理需要具备灵活性和快速应对变化的能力。

3. 需求变化：信息项目中需求的变化是常态。由于信息系统常常涉及到大量的业务流程和数据，业务的变化会导致需求的变化。此外，用户需求也常常会在项目过程中被细化和调整。因此，信息项目管理需要具备需求变化管理的能力，包括对需求的及时响应和变更控制的技术手段。

4. 风险管理：信息项目常常伴随着一定的风险。信息系统的复杂性和技术驱动特点，使得项目面临着众多的风险因素，如技术风险、安全风险、成本风险等。信息项目管理需要建立完善的风险管理机制，对项目风险进行预测、评估和控制，同时制定相应的风险应对策略。

5. 沟通协作：信息项目管理需要广泛的沟通和协作。由于涉及到多个部门的合作，信息项目管理需要建立高效的沟通渠道，确保各方之间的需求和期望得到充分的明确和理解。此外，也需要建立项目团队间的良好协作机制，提高项目的整体效能。

为了应对以上特点，信息项目管理可以采用一些特定的方法和工具，如敏捷方法、原型开发、需求管理工具等。这些方法和工具能够提高信息项目的管理效率和质量，使项目达成预期目标。同时，信息项目管理也需要具备灵活性和创新性，及时应对项目中的变化和挑战。

25、论述软件过程改进对软件工程的意义和作用。

在当今信息技术快速发展的时代，软件已经成为各个行业和领域的重要组成部分。为了保证软件的质量和可靠性，软件工程过程的改进变得尤为重要。本文将从软件工程的定义、软件过程改进的意义、软件过程改进的方法和实施过程等方面进行分析和探讨。

一、软件工程的定义

软件工程被定义为一种通过科学和工程化的方法，以规范化的过程、原则和方法来开发和维护软件系统的学科。软件工程的目的是提高软件的开发效率、质量和可维护性，并确保软件满足用户需求。

二、软件过程

软件过程是指 在软件工具的支持下,所进行的一系列软件开发和进化的活动。

三、软件过程改进的意义

1. 提高软件质量：软件过程改进可以通过引入有效的软件质量管理方法和工具，提高软件开发过程中的质量控制和检

测，从而提高最终软件产品的质量。

2. 提高软件开发效率：通过软件过程改进，可以规范和优化软件开发过程中的各个环节，提高开发效率，减少开发周期，降低开发成本。

3. 降低风险：软件过程改进可以帮助识别和管理软件开发过程中的风险，减少项目失败的概率，保证软件项目的成功交付。

四、软件过程改进的方法

1. CMMI (Software Capability Maturity Model Integration) 模型：CMMI 是一种用于评估和改进软件开发过程的模型。它提供了一套评估标准和最佳实践，帮助组织识别并改进软件开发过程中的问题，提高组织的软件工程能力。

2. 敏捷开发方法：敏捷开发是一种迭代、增量、以人为中心的软件开发方法。它强调快速反馈、灵活性和高度合作的团队工作方式，通过多次迭代开发，逐步完善软件系统。

3. 六西格玛方法：六西格玛方法是一种通过测量和分析数据，减少过程中的变异性和缺陷数量的方法。它强调使用统计方法和数据驱动的决策，从而提高软件过程的稳定性和可预测性。

五、软件过程改进的实施过程

1. 确定改进目标：首先，需要明确软件过程改进的目标和期望效果，明确改进的方向和重点。

2. 进行现状评估：通过对当前软件开发过程的评估，分析现状，识别存在的问题和瓶颈，并确定改进的优先级和措施。

3. 设计改进计划：根据评估结果，制定详细的软件过程改进计划，包括改进的方法、步骤、时间计划和资源安排等。

4. 实施改进措施：根据改进计划，逐步实施改进措施，引入新的方法、工具和技术，优化软件开发过程中的各个环节。

5. 监控和评估：在改进过程中，需要不断监控和评估改进效果，及时调整和纠正，确保软件过程改进取得可持续的效果。

总结：

软件过程改进是提高软件开发过程质量和效率的重要手段，通过引入适当的改进方法和工具，可以帮助组织规范和优化软件开发过程，提高软件质量和开发效率，并降低风险。在实施软件过程改进时，需明确目标、评估现状、设计改进计划、实施改进措施，并持续监控和评估。只有不断改进和优化软件过程，才能适应快速变化的市场需求，保持竞争优势。

五、应用题 (20 题)

1. 假设一家工厂的采购部每天需要一张定货报表，报表按零件编号排序，表中列出所有需要再次定货的零件。对于每个需要再次定货的零件应该列出下述数据：零件编号，零件名称，定货数量，当前价格，主要供应者，次要供应者。零件入库或出库称为事务，通过放在仓库中的显示终端把事务报告给定货系统。当某种零件的库存数量少于库存量临界值时就应该再次定货。

要求：(1) 画出系统的数据流程图 (15 分)；

答：没有指明要哪一级别的数据流程图，那就按照题目描述中的颗粒度来画。

从问题描述中提取数据流图的四个成分：

数据的源点 / 终点：

订单报表就是数据

采购部每天需要一张订单报表，所以采购部采购员是数据终点

仓库中的 CRT 终端把事务报告给订货系统，所以仓库管理员是数据源点

数据处理

产生报表是一次处理

出入库是对库存数据进行处理，所以事务也是处理

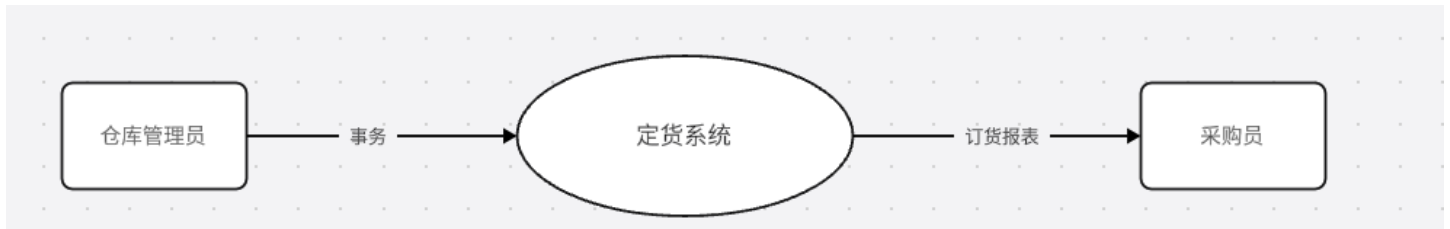
数据流和数据存储：

订单报表是数据流

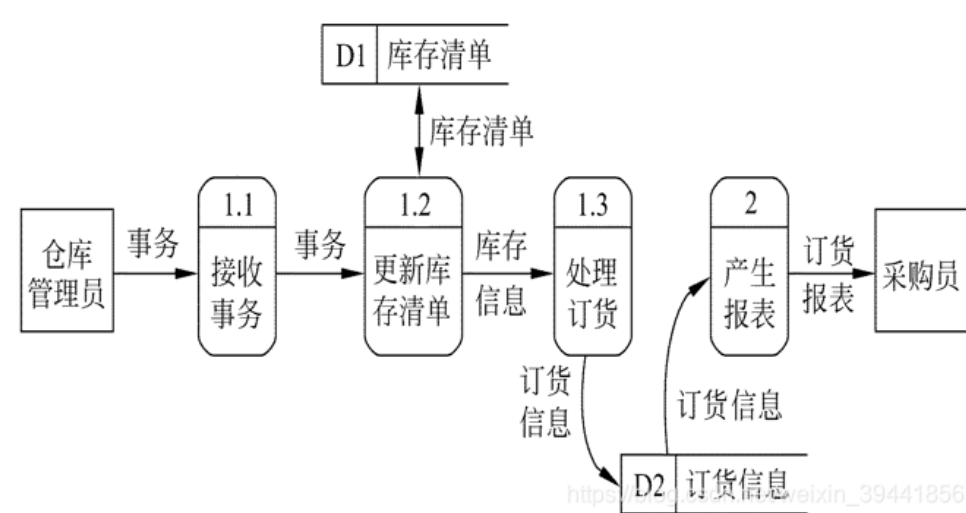
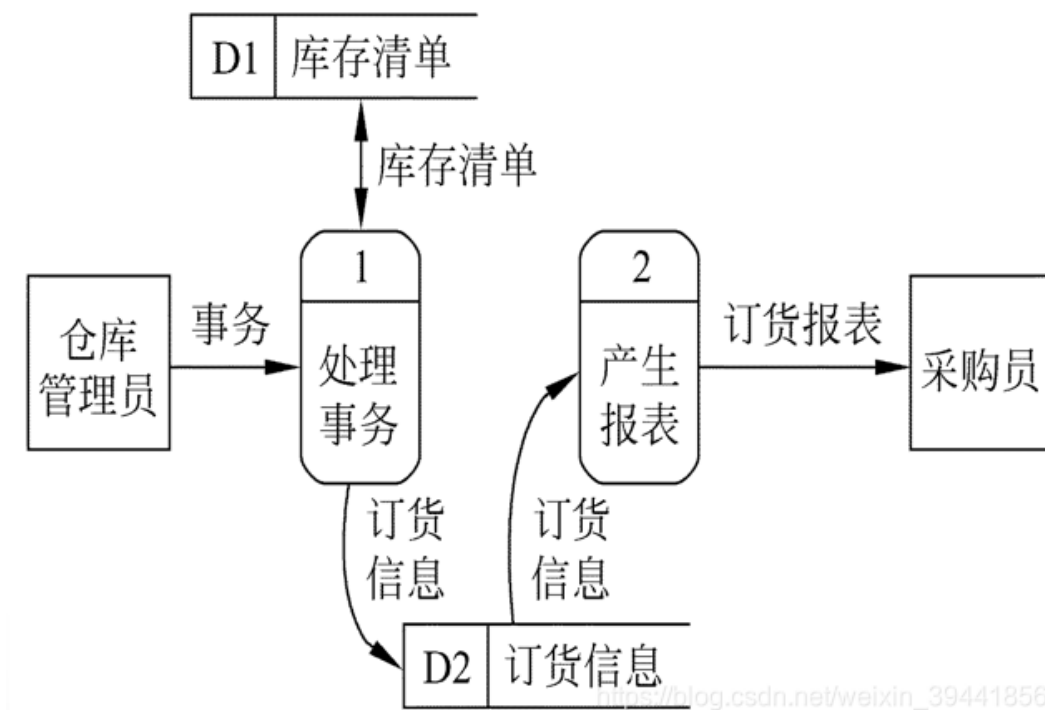
事务也是数据流

库存需要库存清单存储

订货需要订货清单存储



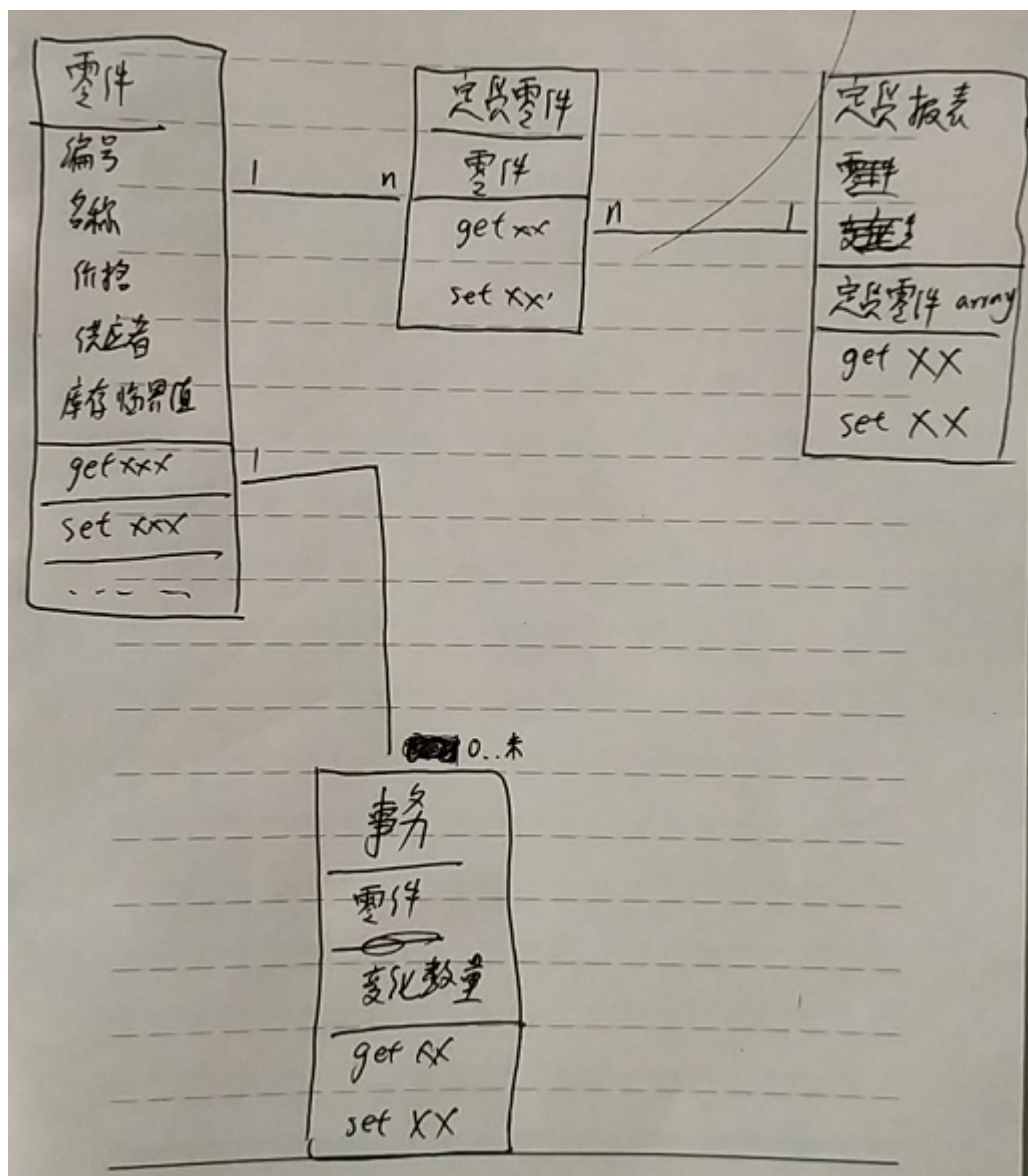
然后把中间的火锅展开，就形成了 0 层数据流图：



(2) 如果采用面向对象分析方法，请识别可能的类，画出类图，说明可能的属性和操作（15 分）。

答：

识别类的方法，就是寻找题目中的名词作为类，其动词作为方法，至于属性，则根据上下文自己判断：



注意，重数 n 代表数目不定，具体是 0..* 还是 1..* 呢，并不明确。

考试的时候，如果你能确定，那就写清楚了。如果不确定 0 行不行，那就写 n

其中每一个属性都有对应的 get 和 set 方法，就略去不写了，考试的时候要加上。

2. 现在有一个医院病房监护系统，用户提出的系统功能要求如下：

在医院病房监护系统中，病症监视器安置在每个病房，将病人的病症信号实时传送到中央监视系统进行分析处理。

在中心值班室里，值班护士使用中央监视系统对病员的情况进行监控，根据医生的要求随时打印病人的病情报告，系统会定期自动更新病历。

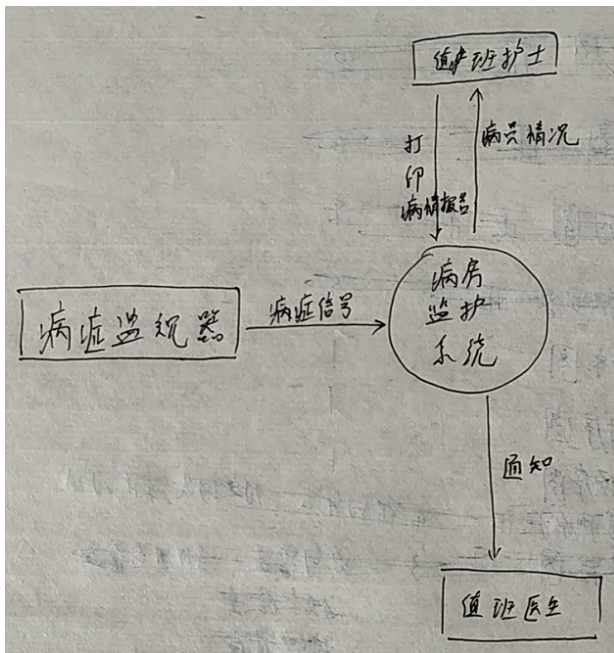
当病症出现异常时，系统会立即自动报警，通知值班医生及时进行处理，同时立即打印病人的病情报告和更新病历。

要求：

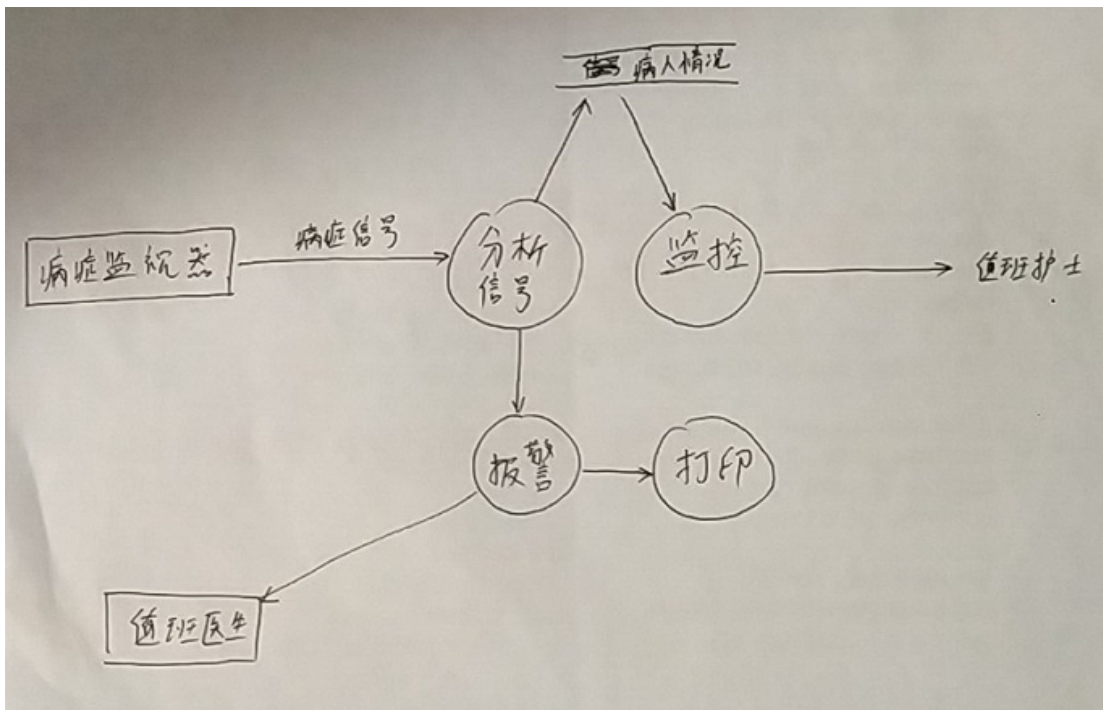
(1) 请画出数据流图（10 分），并采用 UML 建模语言画出用例图（10 分）。

答：

顶层数据流图(实际考试直接画 0 层就可以了)：

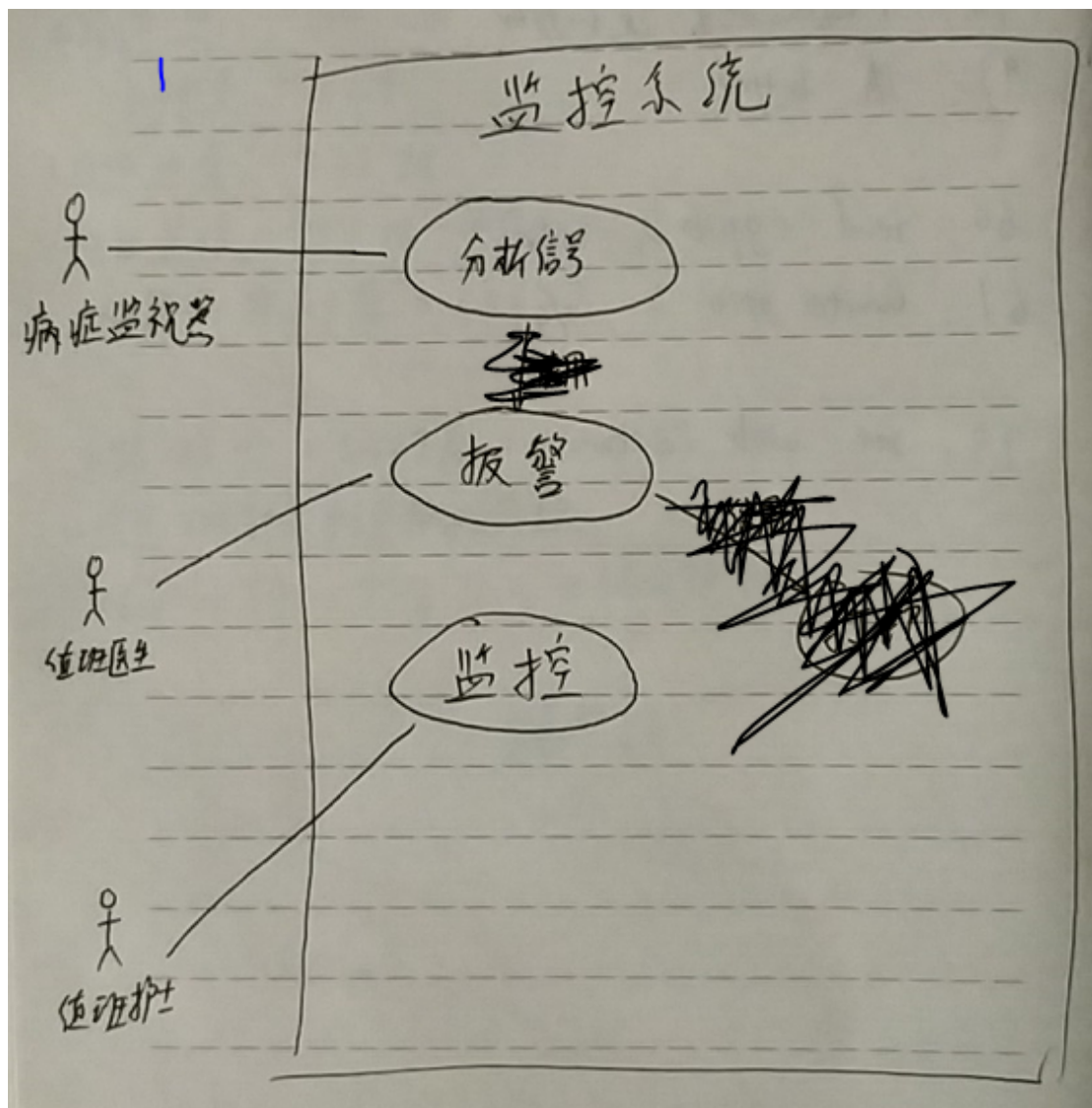


0 层数据流图:



用例图,(最简单情况下)包含三部分: 参与者, 用例, 关系.

参与者, 就是软件的使用者. 用例, 就是从使用者的角度, 软件能提供的-一个相对独立的小功能. 关系, 就是在参与者和用例之间画一条线, 表示两者有关系.



原来答案中用例和用例之间有使用关系，这是错的。
用例和用例之间的关系只能是无/扩展/包含/泛化

(2) 以医院病房监护系统为例，分析比较结构化开发方法与面向对象的开发方法的主要特点，结合该实例说明，为什么面向对象的开发方法更加优越 (10 分)?

答：面向对象的方法具有以下主要特点：

- ① 对软件开发过程所有阶段进行综合考虑。
- ② 软件生存期各阶段所使用的方法、技术具有高度的连续性，用符合人类认识世界的思维方式来分析、解决问题。

在医院病房监护系统中，一旦建立了系统的 USE/CASE 模型后，无论在需求分析、软件设计，甚至实现阶段，所建立的静态模型和动态模型都具有相同的描述方式，描述模型的一致性，不仅为开发者带来方便，也更直观的描述系统的特性。

而结构化方法，是以数据流、数据封闭性为基础，从对数据进行加工的角度进行需求分析，建立分层的 DFD 图 (如上图)。而在软件总体设计阶段，则用模块结构图来进行描述，详细设计阶段又是新的描述方式 (如 N-S 图、PAD 图等)，总之，由于不同阶段有不同的描述方式，需要进行多次转换，不仅为开发者带来诸多不便，也影响了对系统直观、准确的描述。

3. 连锁超市进货操作流程为：厂家持“定货单”及“厂家送货单”送货，库管科检查货物。

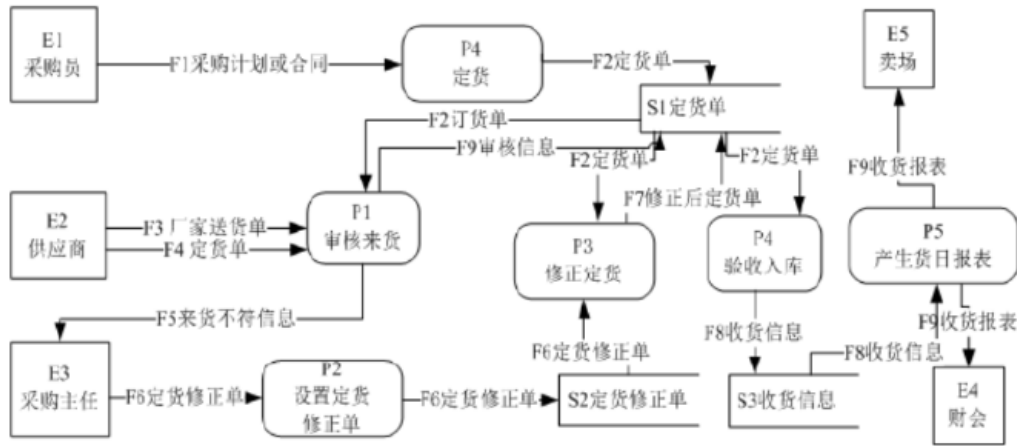
如果来货相符，库管科做收货确认，(包括：在“定货单”之“点核数量”栏手工填写收货数量；在电脑中找出“定货单”，在“点核数量”栏输入收货数量；在“收货日报表”上记录该收货；在“定货单”、“厂家送货单”上加盖库管科公章)，然后库管科打印商品条码，收送货组与卖场办理移交手续，卖场收货人在“定货单”上签名盖章，日末，库管可将单据汇总，连同“收货日报表”分别送卖场、财会，并自留一份，第二天电脑科打印“验收进货单”，分送卖场、采购、财会。

如果来货不符，通知负责该次定货之采购主任，采购主任填写“定货修正单”，报采购部主管签名同意，“定货修正单”交库管科，库管科从电脑中提出原“定货单”，按“定货修正单”内容进行修改，将修改后之新“定货单”打印（一式三份），和原“定货单”连在一起，最后进行上述来货相符后的工作。

要求：

- (1) 画出该系统的数据流图（15 分）；

答：



- (2) 给出该系统主要处理、数据存储或外部实体的数据字典（15 分）。

定货单=订货单编码+商品+下单数量+供货商+点核数量

验收进货单=商品+数量+供货商+订货单编码

商品=编码+名称+{供货商}

如果还有时间，就把其他的人工部分的也写上吧。多写的应该不扣分。

厂家送货单=送货单编码+订货单编码+商品+数量+供货商

收货日报表= 日期+商品+数量+收货单编码+收货人

定货修正单= 定货单编码+修改内容

4.设计一个简单的学生选课系统，系统能够注册符合要求的学生，学生注册以后就能选课。选课过程如下：

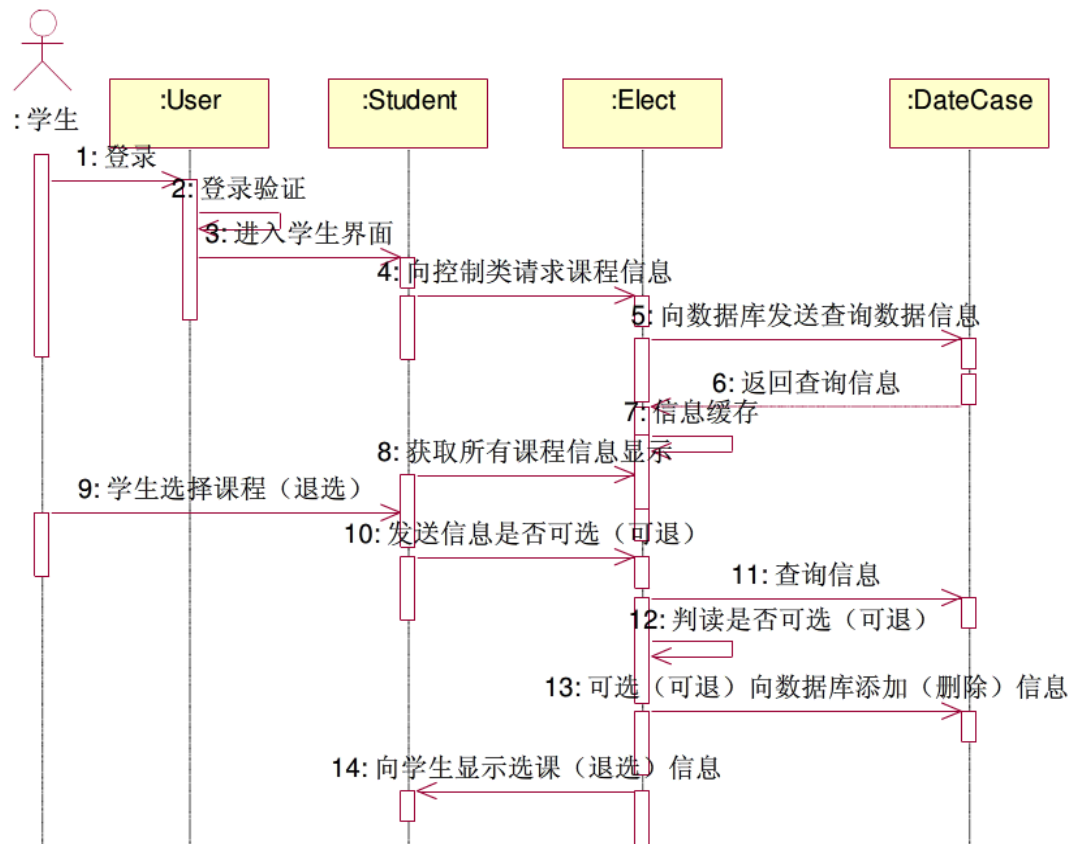
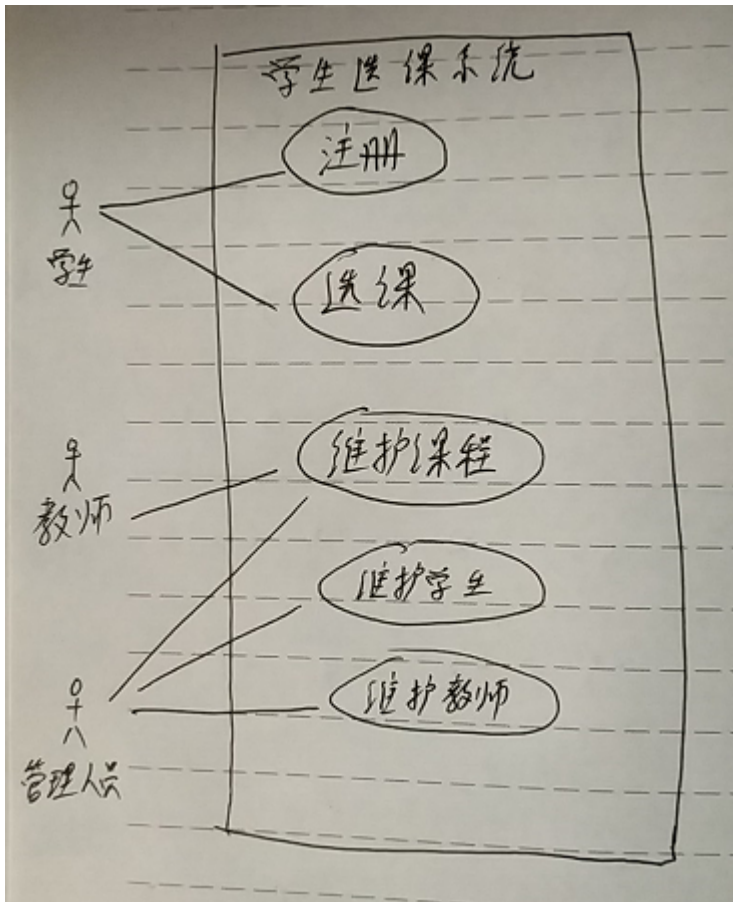
- (1) 浏览限选课名列表，然后根据课程的任课教师、时间、地点和人数限制决定是否选课成功；
- (2) 任课教师也可以根据情况，提供新的课程或修改过去课程的说明；
- (3) 教务管理人员统一管理注册的情况，可以删除、修改学生、教师和课程的权限。

要求：

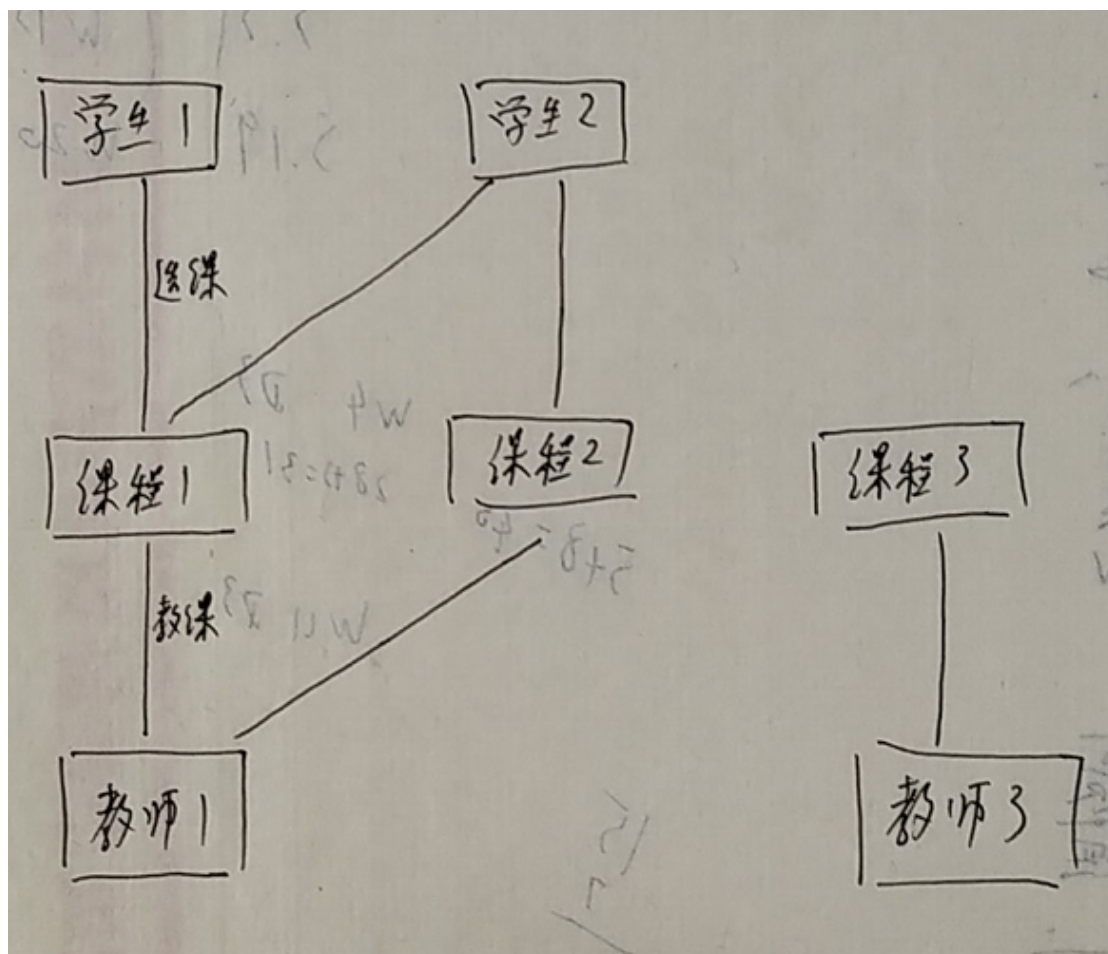
- (1) 请设计出功能模型，用用例（use-case）图表示（10 分）。
- (2) 请设计出对象模型，要求画出对象图，标出多重性、角色、限定词和关系（10 分）；
- (3) 请设计出动态模型，要求画册出学生成功注册后选课的时序图（10 分）；

答：

用例图：



对象图:



5.某铁路售票系统，其相关角色可简化为乘客、售票员和管理人员。业务过程如下：

- 售票窗口售票，乘客先指名车次和乘车时间，确认有余票后销售。
- 售票员根据乘客提供的信息查询余票，并卖出车票或取消本次购票，等待下一次的交易。
- 在购票期间，乘客可以按取消键取消本次购票。
- 管理人员可以取出钱币，并重置系统余额。
- 管理人员可以打印每天的交易账单。

其中部分数据结构的组成如下：

购票申请单 = 乘客身份证号 + 姓名 + 目的地 + 价格

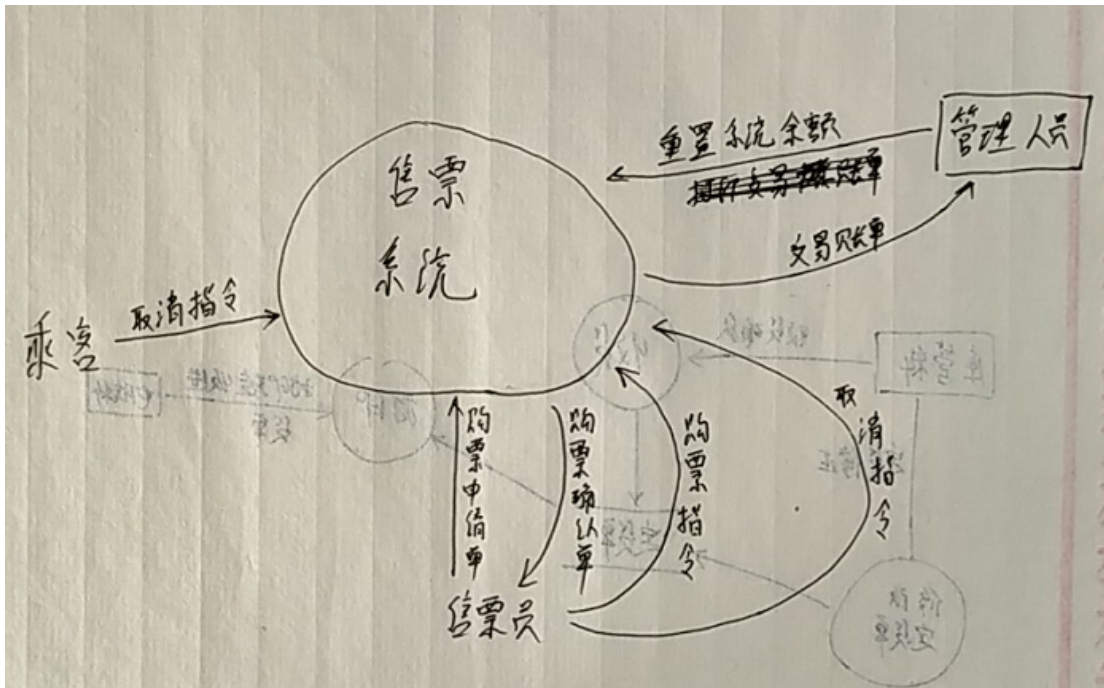
购票确认单 = 身份证号 + 姓名 + 目的地 + 是否有余票

每日账单 = 当日初余额 + 当日末余额 + 日期 + 售票张数

要求：

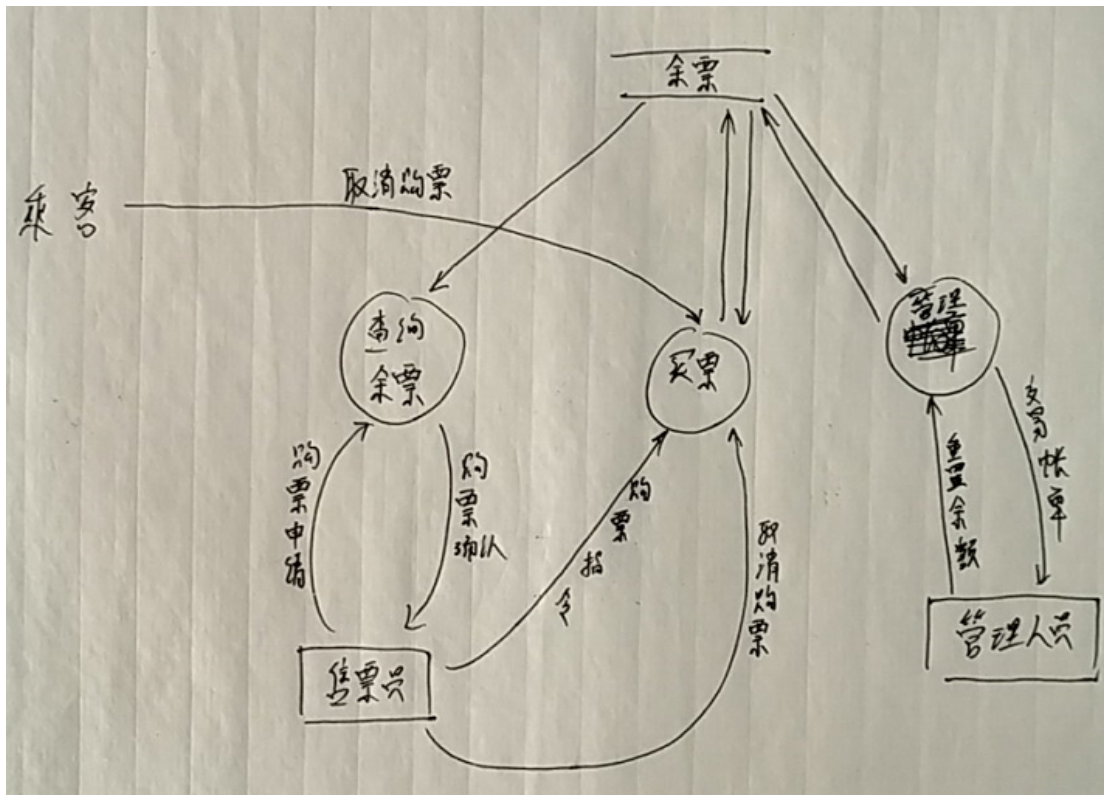
(1) 请画出分层数据流图（10分），并写出第一层数据流图的数据字典（5分）。

答：顶层数据流图：

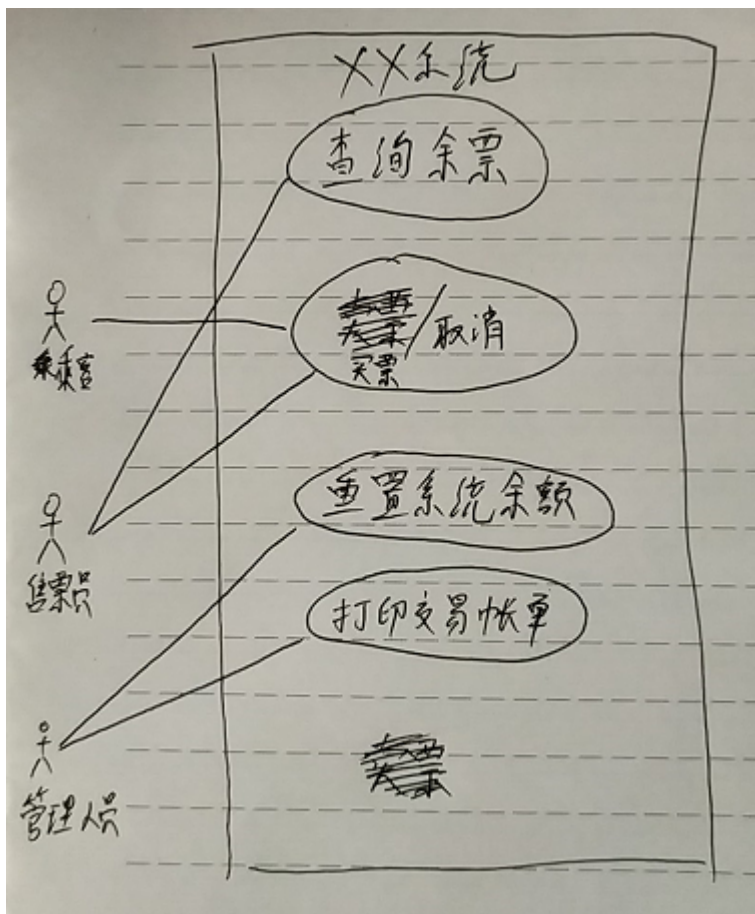


注意，其实交易账单没必要发送给管理人员，而是管理人员发送一个[打印交易账单]的指令给系统。但是题目既然提到了[每日账单]的数据结构，那就只好配合一下。
[取消指令]也没必要加，但是提到了，那就加上。

0 层数据流图:



(2) 请采用 UML 的用例图描述系统功能需求，并识别出可能的类 (15 分)。



从用例图识别类，那就是把所有的参与者分别当作一个类，把每个参与者关联的用例作为那个类的方法：

乘客类：方法有买票，取消买票

售票员类：方法有买票，取消买票，查询余票

管理人员类：方法有重置系统余额，打印交易账单

6. 某个银行的存取款业务处理系统有以下功能：

- 其相关角色可简化为储户和银行工作人员；
- 储户将填好的存 / 取款单和存折交给银行工作人员输入系统；
- 系统要求进行以下处理：
 - (1) 业务分类处理：系统审查存 / 取款单，不合格则退回；合格则确定本次业务的性质(存款或取款)；
 - (2) 存款处理：系统将存款单上的存款金额分别记录在存折和账目文件中；记录现金账；打印存款通知单给储户；最后将存折还给储户；
 - (3) 取款处理：系统将取款单上的取款金额记录在账目文件和存折中并修改现金账。

要求：

- (1) 请画出分层数据流图 (10 分)，并写出第一层数据流图的数据字典 (5 分)。
- (2) 请采用 UML 的用例图(use case diagram)描述系统功能需求，并识别出可能的类 (15 分)。

答：

(1) 数据字典：

存款单=流水号+储户+金额

取款单=流水号+储户+金额

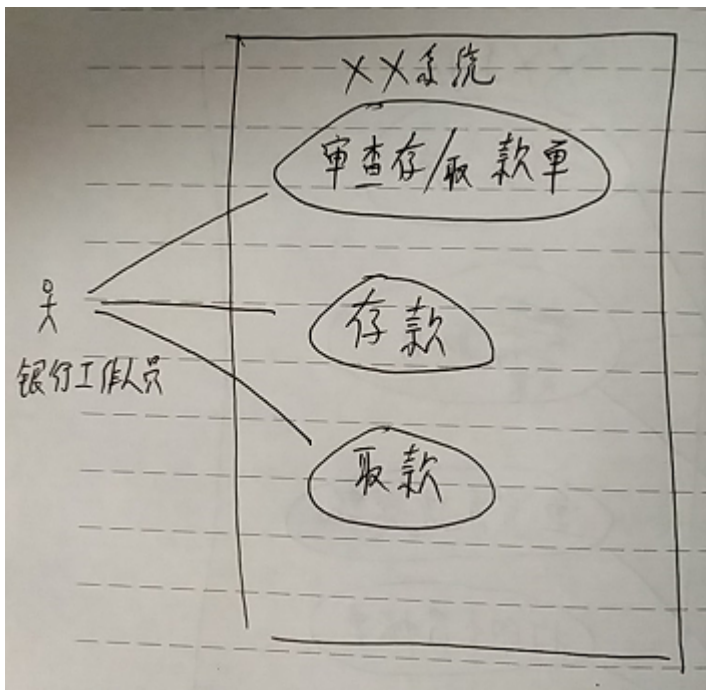
储户=身份证号码+姓名+账号

账目文件=时间+储户+金额+操作类型

操作类型=存款|取款

存款通知单=时间+流水号+存款单流水号+储户+金额

(2)用例图：



根据用例图，识别出来的类有银行工作人员，其方法有审查存/取款单,存款,取款
另外，存款单和取款单也可以认为是类

7.某公司准备开发公共汽车刷卡计费系统，具体需求如下：

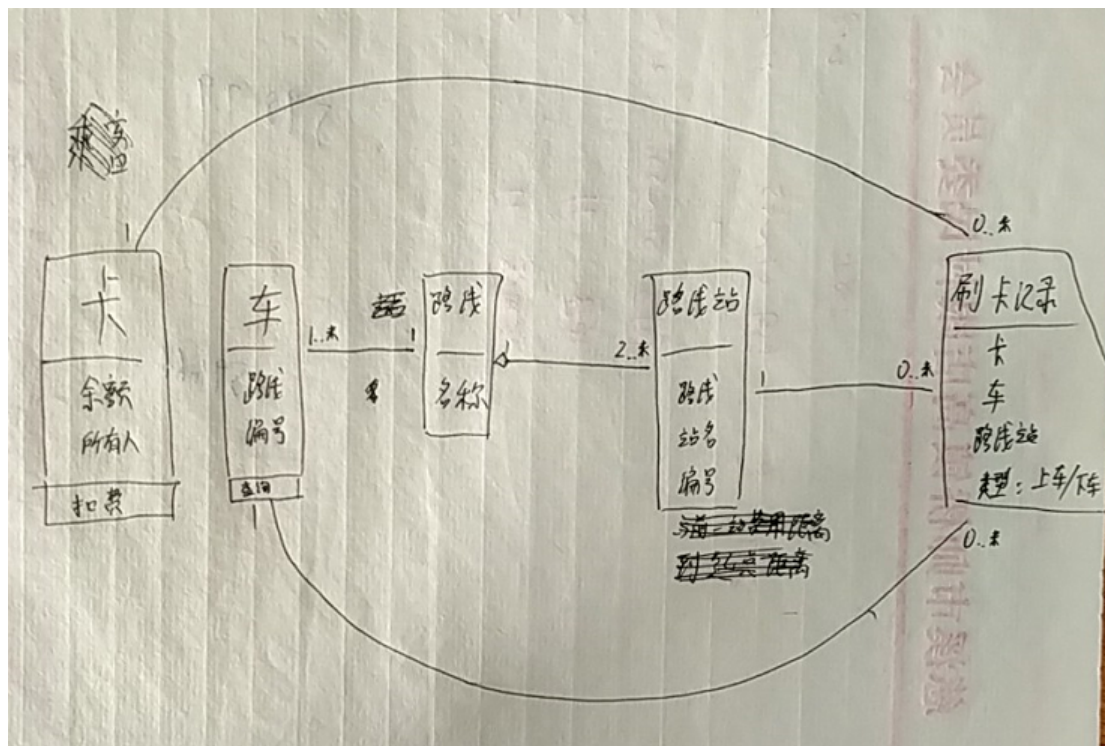
- (1) 司机在每到达一站时，点击到站按钮，通知系统已到达一站；
- (2) 乘客上下车时都要刷卡。上车刷卡通知系统该乘客已上车。在下车刷卡时，系统根据该乘客乘坐的站数，计算票价，并从卡中的余额中扣除。
- (3) 乘务员可查询某乘客的卡是否在上车时刷过，也可查询每个班次的运营情况（如，载客量和运营额）。
- (4) 管理员可以通过系统调节费率，记费方式等。

要求：

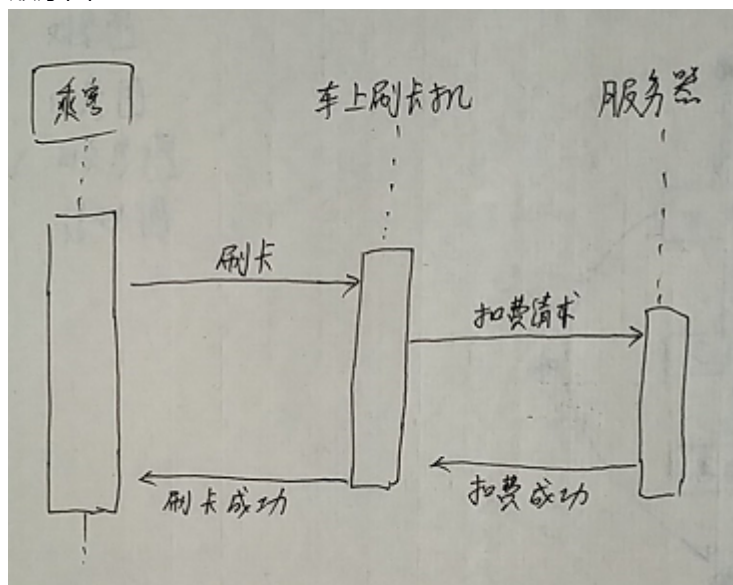
- (1) 请根据以上对该系统主要需求的描述，并尽可能地考虑一些特殊的情况，进行需求分析，建立该系统完整的用例图，并附用例的文本说明（10分）。
- (2) 请建立该系统的 OOA 类图（10分）。
- (3) 用顺序图描述乘客下车刷卡时，系统中有关的对象的交互序列（10分）。

答：

类图：



顺序图:



8.现在有一系统正在进行集成测试，模块 A 提供的文件接口能够读取 txt 格式的文本文件，不论该文件是否被其它进程使用。请补充完成如下测试用例中的 (1)、(2)、(3) 和 (4) 部分。

测试用途编号	FILE01_READFILE
测试项目	测试模块 A 提供的文件接口
测试标题	文件 B 正在被其他进程执行读/写操作，通过 A 模块的文件接口读取文件 B 中的数据
重要级别	高
预置条件	(1) (5分)
输入	(2) (5分)
操作步骤	(3) (15分)
预期输出	(4) (5分)

答:

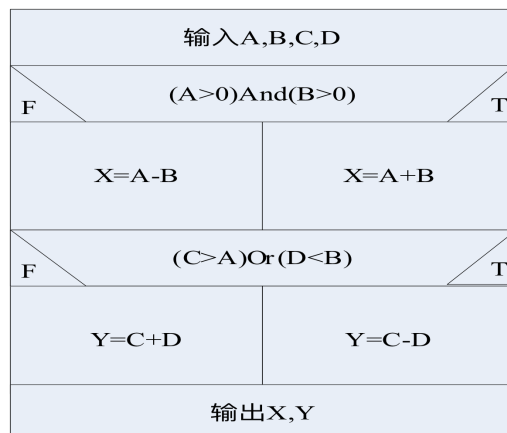
预置条件:无

输入:txt 格式的文本文件的路径

操作步骤:调用模块 A 提供的文件接口

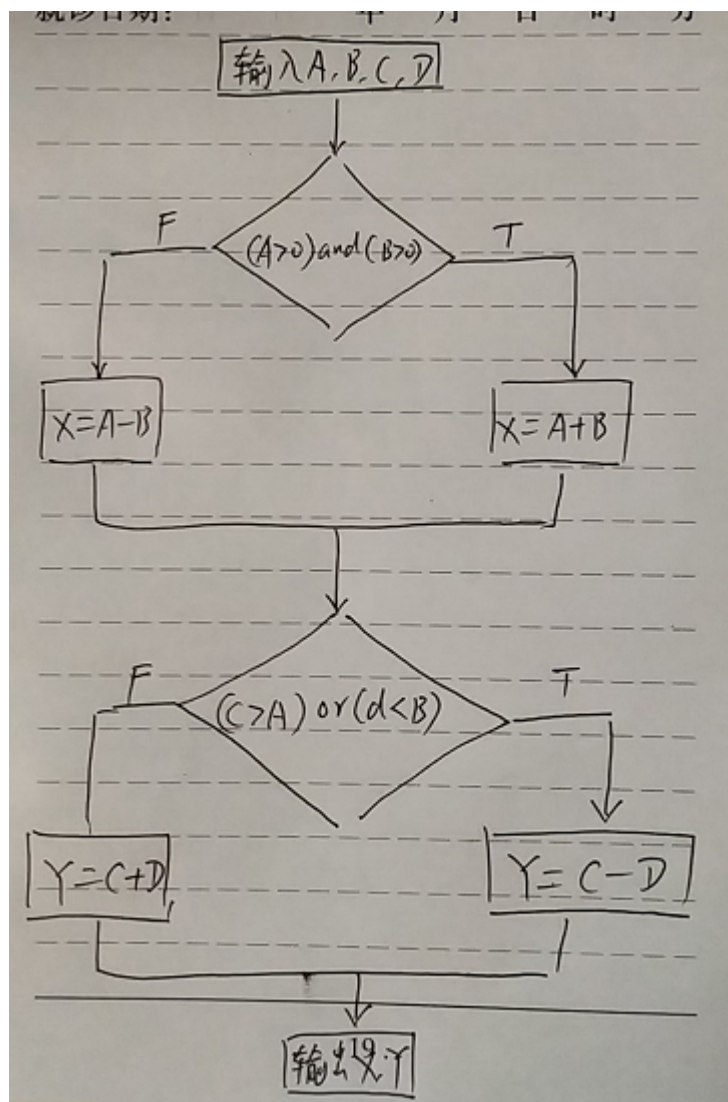
预期输出:能够正确读取, 不管该文件是否被其他进程使用

9.依据下面给出的 N—S 图画出其流程图, 在此基础上设计最少测试用例, 实现语句覆盖和条件组合覆盖。



答:

流程图:



实现语句覆盖的测试用例:

所谓语句覆盖, 就是每个语句都被执行到. 在流程图上, 就是每条线都要走过.

那么最简单的办法, 就是设计两个测试用例, 他们在每个判断上的结果是一真一假. 这两个测试用例合起来, 就是把每条线都走了.

令 A=1, B=1, 那么第一个判断就是 T. 令 C=2, D=2 则第二个判断是 T
 令 A=-1, B=-1, 那么第一个判断是 F. 令 C=-2, D=0, 则第二个判断是 F
 这两个测试用例就视线了语句覆盖

实现条件组合覆盖的测试用例:

所谓条件组合覆盖, 就是对于流程中的每一个判断, 要把所有的条件组合都走一次.

比如第一个判断 (A>0) and (B>0), 包含了两个子判断(A>0)和(B>0). 两个子判断都可真可假, 合起来就是 4 种可能性. 把这 4 种可能性都覆盖了, 就实现了条件组合覆盖.

第二个判断, 也有 4 种可能性.

那么最理想的情况, 构造 4 个测试用例, 每个测试用例, 分别覆盖了两个判断种的一个条件组合, 那么合起来, 就把所有的条件组合覆盖到了.

	A>0	B>0	C>A	D<B
A=1 B=1 C=2 D=0	T	T	T	T
A=1 B=-1 C=2 D=-2	T	F	T	F
A=-1 B=1 C=-2 D=0	F	T	F	T
A=-1 B=-1 C=-2 D=0	F	F	F	F

10.有一款 8 位微机, 其十六进制常数定义为: 以 0x 或 0X 开头的数是十六进制整数, 其值的范围是-7f~7f (表示十六进制的大小写字母不加区别), 如 0X13, 0X6A, -0X3c. 根据上述条件使用等价划分法设计测试用例。

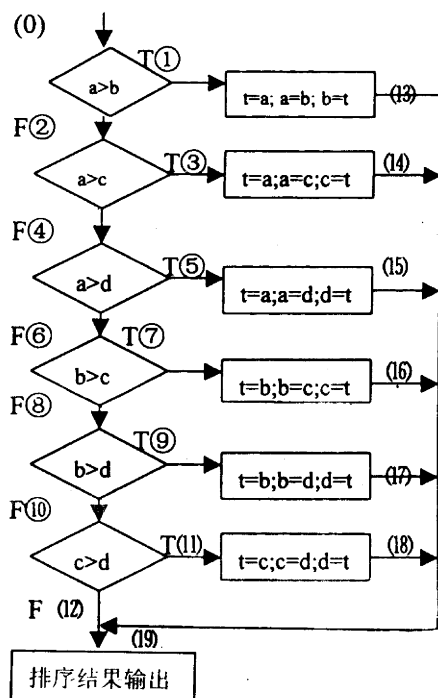
答: 有效范围是-7F~7F, 那么等价类有三个, 一个是有效范围内的, 比如 0x7F.
 一个是有效范围左边的, 比如-0x80, 一个是有效范围右边的, 比如是 0x80

11.下面是某程序的流程图

(1) 计算它的环路复杂性 (10 分)。

环路复杂性数量=判断节点数+1 = 6+1 = 7

(2) 为了完成基本路径测试, 求它的一组独立的路径 (20 分)。



答:

所谓基本路径测试, 就是实现语句覆盖(概念不同, 形式一样, 考试的时候不要这么说. 可以这么理解). 就是让流程图里面每一个地方都能走到.

实现了基本路径测试的一组路线, 成为一组独立路径.

了解了概念以后, 题目就很好做了:

如果流程图里有 N 个 T/F 判断, 那么就需要 $N+1$ 个独立路径. 这个可以用来验证是否缺了或者多了一个独立路径.

独立路径: 至少沿一条新的边移动的路径

路径 1: 0->1->13->19

路径 2: 0->2->3->14->19

路径 3: 0->2->4->5->15->19

路径 4: 0->2->4->->6->7->16->19

路径 5: 0->2->4->->6->8->9->17->19

路径 6: 0->2->4->->6->8->10->11->18->19

路径 7: 0->2->4->->6->8->10->12

12.某网站开发新的邮件系统供用户使用, 现需要做系统测试。请以“输入正确的用户邮件地址和错误的密码”为例, 完成如下的测试用例中 (1)、(2)、(3) 和 (4) 部分。

测试用例:

- 测试目标: (1) (5分);
- 测试环境: Windows XP 操作系统和浏览器 IE8;
- 输入数据: (2) (5分);
- 步骤: (3) (15分);
- 期望结果 (4) (5分)。

答:

- 测试目标: (检验客户输入正确邮箱地址和错误密码时候会出现的现象) (5分);
- 测试环境: Windows XP 操作系统和浏览器 IE8;
- 输入数据: (用户名, 密码) (5分);
- 步骤: (①输入正确用户邮件地址②输入错误密码③提交观察结果) (15分);
- 期望结果 (系统反馈输入密码错误提示, 不会进入邮箱) (5分)。

13.下列是一段伪码程序:

```

START
INPUT (A, B, C)
IF A>5
    THEN X=10
    ELSE X=1
ENDIF
IF B>10
    THEN Y=20
    ELSE Y=2
ENDIF
IF C<15
    THEN Z=30
    ELSE Z=3
ENDIF
PRINT(X,Y,Z)

```

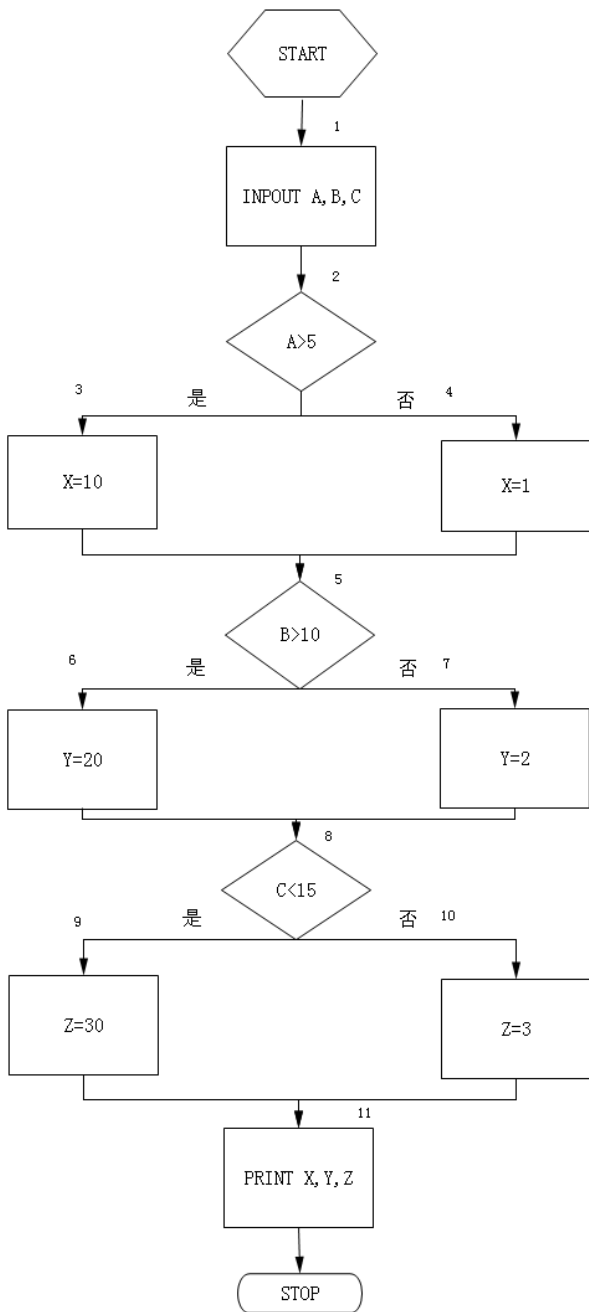
STOP

问题：(1) 画出控制流程图（10 分）

(2) 采用基本路径测试，给出独立的路径（10 分）

(3) 写出路径覆盖用例（10 分）

答：(1) 控制流程图：



(3) 独立路径

所谓一套独立路径，就是他们实现了语句覆盖。路径的数量的上限，是环路复杂度，也就是判断节点数量 +1。

在这个例子中，上限没有达到，只需要两个用例就能实现语句覆盖。

1,2,3,5,6,8,9,11

1,2,4,5,7,8,10,11

(3) 路径覆盖用例

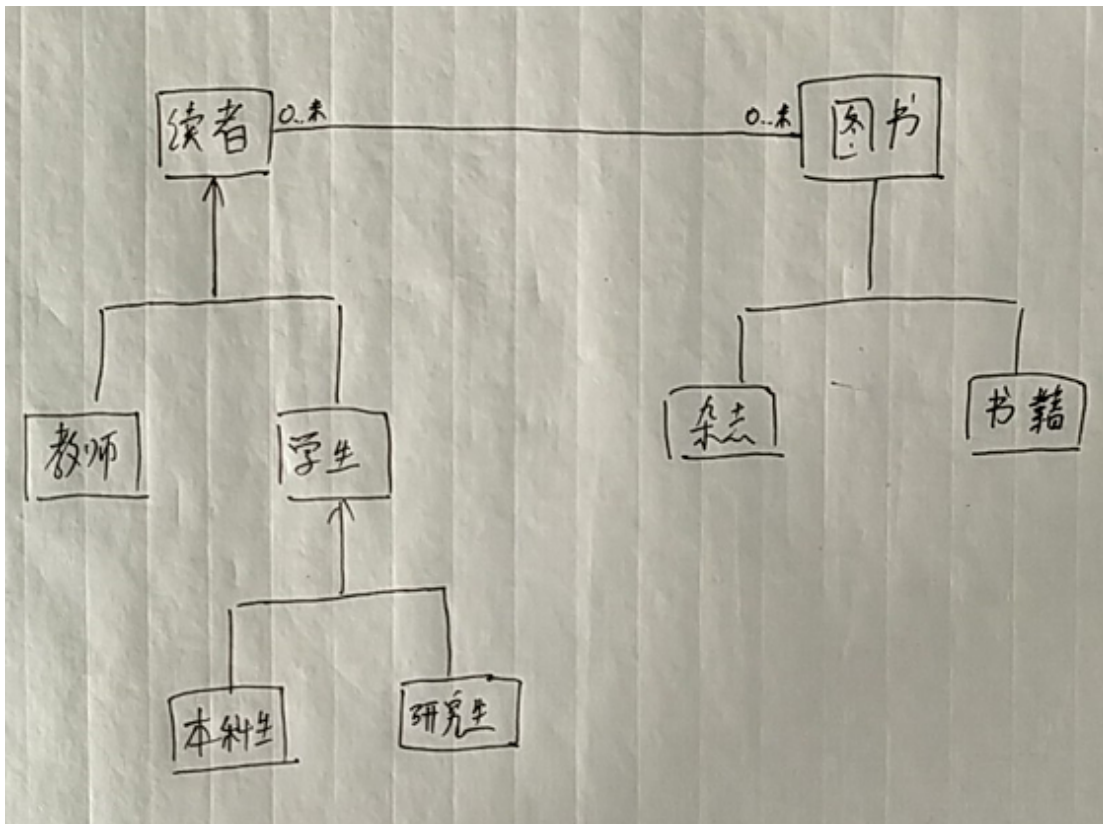
略。

总的来说，就是 3 个判断分别可能是 T 或者 F。组合起来就是 8 种可能。

把 8 种可能都来一遍，就覆盖了。也就是要构造 8 个测试用例，分别对应这 8 种可能。

14.图书馆管理系统中，读者包括教师、学生（学生中有本科生、研究生、）等等，图书包括杂志、各类书籍，请用 UML 建立 UseCase 图（5 分），读者和图书的类图（10 分），图书的状态图（5 分），以及借还书过程的活动图（10 分）。

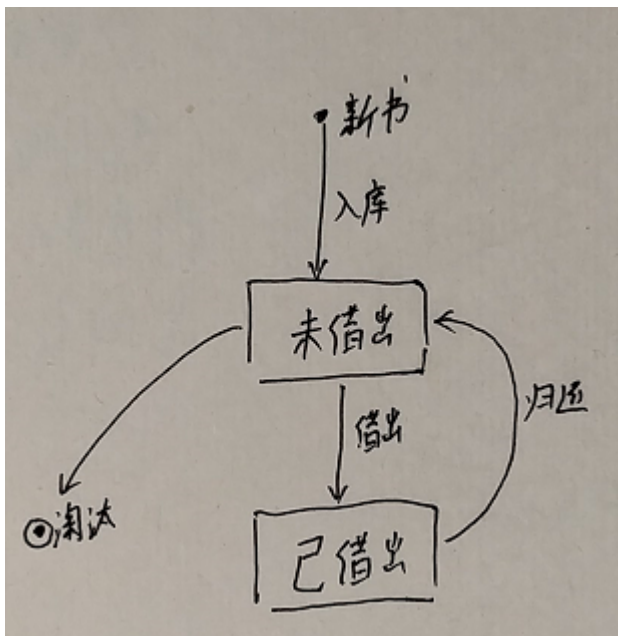
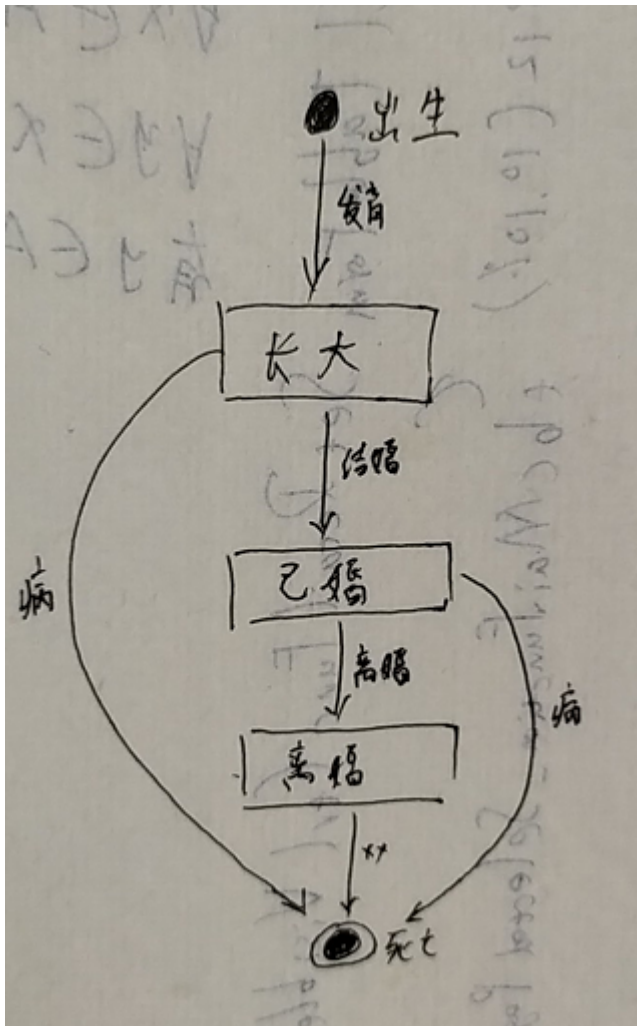
答：类图：



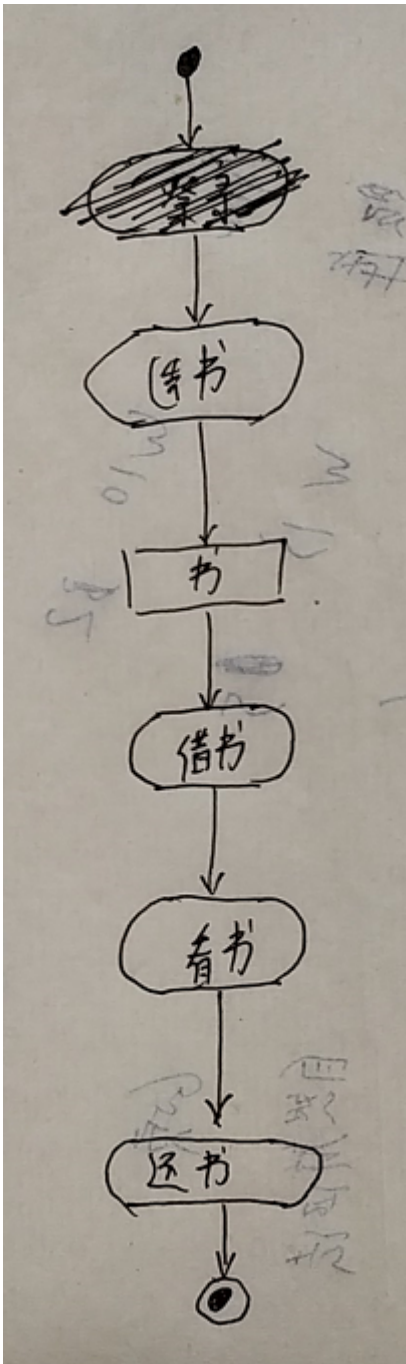
注意，只写了类名，属性和方法省略了，考试的时候随便加几个吧

状态图：

所谓状态图，就是把一个东西的各种状态列出来，然后画线和文字表示各种状态之间的跳转。还有个开始状态(实心圆圈表示)和结束状态(实心圆圈+外面一个圆表示)。比如人的状态图：



活动图:

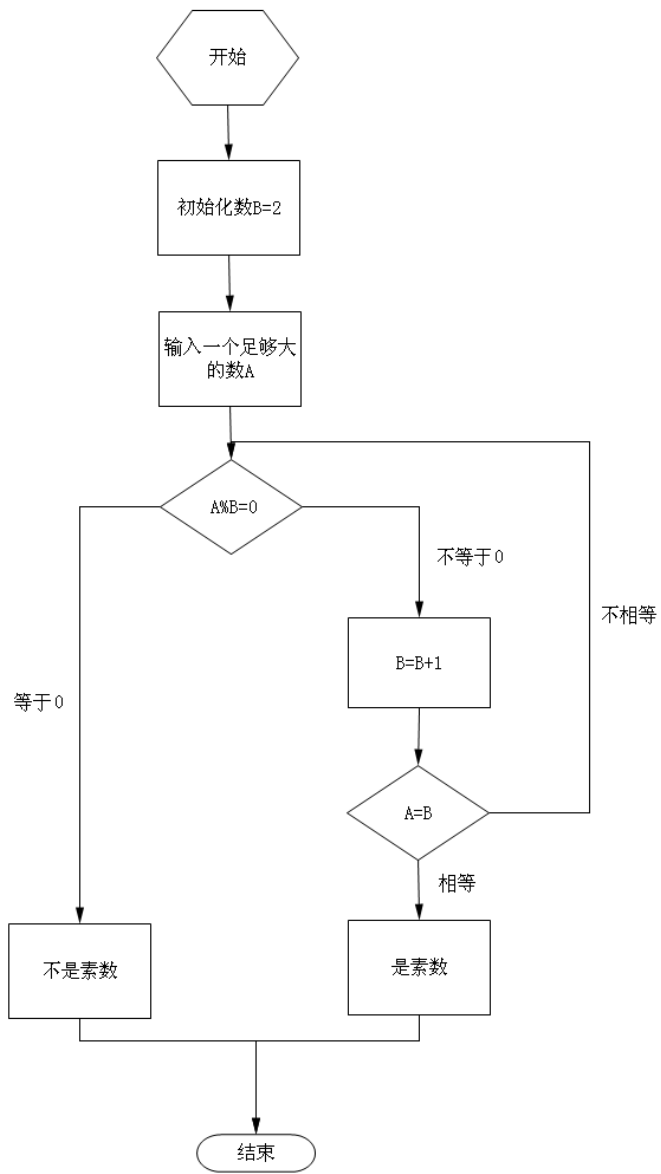


15.由键盘输入一个足够大的数，判断该数是否为素数（除一和本身外不能被其他数整除）。

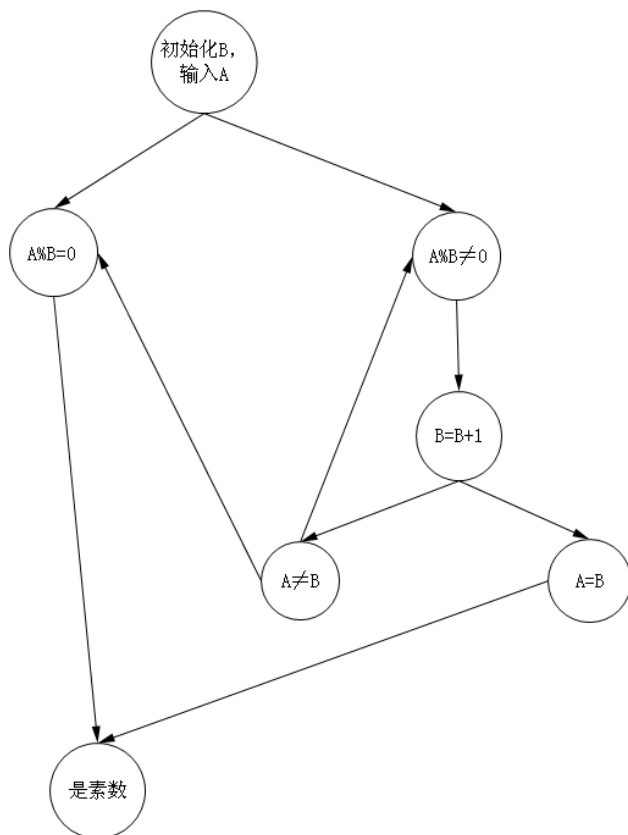
要求：

- (1) 画出实现该算法的程序流程图，画出控制流图（5分）；
- (2) 写出判定测试和条件测试的测试用例（5分）；
- (3) 如果采用基本路径测试法，写出测试用例（10分）；
- (4) 如果对循环进行测试，应考虑那些测试点，写出测试用例（10分）。

答：(1) 流程图



控制流图：



(2) 判定测试用例:

判定表:

	1	2	3
条件	能否被 1 整除?	Y	Y
	能否被本身整除?	Y	Y
	能否被除了 1 和本身外的数整除?	Y	N
动作	是素数	✗	✓
	不是素数	✓	✗

测试用例:

A1=100, 能被 1 整除, 能被 100 整除, 能被 2、4、5.....整除, 不是素数

A2=101, 能被 1 整除, 能被 101 整除, 不能被其他数整除, 是素数

条件测试用例: A=200, 能被 1 整除, 能被 200 整除, 能被 2、4、5...整除, 不是素数

(3) 基本路径测试用例:

①输入 A=10001, $10001 \% 2 \neq 0$, $B=B+1=3$, $10001 \% 3 \neq 0$, $B=B+1=4$, $10001 \% 10001 = 0$, $A=B$, 结论: A 是素数

②输入 A=2, $2 \% 2 = 0$, 结论: A 不是素数

③输入 A=15, $15 \% 2 \neq 0$, $B=B+1=3$, $15 \% 3 = 0$, $A \neq B$, 结论: A 不是素数

(4) 循环测试:

应该考虑①B 为初始值, A=B 的点②B=B+1, A=B 的点③B=B +1,A≠B 的点

16.某考务处理系统具有以下功能:

输入报名单; 自动编制准考证号; 输出准考证; 输入成绩清单; 输出成绩通知单; 输出成绩分布表; 输入合格标准、输出录取通知单; 试题难度分析,并输出试题难度分析表。

其中部分数据流的组成如下所示:

报名单 = 报名号 + 姓名 + 通信地址

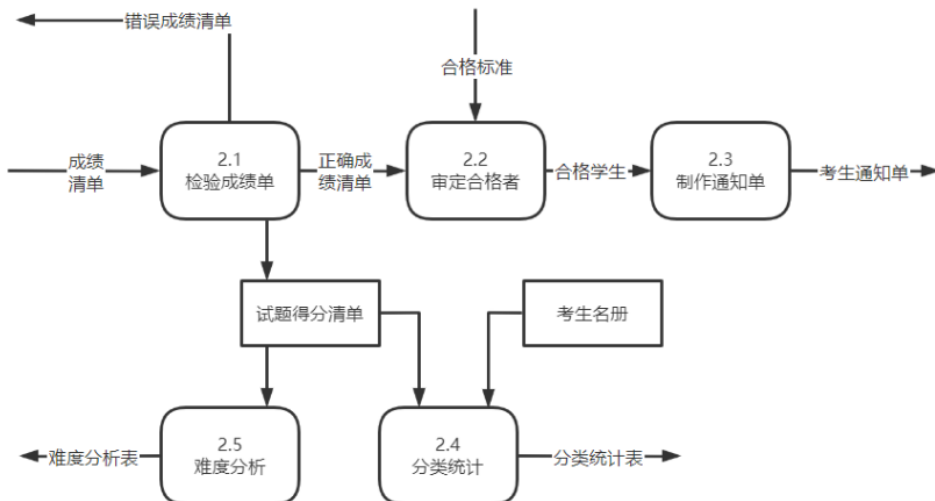
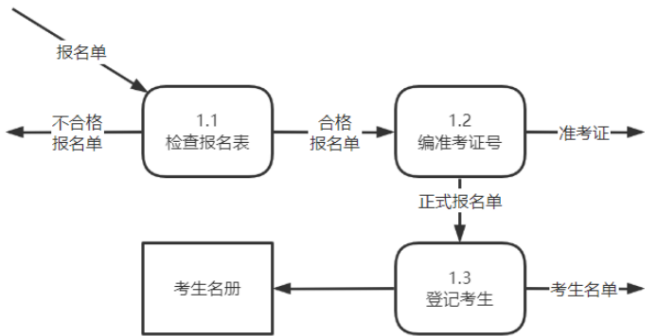
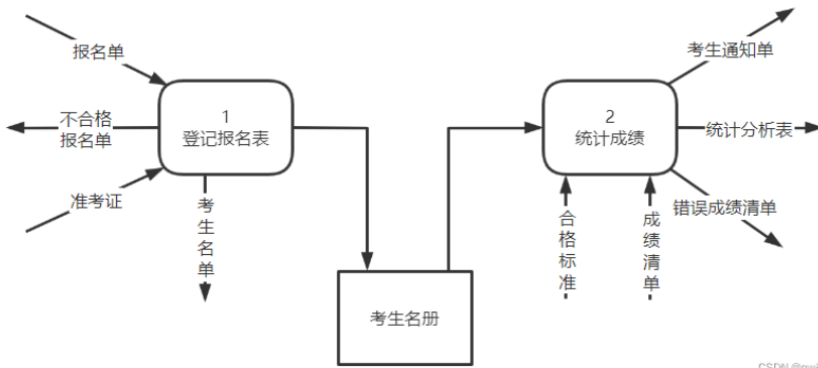
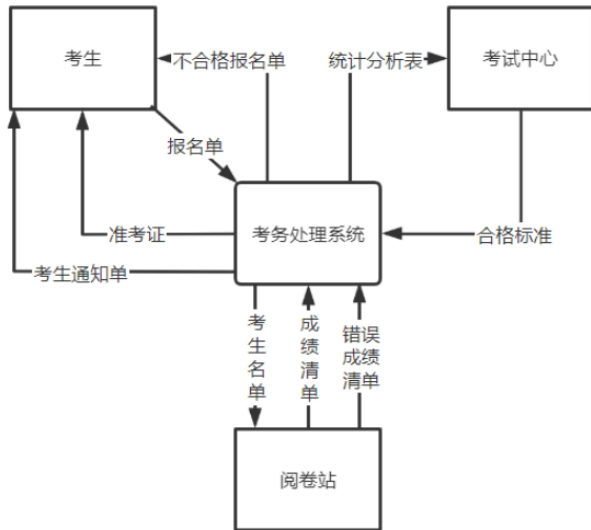
考生名册 = 报名号 + 准考证号 + 姓名 + 通信地址

成绩册 = 准考证号 + { 课程号 + 成绩 } (其中{W}表示 W 重复多次)

准考证 = 报名号 + 姓名 + 准考证号

要求:

(1) 画出顶层数据流图和底层数据流图 (20 分);



(2) 画出 USE/CASE 图 (10 分)。

17.某房屋租赁服务系统主要功能如下：系统对出租的房屋信息、房主信息、需要租房的人员信息，以及租房人和房主的会面情况进行管理和维护。

房主信息包括：房主编码、姓名、地址、电话号码和密码；

房屋信息包括：房主编码、房屋地址、类型（单间/套间）、适合住宿的人数、房租、是否可以出租。

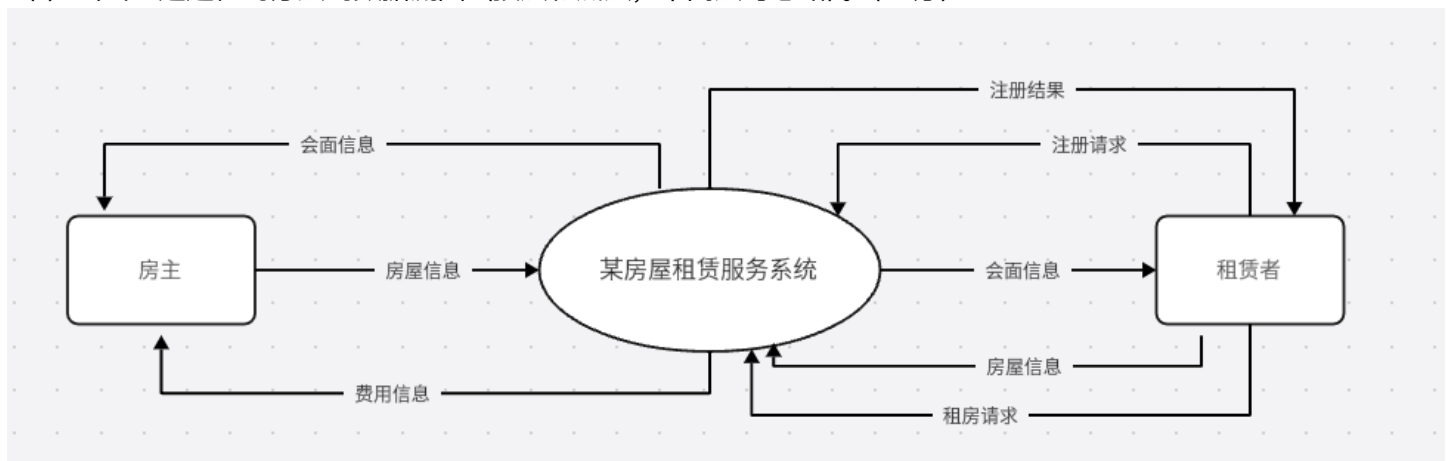
租房人信息包括：编码、姓名、现住址、电话号码、出生日期、性别、密码。

每当房屋信息发生变化时，房主可以通过系统维护信息，但需交纳一定的费用，由系统自动给出费用信息。

租房人可通过系统查询现有的可租用的房屋，但必须先系统中注册。若租房人希望租用某房屋，则需要发出租房请求，请求中包含房屋的详细信息，系统将安排租房人与房主会面的时间和地点，并将会面信息通知租房人和房主，会面信息包括会面时间、地点以及会面双方的基本信息，系统将记录会面信息。

要求：

(1) 画出上述过程的分层的数据流图（顶层和底层，中间层可忽略）。(15 分)

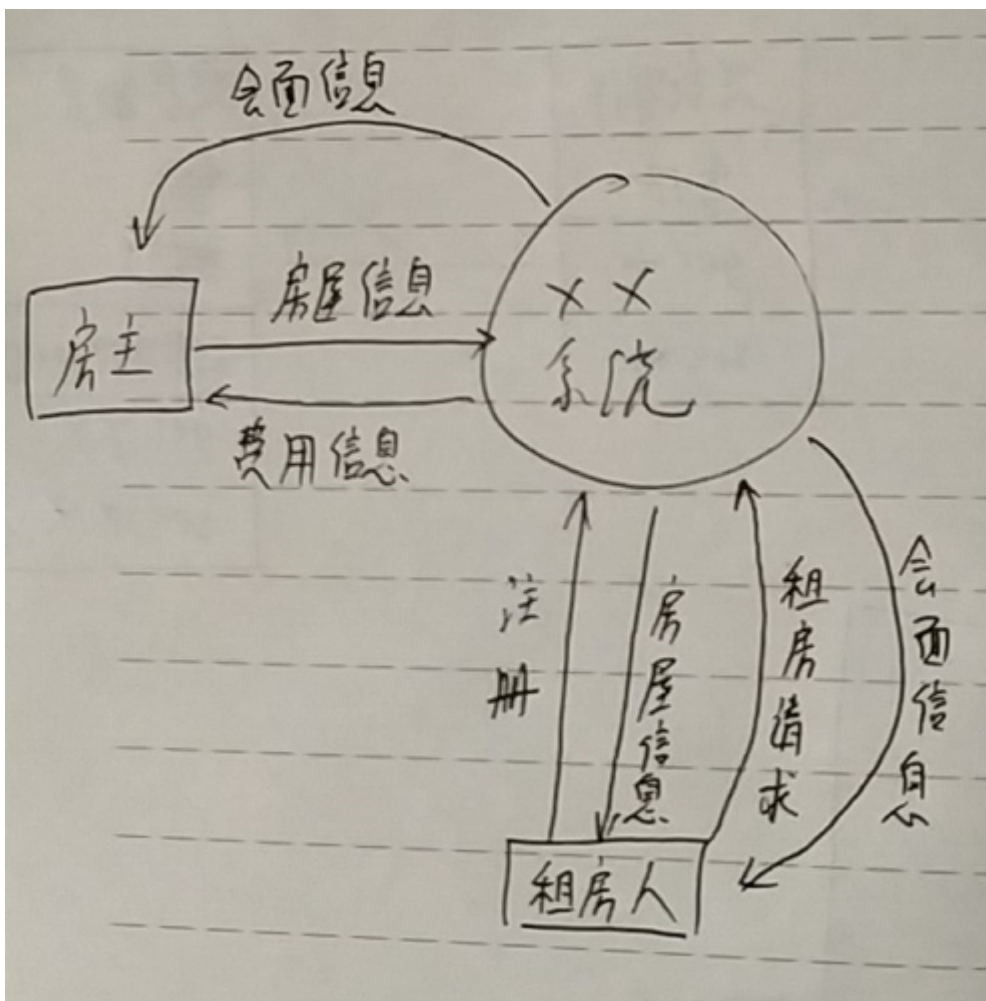


(2) 给出顶层数据流图的数据字典。(5 分)

(3) 应用 Use/Case 图表达系统的交互。(10 分)

答：

(1) 顶层数据流图：



还缺了一个注册结果, 从 XX 系统发送到租房人

(2)

房屋信息=房主编码+房屋地址+类型+适合住宿的人数+房租+是否可以出租

费用信息=时间+房主编码+房屋地址+金额

注册请求=姓名+现住址+电话号码+出生日期+性别+密码

注册结果=租房人编码(假设永远成功)

租房请求=租房人编码+房主编码+房屋地址+类型+适合住宿的人数+房租+是否可以出租

会面信息=时间+地点+房主编码+租房人编码

18.某医院病人住院与治疗结帐业务处理过程及说明如下:

(1) 病人经医生建议提出申请住院治疗, 选择住院病房标准, 经核查病房档案文件, 有相应床位时可办理住院。

(2) 办理时由病人填写住院登记卡, 内容包括: 病历号, 姓名, 病床号, 支付方式和其他内容。支付方式可选择现金支付或银行转帐两种, 但每个病人只能选择一种方式。病床租出后在病床档案中作“使用”标志。

(3) 病人在住院期间除治疗外医院还提供其他收费服务, 如用餐、保健等。所有费用按病床号记入计费文件中, 病人出院时一起结算。

(4) 病人出院时, 医院根据计费文件和住院登记卡的支付方式办理结帐手续。并修改病床档案文件中的标志。

主要文件和档案格式如下:

病床档案: 病床号, 病床标准, 日租金, 是否空闲。

住院登记卡: 病历号, 姓名, 病床号, 支付方式, 入住日期, ...。

计费文件: 病床号, 服务项目, 费用, 日期。

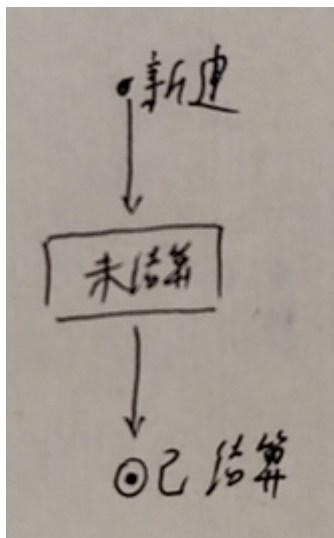
请画出病人住院与结帐业务的分层数据流图 (15 分)。

采用面向对象方法, 请你给出“计费文件”类的可能操作 (只要给出操作名) (5 分), 并画出“计费文件”的状态图 (10 分)。

答: 根据前面的流程描述, 理解为: 一个人在一次住院期间, 只有一个计费文件, 里面包含了所有相关的消费记录。那么[计费文件]类的操作有: get 病人, set 病人, get 欠费总额, add 消费记录

但是后面又提到计费文件的格式, 从格式上看, 把计费文件解释为[单次消费]。那么操作就是四个属性的 get 和 set。

计费文件状态图:



19.某制造企业的物料出入库管理的工作流程叙述如下:

1. 出库工作流程

- (1) 领料人提交领料单 (每一种物料有一张领料单);
- (2) 仓库保管员根据领料计划单检验该领料单是否有效;
- (3) 若经检验没有相应的领料计划, 则通知领料人该领料单无效;
- (4) 若领料单有效, 仓库保管员根据领料单上的物料代码核对是否有足够的库存;
- (5) 若没有足够的库存, 仓库保管员向领料人发缺货单;
- (6) 若有足够的库存, 仓库保管员在领料单上签字, 并登记出库单, 修改物料主文件中的现有库存数; 相应的物料出库, 物料清单交领料人。

2. 入库工作流程

- (1) 采购员提交入库申请单 (每一种物料有一张入库申请单);
- (2) 仓库保管员根据采购计划单验收入库申请单;
- (3) 若验收发现没有相应的采购计划, 则仓库保管员向采购员发无效申请单;
- (4) 若验收合格, 则仓库保管员向检验员申请物料检验; 检验员根据检验结果填写物料检验单;
- (5) 如果物料或供货方不合格, 则向采购员发出退货单;
- (6) 如果检验合格, 则仓库保管员登记入库单, 修改物料主文件中的现有库存数, 相应物料入库。

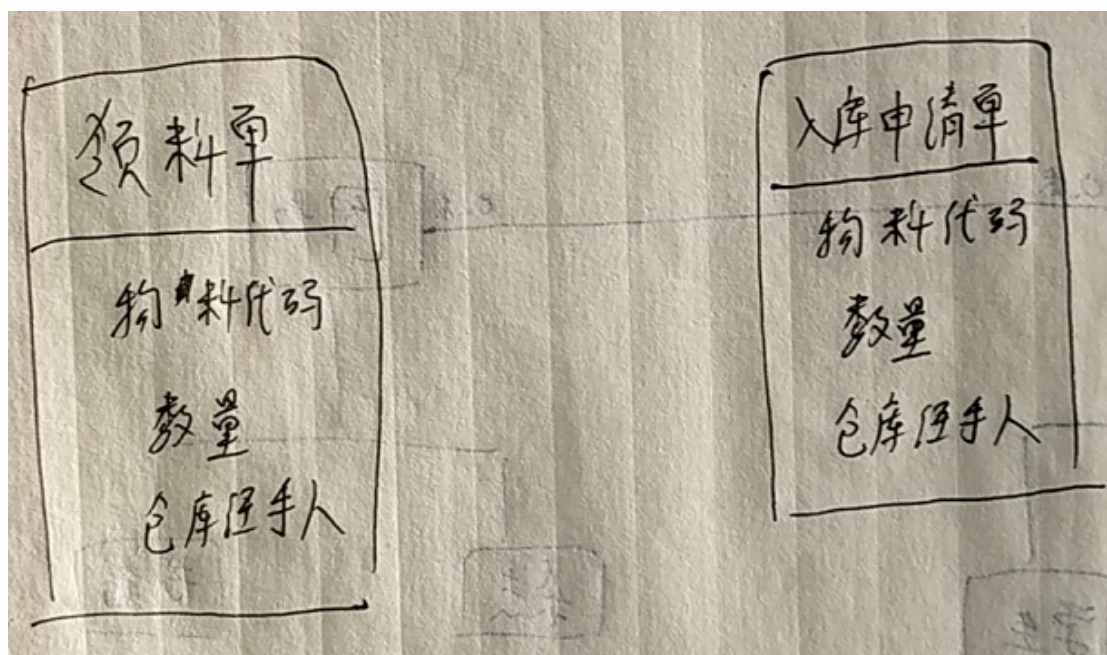
要求:

- (1) 画出上述业务分层的数据流程图。(15 分)

(2) 如果采用面向对象的方法, 画出 USE/CASE 图 (10 分), 其中“领料单”和“入库申请单”可以定义为两个类, 请给出这两个类至少应具有的属性。(5 分)

答:

两个类的属性:



20.某大学校园一卡通管理系统包括以下功能：

(1) 办理新卡，新入校的教师、学生或丢失卡者可提交院系盖章的证明申请办理新卡，系统根据人事处和学生处提供的名单核实办理，同时更新持卡基本信息文件。

(2) 存款，师生可向卡上存现金或通过指定账户向卡上划拨款，并修改明细账文件和账户文件。

(3) 消费，可在校园内用一卡通消费，每次消费记入明细帐文件。

(4) 查询对账，师生可查询每次存入或消费时的时间和金额。

(5) 挂失，师生可根据证件提出挂失申请，挂失后该卡不能消费

要求：

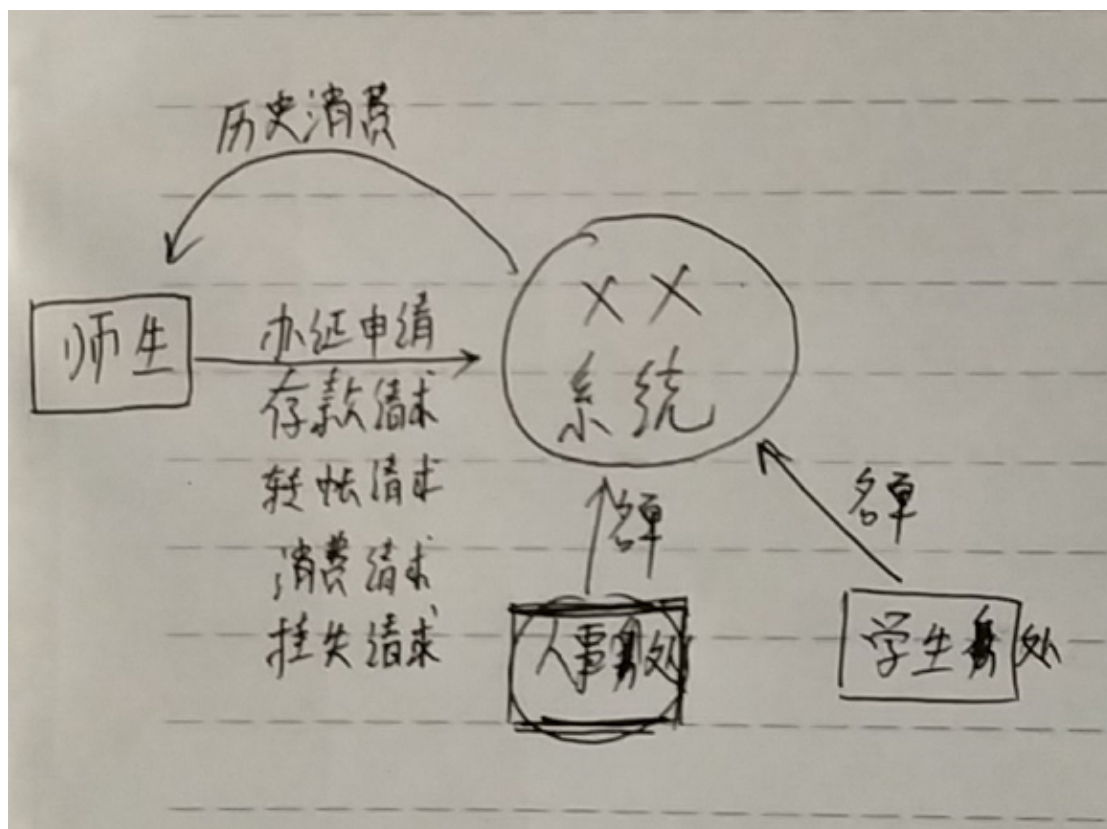
(1) 请根据所述分析业务，并画出数据流图。要求功能完整，逻辑合理（15分）；

(2) 给出顶层数据流图的数据字典（5分）；

(3) 画出 USE/CASE 图（10分）。

答：

(1) 顶层数据流图：



(2)

数据字典:

办证申请=身份证号码

存款请求=身份证号码+金额

转账请求=出钱身份证号码+收钱身份证号码+金额

消费请求=身份证号码+金额+说明

挂失请求=身份证号码

名单={身份证号码}

历史消费={单次历史消费}

单次历史消费=身份证号码+时间+金额+说明

说明: 这 20 题应用题, 好多没有答案, 好多答案错误(主观判断), 所以我提供了答案. 有些题没有给答案, 因为类似题型已经在前面出现过.

有些答案给的是简化版, 严格来说是会扣分的. 但是如果你能搞出完整版来, 你也没必要看我这份了. 完整版需要更多的理解, 会需要多投入很多时间, 收到的效益很低, 所以没必要.

孙超

21.共享单车 APP 作为日常出行租用自行车的软件, 旨在给人们提供一个优雅、方便、时尚的方式, 解决了最后一公里。共享单车 APP 的功能如下:

- 1.实时车辆地图: 车辆实时定位, 导航引导到达。
- 2.预约功能: 提前预约周边单车, 掌控行程安排。
- 3.一键扫码解锁: 扫码车身二维码, 即可解锁用车。
- 4.随时随地轻松还车: 无固定车桩, 在任意公共停车区域即可停车。

5.在线支付：提供多种电子支付方式。

要求用结构化分析方法（SA）建模：

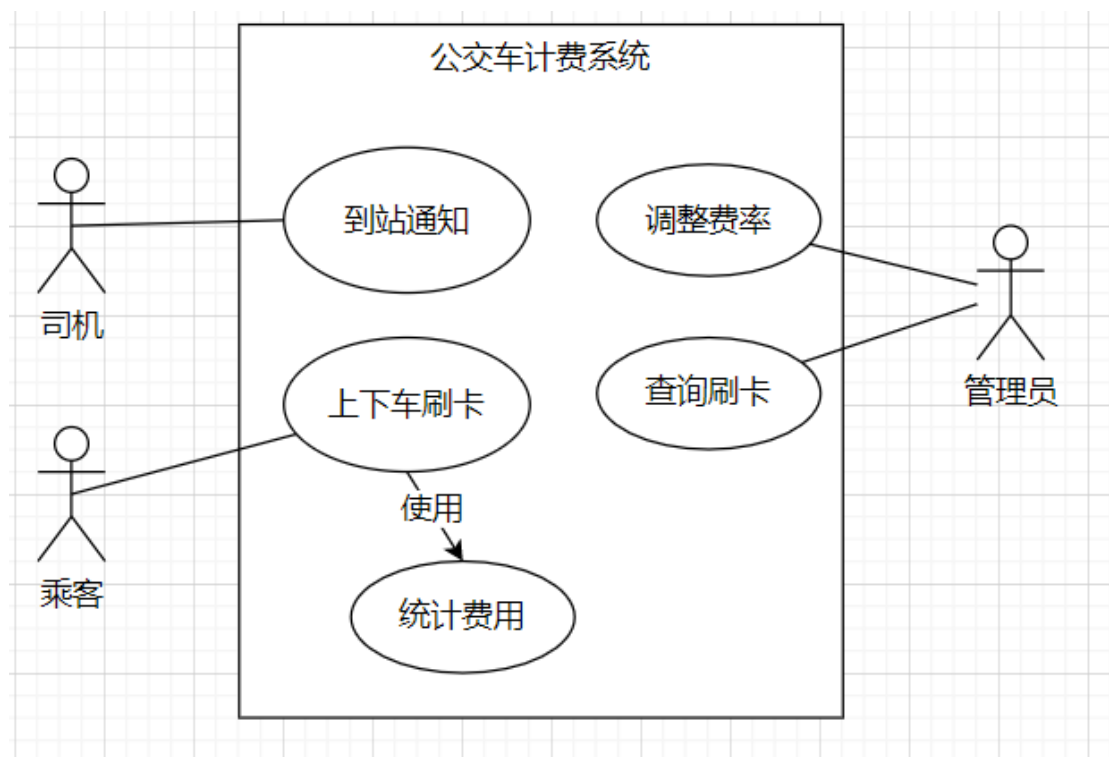
- 1.（15分）画出系统的顶层（0层）和第一层数据流图。
- 2.（15分）列出主要的数据字典（DD）项，包括源点和汇点，主要数据流、数据文件。

22、一个公交车计费系统，司机到站点击到站按钮，通知系统到站，乘客上车刷卡，下车刷卡，乘务员查询乘客是否刷卡，统计费用，管理员调整费率，计费方式

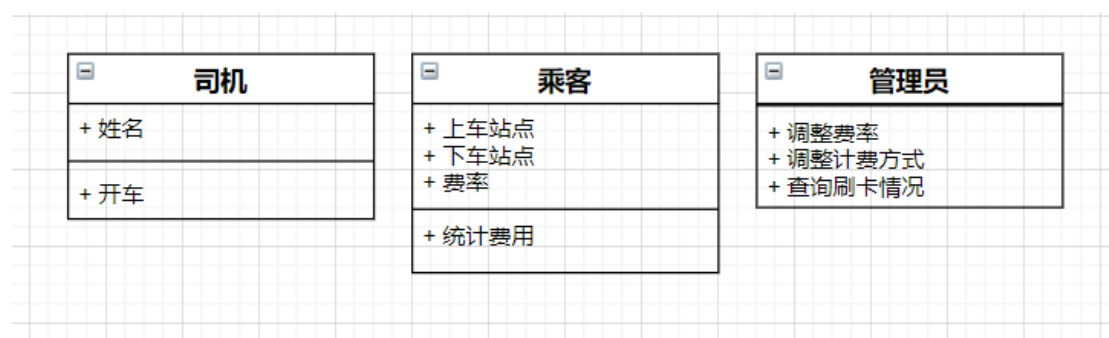
- (1) 画用例图
- (2) 画类图
- (3) 画时序图

答：

- (1) 用例图

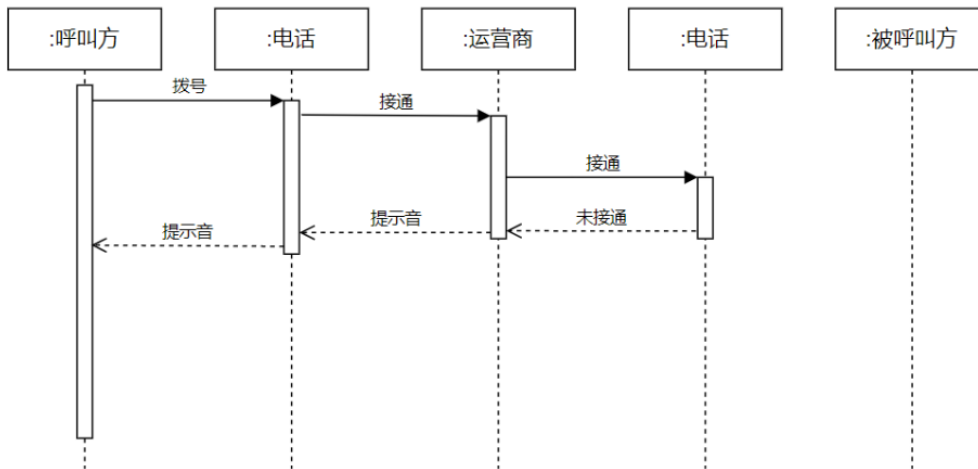
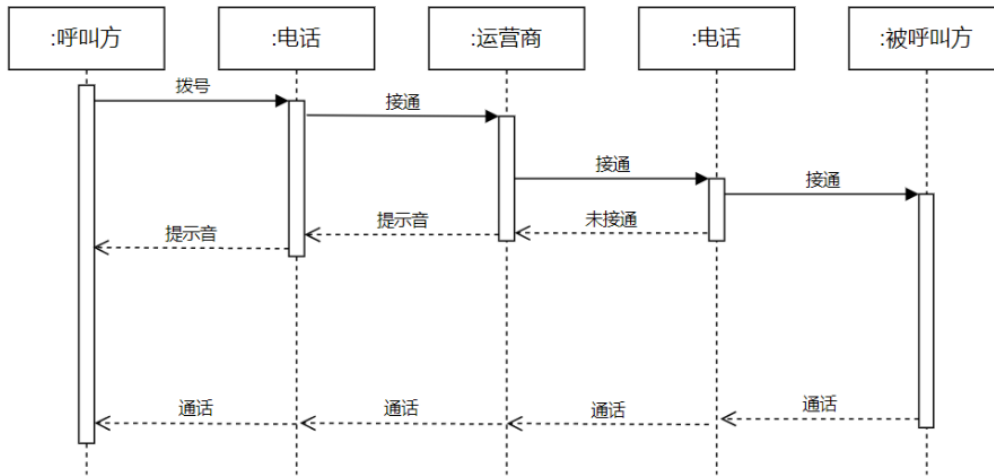


- (2) 类图：



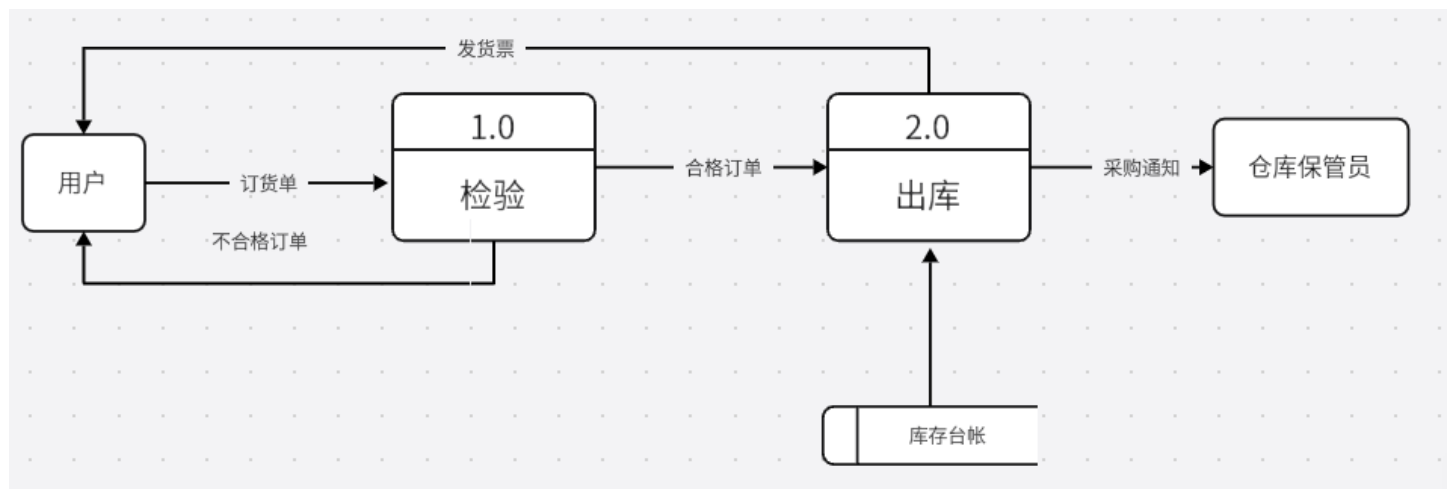
- (3) 时序图（顺序图）

23、A 和 B 打电话-时序图（顺序图） ①打通电话 ②未打通电话



24、(练习) 根据以下内容画出数据流程图

(1) 请根据以下销售过程画出数据流程图:用户将定货单交给某企业的业务经理, 经检验后, 对不合格的订单要由用户重填:合格的订单交仓库保管员做出库处理, 即:查阅库存台帐, 如果有货则向用户开票发货, 如缺货, 则通知采购员采购。



(2) 请根据以下定货过程画出数据流程图采购员从仓库收到缺货通知单后立即进行定货处理，即，查阅订货合同单，若已定货，则向供货单位发出催货单，否则，填写定货单送供货单位。供货单位发运货物后，立即向采购员发出取货通知。

